

算法竞赛模板

GNU++17 修订打印版

完整模板 · 使用说明 · 页码目录

按原资料全部 163 个条目复核，正文包含 104 份代码模板。

代码使用空格、四空格缩进与简短常用命名；无 `bit_width / optional`。

困难模板配可运行例子，题意相关空框架改为条件与公式速查。

版本日期：2026-10-02

先读下一页的下标、区间和依赖约定。

比赛前与使用时

页码：本册全部页面连续编号，包含封面和目录；目录页码就是阅读器/打印纸的实际页码。电子版目录与书签可点击。

编译：GNU++17, `#include <bits/stdc++.h>`, `using namespace std`, `using i64 = long long`。结构化绑定和普通 `lambda` 可以使用；不依赖 C++20。打印主体省略重复头文件，PBDS 保留专用 `include`。

下标：数组/普通图多为 1-based；Dinic、MCF、BoundFlow 的顶点为 0-based；XOR MST、三维偏序、整体二分答案编号为 0-based。模板旁说明优先，不能凭习惯混用。

区间：多数接口为闭区间 $[l, r]$ ；LCA 的 `tin/tout`、HLD 的 `in/out` 是半开子树区间，DSU on Tree 则是闭区间。接线段树时按说明转换。

类型与范围：int 存点/下标，i64 存常见权值，乘法大于 64 位先转 i128。有限距离和、流量、费用、容量总和按各模板说明控制；INF 是哨兵，不能当普通数直接加减。

复制：code/ 中每份 hpp 是独立完整模板；不同章节可能复用同名类型，按题目挑选并合并必要定义。正文为省纸只打印一次基础依赖，明确写出其编号；调用例子还需该条模板主体。

使用难板：先看前提与接口，再看完整调用例子。先运行例子确认理解，再改 Info/Tag/Policy、函数或题意转移；更改后用大数据暴力对照。

验证范围：104 份独立编译，全部有边界/固定种子对照检查；40 个示例（含 3 个 Min_25 改法）及 20 万点链检查通过。测试不等于形式化证明，浮点精度、随机算法耗时和题目范围仍以注明条件为准。

文件：原 157 页 PDF 保留；source/算法竞赛模板_完整源稿.md 是可编辑全文；code/ 是模板，examples/ 是调用例子，tests/revised/ 与复核记录保存验证依据。

目录

正文编号用于交叉引用；“速查”只保留有用的条件与公式，不提供题意未定义的空代码。

1. 数据结构	7
1.1 并查集 DSU	7
1.2 可撤销并查集	7
1.3 带权并查集	8
1.4 跳跃并查集	9
1.5 树状数组 BIT	10
1.6 稀疏表 ST	10
1.7 线段树 Seg	11
1.8 懒标记线段树 LazySeg	13
1.9 可持久化线段树 PST	16
1.10 动态开点线段树	19
1.11 Segment Tree Beats	20
1.12 FHQ Treap 多重集	22
1.13 隐式 Treap 序列	24
1.14 PBDS 可重多重集	27
1.15 李超线段树	28
1.16 凸包优化 DP	29
1.17 笛卡尔树	30
1.18 Link-Cut Tree 动态森林	31
1.19 按类别维护 Top-K	33
2. 图论与建模	35
2.1 Dijkstra	35
2.2 0-1 BFS	35
2.3 Floyd	36
2.4 拓扑排序	36
2.5 Kruskal 最小生成森林	37
2.6 二分图度数序列构造	38
2.7 增量 APSP	38
2.8 强连通分量 SCC	39
2.9 2-SAT	40
2.10 桥与割点 Lowlink	42
2.11 边双连通分量	43
2.12 点双连通分量与圆方树	43
2.13 欧拉通路	45
2.14 二分图最大匹配 Hopcroft-Karp	46
2.15 二分图最大权匹配 Hungarian	48
2.16 最大流 Dinic	50
2.17 最小费用流 MCF	52
2.18 上下界网络流	54
2.19 差分约束	56

2.20 函数图	58
2.21 线段树优化建图	59
2.22 增量最大流 [速查]	60
2.23 严格次小生成树	60
2.24 隐式 XOR 完全图 MST	62
2.25 最大权闭合子图	64
2.26 Boruvka 最小生成森林	64
3. 树上算法	66
3.1 树的直径	66
3.2 树的重心	66
3.3 Kruskal 重构树	67
3.4 倍增 LCA	69
3.5 重链剖分 HLD	70
3.6 树上差分	73
3.7 虚树	73
3.8 带信息倍增	74
3.9 DSU on Tree	75
3.10 点分治与点分树	77
3.11 换根 DP 速查 [速查]	78
3.12 Small-to-Large [速查]	78
3.13 长链剖分 [速查]	78
4. DP: 实用代码与条件速查	79
4.1 LIS / 最长不下降子序列	79
4.2 多重背包: 单调队列	79
4.3 数位 DP 速查 [速查]	79
4.4 轮廓线 DP [速查]	80
4.5 中位数阈值变换 [速查]	80
4.6 排列 LCS 转 LIS [速查]	80
4.7 背包循环方向 [速查]	80
4.8 单调队列优化 DP [速查]	80
4.9 区间 DP [速查]	80
4.10 树上背包 [速查]	80
4.11 SOS 子集和与反演 [速查]	80
4.12 分治 DP 优化 [速查]	80
4.13 状态压缩与 Gray code [速查]	80
4.14 Knuth 优化 [速查]	81
4.15 WQS / Aliens trick [速查]	81
4.16 bitset 优化 [速查]	81
5. 数论与组合	82
5.1 模整数 MInt	82
5.2 exgcd / 扩展 CRT	83
5.3 阶乘组合数	84
5.4 Lucas	85

5.5 连续点拉格朗日插值	85
5.6 线性筛	87
5.7 φ / μ 前缀与反演	87
5.8 Miller-Rabin 64 位素性	88
5.9 Pollard-Rho 分解	89
5.10 整除分块	91
5.11 类欧几里得 floor sum	91
5.12 扩展 BSGS	92
5.13 模矩阵与快速幂	94
5.14 杜教筛: φ 与 μ	95
5.15 Min_25 积性函数求和	96
5.16 半环矩阵 [速查]	99
5.17 扩展欧拉降幂 [速查]	100
6. 多项式	101
6.1 FFT 整数卷积	101
6.2 NTT 模 998244353	102
6.3 FWT: OR / AND / XOR	104
6.4 形式幂级数 inv / ln / exp	106
7. 线性代数	110
7.1 Gauss-Jordan 消元	110
7.2 64 位异或线性基	112
7.3 GF(2) bitset 方程组	114
8. 字符串	116
8.1 KMP	116
8.2 Z / 扩展 KMP	116
8.3 64 位滚动哈希	117
8.4 子序列自动机	118
8.5 Trie 多重字符串集	118
8.6 AC 自动机	120
8.7 后缀自动机 SAM	121
8.8 后缀数组 SA	123
8.9 最小循环表示	125
8.10 Manacher	125
8.11 回文自动机 PAM	126
9. 计算几何	129
9.1 点与基础几何运算	129
9.2 直线	130
9.3 线段交与距离	131
9.4 凸包 Andrew	133
9.5 点在简单多边形内	133
9.6 最近点对	134
9.7 旋转卡壳: 凸包直径	135
9.8 外接圆与圆交	135

9.9 半平面交	136
9.10 Minkowski 和	138
9.11 矩形面积并	140
10. 离线算法与技巧	142
10.1 二分答案边界 [速查]	142
10.2 普通莫队：不同数计数	142
10.3 CDQ 三维偏序	143
10.4 整体二分：动态区间第k小	144
10.5 时间线段树 + 回滚 DSU	146
10.6 根号分块 [速查]	147
10.7 图上重轻点 [速查]	148
10.8 时间分块 [速查]	148
10.9 倒序离线 [速查]	148
10.10 双指针 [速查]	148
10.11 折半搜索 [速查]	148
10.12 坐标压缩 [速查]	148
10.13 扫描线 [速查]	148
10.14 Log Trick [速查]	148
10.15 前缀与差分 [速查]	148
10.16 贡献法 [速查]	148
10.17 有限群状态 [速查]	149
10.18 容斥 [速查]	149
10.19 反悔贪心 [速查]	149
10.20 上次出现位置 [速查]	149
10.21 二进制对齐区间 [速查]	149
10.22 相邻交换排序 [速查]	149
10.23 区间与点匹配 [速查]	149
10.24 可行性判定构造 [速查]	149
10.25 加权中位数 [速查]	149
10.26 固定半径空间哈希 [速查]	149
10.27 L1 坐标变换 [速查]	150
10.28 预留坐标 [速查]	150
10.29 区间取模剪枝 [速查]	150
11. 公式与博弈	151
11.1 常用恒等式 [速查]	151
11.2 SG / Nim [速查]	151

1. 数据结构

1.1 并查集 DSU

点为 $1..n$ 。merge(u,v) 返回是否真的合并；same、size 查询。路径压缩 + 按大小合并，均摊 $O(\alpha(n))$ 。

1.1 并查集 DSU · 主体

```
using namespace std;
struct DSU {
    vector<int> fa, sz;
    int cnt;
    DSU(int n = 0) : fa(n + 1), sz(n + 1, 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {
        return x == fa[x] ? x : fa[x] = find(fa[x]);
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) {
            return false;
        }
        if(sz[x] < sz[y]) {
            swap(x, y);
        }
        fa[y] = x;
        sz[x] += sz[y];
        cnt--;
        return true;
    }
    int size(int x) {
        return sz[find(x)];
    }
    int groups() const {
        return cnt;
    }
};
```

1.2 可撤销并查集

snapshot() 记录栈大小，rollback(s) 回到该状态。不能做路径压缩；按大小合并，单次 $O(\log n)$ 。一次失败的合并也必须按实现的栈约定处理，直接使用快照回滚最稳妥。

1.2 可撤销并查集 · 主体

```
using namespace std;
struct UndoDSU {
    vector<int> fa, sz;
    vector<array<int, 3>> his;
    int cnt;
    UndoDSU(int n = 0) : fa(n + 1), sz(n + 1, 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) const {
        while(x != fa[x]) {
            x = fa[x];
        }
        return x;
    }
    bool same(int x, int y) const {
        return find(x) == find(y);
    }
};
```

1.2 可撤销并查集 · 主体 · 续

```

    }
    int size(int x) const {
        return sz[find(x)];
    }
    int groups() const {
        return cnt;
    }
    int snapshot() const {
        return int(his.size());
    }
}
bool merge(int x, int y) {
    x = find(x);
    y = find(y);
    if(x == y) {
        return false;
    }
    if(sz[x] < sz[y]) {
        swap(x, y);
    }
    his.push_back({x, y, sz[x]});
    fa[y] = x;
    sz[x] += sz[y];
    cnt--;
    return true;
}
void rollback(int t) {
    while(int(his.size()) > t) {
        auto [x, y, oldSize] = his.back();
        his.pop_back();
        fa[y] = y;
        sz[x] = oldSize;
        cnt++;
    }
}
};

```

1.3 带权并查集

维护势能差； $\text{merge}(u,v,w)$ 表示 $\text{value}[v] - \text{value}[u] = w$ 。返回 `false` 表示与已有关系矛盾。跨连通块不能直接读取差值；权值及沿链累加需在 `i64` 内。

1.3 带权并查集 · 主体

```

using namespace std;
using i64 = long long;
template <class T = i64> struct WeightDSU {
    vector<int> fa, sz;
    vector<T> d;
    int cnt;
    WeightDSU(int n = 0) : fa(n + 1), sz(n + 1, 1), d(n + 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {
        if(x == fa[x]) {
            return x;
        }
        int y = fa[x];
        fa[x] = find(y);
        d[x] += d[y];
        return fa[x];
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
    int groups() const {
        return cnt;
    }
};

```

1.3 带权并查集 · 主体 · 续

```

    }
    T diff(int x, int y) {
        int a = find(x);
        int b = find(y);
        assert(a == b);
        return d[y] - d[x];
    }
    // value[y] - value[x] = w
    bool merge(int x, int y, T w) {
        int a = find(x);
        int b = find(y);
        if(a == b) {
            return d[y] - d[x] == w;
        }
        T t = w + d[x] - d[y];
        if(sz[a] >= sz[b]) {
            fa[b] = a;
            d[b] = t;
            sz[a] += sz[b];
        } else {
            fa[a] = b;
            d[a] = -t;
            sz[b] += sz[a];
        }
        cnt--;
        return true;
    }
};

```

1.4 跳跃并查集

erase(x) 删除位置, find(x) 返回不小于 x 的第一个未删位置; n+1 是哨兵。只支持永久删除, 不支持插回。查找用循环压缩, 连续删除不会形成递归栈风险。

1.4 跳跃并查集 · 主体

```

using namespace std;
struct NextDSU {
    vector<int> fa;
    NextDSU(int n = 0) : fa(n + 2) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {
        int r = x;
        while(r != fa[r]) {
            r = fa[r];
        }
        while(x != r) {
            int y = fa[x];
            fa[x] = r;
            x = y;
        }
        return r;
    }
    int next(int x) {
        return find(x);
    }
    void erase(int x) {
        fa[x] = find(x + 1);
    }
};

```

1.5 树状数组 BIT

1-based; add(x,v)、query(x) 前缀和、query(l,r) 闭区间和。select(k) 找前缀和首次达到 k 的位置：所有频次非负， $1 \leq k \leq$ 总和；无答案会返回 n+1。O(log n)。

1.5 树状数组 BIT · 主体

```
using namespace std;
using i64 = long long;
template <class T = i64> struct BIT {
    int n;
    vector<T> tr;
    BIT(int n_ = 0) : n(n_), tr(n_ + 1) {}
    void add(int x, T v) {
        for(; x <= n; x += x & -x) {
            tr[x] += v;
        }
    }
    T query(int x) const {
        T res{};
        for(; x > 0; x -= x & -x) {
            res += tr[x];
        }
        return res;
    }
    T query(int l, int r) const {
        return query(r) - query(l - 1);
    }
    int select(T k) const {
        int x = 0;
        T cur{};
        int step = (n ? 1 << (31 - __builtin_clz((unsigned)n)) : 0);
        for(; step; step >>= 1) {
            if(x + step <= n && cur + tr[x + step] < k) {
                x += step;
                cur += tr[x];
            }
        }
        return x + 1;
    }
};
```

1.6 稀疏表 ST

输入数组含下标 0 占位，query(l,r) 为闭区间。Op 满足结合律与幂等性，例如 min/max/gcd；不能用 sum。预处理 O(n log n)，查询 O(1)。重叠两段按原序合并，不强制交换律。

1.6 稀疏表 ST · 主体

```
using namespace std;
template <class T, class F> struct ST {
    int n;
    vector<vector<T>> st;
    F op;
    ST(const vector<T>& a, F f) : n(int(a.size()) - 1), op(move(f)) {
        int m = int((n ? 32 - __builtin_clz((unsigned)n) : 0));
        st.assign(m, vector<T>(n + 1));
        if(n == 0) {
            return;
        }
        st[0] = a;
        for(int k = 1; k < m; k++) {
            for(int i = 1; i + (1 << k) - 1 <= n; i++) {
                st[k][i] = this->op(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }
};
```

1.6 稀疏表 ST · 主体 · 续

```

    }
    T query(int l, int r) const {
        int k = int((32 - __builtin_clz((unsigned)(r - l + 1)))) - 1;
        return op(st[k][l], st[k][r - (1 << k) + 1]);
    }
};

```

1.7 线段树 Seg

输入 1-based; modify 是单点赋值; query(l,r) 为闭区间。Info() 是单位元, operator+ 按左到右合并。first/last 的判定须满足区间整体可排除性, 不能把任意判断当作二分条件。

1.7 线段树 Seg · 主体

```

using namespace std;
template <class Info> struct Seg {
    int n;
    vector<Info> tr;
    Seg(int n_ = 0) : n(n_), tr(2 * n_) {}
    Seg(const vector<Info>& a) : Seg(int(a.size()) - 1) {
        for(int i = 1; i <= n; i++) {
            tr[n + i - 1] = a[i];
        }
        for(int p = n - 1; p > 0; p--) {
            pull(p);
        }
    }
    void pull(int p) {
        tr[p] = tr[p << 1] + tr[p << 1 | 1];
    }
    void modify(int p, const Info& v) {
        tr[p += n - 1] = v;
        while(p > 1) {
            p >>= 1;
            pull(p);
        }
    }
    Info query(int l, int r) const {
        Info a, b;
        for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {
            if(l & 1) {
                a = a + tr[l++];
            }
            if(r & 1) {
                b = tr[--r] + b;
            }
        }
        return a + b;
    }
}
template <class F> int first(int l, int r, F pred) const {
    int a[64], b[64], ca = 0, cb = 0;
    for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {
        if(l & 1) {
            a[ca++] = l++;
        }
        if(r & 1) {
            b[cb++] = --r;
        }
    }
    Info cur;
    auto work = [&](int p) {
        Info nxt = cur + tr[p];
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
    };
}

```

1.7 线段树 Seg · 主体 · 续

```

    }
    while(p < n) {
        p <<= 1;
        nxt = cur + tr[p];
        if(!pred(nxt)) {
            cur = nxt;
            p |= 1;
        }
    }
    return p - n + 1;
};

for(int i = 0; i < ca; i++) {
    int p = work(a[i]);
    if(p != -1) {
        return p;
    }
}

for(int i = cb - 1; i >= 0; i--) {
    int p = work(b[i]);
    if(p != -1) {
        return p;
    }
}

return -1;
}

template <class F> int last(int l, int r, F pred) const {
    int a[64], b[64], ca = 0, cb = 0;
    for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {
        if(l & 1) {
            a[ca++] = l++;
        }
        if(r & 1) {
            b[cb++] = --r;
        }
    }
    Info cur;
    auto work = [&](int p) {
        Info nxt = tr[p] + cur;
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        while(p < n) {
            p = p << 1 | 1;
            nxt = tr[p] + cur;
            if(!pred(nxt)) {
                cur = nxt;
                p ^= 1;
            }
        }
        return p - n + 1;
    };

    for(int i = 0; i < cb; i++) {
        int p = work(b[i]);
        if(p != -1) {
            return p;
        }
    }

    for(int i = ca - 1; i >= 0; i--) {
        int p = work(a[i]);
        if(p != -1) {
            return p;
        }
    }

    return -1;
}

};

```

Info: 区间和与前缀找第 k 个

1.7 · Info: 区间和与前缀找第 k 个

```

struct Info {
    long long sum = 0;
    Info operator+(Info b) const {
        return {sum + b.sum};
    }
};

int main() {
    vector<Info> a = {{0}, {2}, {0}, {3}};
    Seg<Info> s(a);
    assert(s.query(1, 3).sum == 5);
    int p = s.first(1, 3, [](Info x) { return x.sum >= 3; });
    assert(p == 3); // 频次必须非负, 判定才单调。
}

```

1.8 懒标记线段树 LazySeg

Info.apply(Tag) 更新区间信息; Tag.apply(Tag) 把新操作接在旧操作之后。所有区间闭合, modify 为单点赋值, apply 为区间操作。合并信息与标记时都要保持时间顺序。下面的完整例子支持区间仿射变换和区间求和。

叶子 len=1, 单位元 len=0; 初始化 LazySeg(n) 的默认 Info 没有实际区间长度, 不能直接用于此 Info。输入与 Tag 的模值先归一化。

1.8 懒标记线段树 LazySeg · 主体

```

using namespace std;
template <class Info, class Tag> struct LazySeg {
    int n = 0;
    vector<Info> tr;
    vector<Tag> lz;
    LazySeg(int n_ = 0, const Info& v = Info()) {
        init(n_, v);
    }
    template <class T> LazySeg(const vector<T>& a) {
        init(a);
    }
    void init(int n_, const Info& v = Info()) {
        init(vector<Info>(n_ + 1, v));
    }
    template <class T> void init(const vector<T>& a) {
        n = int(a.size()) - 1;
        tr.assign(4 * max(1, n), Info());
        lz.assign(4 * max(1, n), Tag());
        if(n) {
            build(1, 1, n, a);
        }
    }
    template <class T> void build(int p, int l, int r, const vector<T>& a) {
        if(l == r) {
            tr[p] = a[l];
            return;
        }
        int m = (l + r) >> 1;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m + 1, r, a);
        pull(p);
    }
    void pull(int p) {
        tr[p] = tr[p << 1] + tr[p << 1 | 1];
    }
    void apply(int p, const Tag& v) {
        tr[p].apply(v);
        lz[p].apply(v);
    }
    void push(int p) {

```

1.8 懒标记线段树 LazySeg · 主体 · 续

```

    apply(p << 1, lz[p]);
    apply(p << 1 | 1, lz[p]);
    lz[p] = Tag();
}
void modify(int p, int l, int r, int x, const Info& v) {
    if(l == r) {
        tr[p] = v;
        return;
    }
    push(p);
    int m = (l + r) >> 1;
    if(x <= m) {
        modify(p << 1, l, m, x, v);
    } else {
        modify(p << 1 | 1, m + 1, r, x, v);
    }
    pull(p);
}
void modify(int x, const Info& v) {
    modify(1, 1, n, x, v);
}
Info query(int p, int l, int r, int x, int y) {
    if(x <= l && r <= y) {
        return tr[p];
    }
    push(p);
    int m = (l + r) >> 1;
    if(y <= m) {
        return query(p << 1, l, m, x, y);
    }
    if(x > m) {
        return query(p << 1 | 1, m + 1, r, x, y);
    }
    return query(p << 1, l, m, x, y) + query(p << 1 | 1, m + 1, r, x, y);
}
Info query(int l, int r) {
    return query(1, 1, n, l, r);
}
void apply(int p, int l, int r, int x, int y, const Tag& v) {
    if(x <= l && r <= y) {
        apply(p, v);
        return;
    }
    push(p);
    int m = (l + r) >> 1;
    if(x <= m) {
        apply(p << 1, l, m, x, y, v);
    }
    if(y > m) {
        apply(p << 1 | 1, m + 1, r, x, y, v);
    }
    pull(p);
}
void apply(int l, int r, const Tag& v) {
    apply(1, 1, n, l, r, v);
}
template <class F> int first(int p, int l, int r, int x, int y, Info& cur, F& pred) {
    if(r < x || y < l) {
        return -1;
    }
    if(x <= l && r <= y) {
        Info nxt = cur + tr[p];
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
    }
    if(l == r) {

```

1.8 懒标记线段树 LazySeg · 主体 · 续

```

        return l;
    }
}
push(p);
int m = (l + r) >> 1;
int res = first(p << 1, l, m, x, y, cur, pred);
if(res == -1) {
    res = first(p << 1 | 1, m + 1, r, x, y, cur, pred);
}
return res;
}
template <class F> int first(int l, int r, F pred) {
    Info cur;
    return first(1, 1, n, l, r, cur, pred);
}
template <class F> int last(int p, int l, int r, int x, int y, Info& cur, F& pred) {
    if(r < x || y < l) {
        return -1;
    }
    if(x <= l && r <= y) {
        Info nxt = tr[p] + cur;
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) {
            return l;
        }
    }
    push(p);
    int m = (l + r) >> 1;
    int res = last(p << 1 | 1, m + 1, r, x, y, cur, pred);
    if(res == -1) {
        res = last(p << 1, l, m, x, y, cur, pred);
    }
    return res;
}
template <class F> int last(int l, int r, F pred) {
    Info cur;
    return last(1, 1, n, l, r, cur, pred);
}
};

```

Info / Tag: 区间仿射与区间和

1.8 · Info / Tag: 区间仿射与区间和

```

const int P = 998244353;
struct Tag {
    long long mul = 1, add = 0;
    void apply(Tag t) {
        mul = mul * t.mul % P;
        add = (add * t.mul + t.add) % P;
    }
};
struct Info {
    long long sum = 0;
    int len = 0;
    void apply(Tag t) {
        sum = (sum * t.mul + t.add * len) % P;
    }
    Info operator+(Info b) const {
        return {(sum + b.sum) % P, len + b.len};
    }
};
int main() {
    vector<Info> a = {{0, 0}, {1, 1}, {2, 1}, {3, 1}};
    LazySeg<Info, Tag> s(a);
    s.apply(1, 3, {2, 1}); // x -> 2x+1, 得到 3,5,7。
    s.apply(2, 3, {0, 4}); // 赋值为4; 加c用 {1,c}。
    assert(s.query(1, 3).sum == 11);
}

```

1.9 可持久化线段树 PST

modify(old,x,v) 生成新根，旧根不变；叶子调用 Info.apply(v)，赋值还是增量由此函数决定；频率例子用 +=。版本根单独保存。查询两个前缀版本的差得到区间频率；first 的谓词基于累计 Info，需单调。单次新建 $O(\log n)$ 节点。

输入位置和值域下标是两套编号，rt[i] 是前 i 个元素形成的值域频率版本。减根的顺序为 rt[r]-rt[l-1]。

1.9 可持久化线段树 PST · 主体

```

using namespace std;
template <class Info> struct PST {
    struct Node {
        int ls = 0, rs = 0;
        Info val;
    };
    int n;
    vector<Node> tr;
    PST(int n_ = 0, int q = 0) : n(n_), tr(1) {
        if(n && q) {
            tr.reserve(size_t(q) * ((n ? 32 - __builtin_clz((unsigned)n) : 0) + 1) + 1);
        }
    }
    int clone(int p) {
        tr.push_back(tr[p]);
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].val = tr[tr[p].ls].val + tr[tr[p].rs].val;
    }
    int modify(int p, int l, int r, int x, const Info& v) {
        int u = clone(p);
        if(l == r) {
            tr[u].val.apply(v);
            return u;
        }
        int m = (l + r) >> 1;
        if(x <= m) {
            tr[u].ls = modify(tr[p].ls, l, m, x, v);
        }
    }
};

```

1.9 可持久化线段树 PST · 主体 · 续

```

    } else {
        tr[u].rs = modify(tr[p].rs, m + 1, r, x, v);
    }
    pull(u);
    return u;
}
int modify(int root, int x, const Info& v) {
    return modify(root, 1, n, x, v);
}
Info query(int p, int l, int r, int x, int y) const {
    if(!p) {
        return Info();
    }
    if(x <= l && r <= y) {
        return tr[p].val;
    }
    int m = (l + r) >> 1;
    if(y <= m) {
        return query(tr[p].ls, l, m, x, y);
    }
    if(x > m) {
        return query(tr[p].rs, m + 1, r, x, y);
    }
    return query(tr[p].ls, l, m, x, y) + query(tr[p].rs, m + 1, r, x, y);
}
Info query(int root, int l, int r) const {
    return query(root, 1, n, l, r);
}
Info query(int a, int b, int l, int r, int x, int y) const {
    if(x <= l && r <= y) {
        return tr[b].val - tr[a].val;
    }
    int m = (l + r) >> 1;
    if(y <= m) {
        return query(tr[a].ls, tr[b].ls, l, m, x, y);
    }
    if(x > m) {
        return query(tr[a].rs, tr[b].rs, m + 1, r, x, y);
    }
    return query(tr[a].ls, tr[b].ls, l, m, x, y) +
        query(tr[a].rs, tr[b].rs, m + 1, r, x, y);
}
Info query(int a, int b, int l, int r) const {
    return query(a, b, 1, n, l, r);
}
template <class F>
int first(int a, int b, int l, int r, int x, int y, Info& cur, F& pred) const {
    if(r < x || y < l) {
        return -1;
    }
    if(x <= l && r <= y) {
        Info nxt = cur + (tr[b].val - tr[a].val);
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) {
            return l;
        }
    }
    int m = (l + r) >> 1;
    int p = first(tr[a].ls, tr[b].ls, l, m, x, y, cur, pred);
    if(p == -1) {
        p = first(tr[a].rs, tr[b].rs, m + 1, r, x, y, cur, pred);
    }
    return p;
}

```

1.9 可持久化线段树 PST · 主体 · 续

```

template <class F> int first(int a, int b, int l, int r, F pred) const {
    Info cur;
    return first(a, b, 1, n, l, r, cur, pred);
}
template <class F>
int last(int a, int b, int l, int r, int x, int y, Info& cur, F& pred) const {
    if(r < x || y < l) {
        return -1;
    }
    if(x <= l && r <= y) {
        Info nxt = (tr[b].val - tr[a].val) + cur;
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) {
            return l;
        }
    }
    int m = (l + r) >> 1;
    int p = last(tr[a].rs, tr[b].rs, m + 1, r, x, y, cur, pred);
    if(p == -1) {
        p = last(tr[a].ls, tr[b].ls, l, m, x, y, cur, pred);
    }
    return p;
}
template <class F> int last(int a, int b, int l, int r, F pred) const {
    Info cur;
    return last(a, b, 1, n, l, r, cur, pred);
}
};

```

前缀版本：静态区间第 k 小

1.9 · 前缀版本：静态区间第 k 小

```

struct Info {
    int sum = 0;
    void apply(Info b) {
        sum += b.sum;
    }
    Info operator+(Info b) const {
        return {sum + b.sum};
    }
    Info operator-(Info b) const {
        return {sum - b.sum};
    }
};
int main() {
    vector<int> a = {0, 5, 1, 5, 3}, v(a.begin() + 1, a.end());
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    int n = a.size() - 1, m = v.size();
    PST<Info> s(m, n);
    vector<int> rt(n + 1);
    for(int i = 1; i <= n; i++) {
        int x = lower_bound(v.begin(), v.end(), a[i]) - v.begin() + 1;
        rt[i] = s.modify(rt[i - 1], x, {1});
    }
    int l = 2, r = 4, k = 2; // 闭区间; k需合法。
    int x = s.first(rt[l - 1], rt[r], 1, m, [&](Info t) { return t.sum >= k; });
    assert(v[x - 1] == 3);
}

```

1.10 动态开点线段树

$\text{DynSeg}<\text{Info}>(L,R)$, root 初始为 0; $\text{modify}(\text{root},x,v)$ 是点赋值, $\text{query}(\text{root},l,r)$ 为闭区间。空节点返回 $\text{Info}()$ 。节点用编号访问, 扩容后不能保存 vector 内引用; 内存 $O(q \log(R-L+1))$ 。

1.10 动态开点线段树 · 主体

```
using namespace std;
using i64 = long long;
template <class Info> struct DynSeg {
    struct Node {
        int ls = 0, rs = 0;
        Info val;
    };
    i64 L, R;
    vector<Node> tr;
    DynSeg(i64 l, i64 r) : L(l), R(r), tr(1) {}
    int newNode() {
        tr.push_back({});
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].val = tr[tr[p].ls].val + tr[tr[p].rs].val;
    }
    int modify(int p, i64 l, i64 r, i64 x, const Info& v) {
        if(!p) {
            p = newNode();
        }
        if(l == r) {
            tr[p].val = v;
            return p;
        }
        i64 m = l + (r - l) / 2;
        if(x <= m) {
            tr[p].ls = modify(tr[p].ls, l, m, x, v);
        } else {
            tr[p].rs = modify(tr[p].rs, m + 1, r, x, v);
        }
        pull(p);
        return p;
    }
    void modify(int& root, i64 x, const Info& v) {
        root = modify(root, L, R, x, v);
    }
    Info query(int p, i64 l, i64 r, i64 x, i64 y) const {
        if(!p) {
            return Info();
        }
        if(x <= l && r <= y) {
            return tr[p].val;
        }
        i64 m = l + (r - l) / 2;
        if(y <= m) {
            return query(tr[p].ls, l, m, x, y);
        }
        if(x > m) {
            return query(tr[p].rs, m + 1, r, x, y);
        }
        return query(tr[p].ls, l, m, x, y) + query(tr[p].rs, m + 1, r, x, y);
    }
    Info query(int root, i64 l, i64 r) const {
        return query(root, L, R, l, r);
    }
};
```

动态开点：点赋值

1.10 · 动态开点：点赋值

```

struct Info {
    long long sum = 0;
    Info operator+(Info b) const {
        return {sum + b.sum};
    }
};

int main() {
    DynSeg<Info> s(-1000000000, 1000000000);
    int rt = 0;
    s.modify(rt, -7, {3});
    s.modify(rt, 12, {5});
    s.modify(rt, 12, {1});
    assert(s.query(rt, -10, 20).sum == 4);
}

```

1.11 Segment Tree Beats

本版支持区间 `chmin`、`sum`、`maxi`；不包含区间加、`chmax`。维护最大值、严格次大值和最大值个数；仅在新上界高于次大值时整段修改，否则下推。均摊 $O((n+q) \log n)$ ，区间和必须在 `i64` 内。

1.11 Segment Tree Beats · 主体

```

using namespace std;
using i64 = long long;
struct Beats {
    static constexpr i64 negInf = numeric_limits<i64>::lowest();
    struct Node {
        i64 sum = 0;
        i64 mx = negInf, se = negInf;
        int cnt = 0;
    };
    int n;
    vector<Node> tr;
    Beats(const vector<i64>& a) : n(int(a.size()) - 1), tr(4 * n) {
        build(1, 1, n, a);
    }
    void build(int p, int l, int r, const vector<i64>& a) {
        if(l == r) {
            tr[p] = {a[l], a[l], negInf, 1};
            return;
        }
        int m = (l + r) >> 1;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m + 1, r, a);
        pull(p);
    }
    void pull(int p) {
        auto& a = tr[p << 1];
        auto& b = tr[p << 1 | 1];
        tr[p].sum = a.sum + b.sum;
        if(a.mx == b.mx) {
            tr[p].mx = a.mx;
            tr[p].se = max(a.se, b.se);
            tr[p].cnt = a.cnt + b.cnt;
        } else if(a.mx > b.mx) {
            tr[p].mx = a.mx;
            tr[p].se = max(a.se, b.mx);
            tr[p].cnt = a.cnt;
        } else {
            tr[p].mx = b.mx;
            tr[p].se = max(a.mx, b.se);
            tr[p].cnt = b.cnt;
        }
    }
}

```

1.11 Segment Tree Beats · 主体 · 续

```

void applyChmin(int p, i64 x) {
    if(x >= tr[p].mx) {
        return;
    }
    tr[p].sum -= (__int128(tr[p].mx) - x) * tr[p].cnt;
    tr[p].mx = x;
}

void push(int p) {
    applyChmin(p << 1, tr[p].mx);
    applyChmin(p << 1 | 1, tr[p].mx);
}

void chmin(int p, int l, int r, int x, int y, i64 v) {
    if(r < x || y < l || v >= tr[p].mx) {
        return;
    }
    if(l == r || (x <= l && r <= y && v > tr[p].se)) {
        applyChmin(p, v);
        return;
    }
    push(p);
    int m = (l + r) >> 1;
    chmin(p << 1, l, m, x, y, v);
    chmin(p << 1 | 1, m + 1, r, x, y, v);
    pull(p);
}

void chmin(int l, int r, i64 v) {
    chmin(1, 1, n, l, r, v);
}

i64 sum(int p, int l, int r, int x, int y) {
    if(x <= l && r <= y) {
        return tr[p].sum;
    }
    push(p);
    int m = (l + r) >> 1;
    i64 res = 0;
    if(x <= m) {
        res += sum(p << 1, l, m, x, y);
    }
    if(y > m) {
        res += sum(p << 1 | 1, m + 1, r, x, y);
    }
    return res;
}

i64 sum(int l, int r) {
    return sum(1, 1, n, l, r);
}

i64 maxi(int p, int l, int r, int x, int y) {
    if(x <= l && r <= y) {
        return tr[p].mx;
    }
    push(p);
    int m = (l + r) >> 1;
    i64 res = negInf;
    if(x <= m) {
        res = max(res, maxi(p << 1, l, m, x, y));
    }
    if(y > m) {
        res = max(res, maxi(p << 1 | 1, m + 1, r, x, y));
    }
    return res;
}

i64 maxi(int l, int r) {
    return maxi(1, 1, n, l, r);
}

};

```

1.12 FHQ Treap 多重集

允许重复，erase 删一个；rank(v)=小于 v 的个数+1；kth 从 1 开始且必须合法。prev/next 为严格前驱/后继，用 bool 和输出参数接收。期望 $O(\log n)$ ，历史节点不回收。

1.12 FHQ Treap 多重集 · 主体

```
using namespace std;
struct FHQ {
    struct Node {
        int ls = 0, rs = 0;
        int val = 0, sz = 0;
        uint32_t pri = 0;
    };
    vector<Node> tr;
    int root = 0;
    mt19937 rng;
    FHQ(int q = 0,
        uint32_t seed = uint32_t(chrono::steady_clock::now().time_since_epoch().count()))
        : tr(1), rng(seed) {
        tr.reserve(q + 1);
    }
    int size(int p) const {
        return tr[p].sz;
    }
    int size() const {
        return size(root);
    }
    int newNode(int v) {
        tr.push_back({0, 0, v, 1, rng()});
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].sz = size(tr[p].ls) + size(tr[p].rs) + 1;
    }
    void splitLess(int p, int v, int& x, int& y) {
        if(!p) {
            x = y = 0;
            return;
        }
        if(tr[p].val < v) {
            x = p;
            splitLess(tr[p].rs, v, tr[p].rs, y);
        } else {
            y = p;
            splitLess(tr[p].ls, v, x, tr[p].ls);
        }
        pull(p);
    }
    void splitLE(int p, int v, int& x, int& y) {
        if(!p) {
            x = y = 0;
            return;
        }
        if(tr[p].val <= v) {
            x = p;
            splitLE(tr[p].rs, v, tr[p].rs, y);
        } else {
            y = p;
            splitLE(tr[p].ls, v, x, tr[p].ls);
        }
        pull(p);
    }
    int merge(int x, int y) {
        if(!x || !y) {
            return x | y;
        }
        if(tr[x].pri > tr[y].pri) {
            tr[x].rs = merge(tr[x].rs, y);
        }
    }
};
```

1.12 FHQ Treap 多重集 · 主体 · 续

```

        pull(x);
        return x;
    }
    tr[y].ls = merge(x, tr[y].ls);
    pull(y);
    return y;
}
void insert(int v) {
    int x, y;
    splitLE(root, v, x, y);
    root = merge(merge(x, newNode(v)), y);
}
bool erase(int v) {
    int x, y, z;
    splitLess(root, v, x, y);
    splitLE(y, v, y, z);
    bool ok = y != 0;
    if(y) {
        y = merge(tr[y].ls, tr[y].rs);
    }
    root = merge(x, merge(y, z));
    return ok;
}
int rank(int v) {
    int x, y;
    splitLess(root, v, x, y);
    int res = size(x) + 1;
    root = merge(x, y);
    return res;
}
int kth(int k) const {
    int p = root;
    while(p) {
        int s = size(tr[p].ls);
        if(k == s + 1) {
            return tr[p].val;
        }
        if(k <= s) {
            p = tr[p].ls;
        } else {
            k -= s + 1;
            p = tr[p].rs;
        }
    }
    assert(false);
    return 0;
}
bool prev(int v, int& res) {
    int x, y;
    splitLess(root, v, x, y);
    int p = x;
    while(p && tr[p].rs) {
        p = tr[p].rs;
    }
    if(p) {
        res = tr[p].val;
    }
    root = merge(x, y);
    return p != 0;
}
bool next(int v, int& res) {
    int x, y;
    splitLE(root, v, x, y);
    int p = y;
    while(p && tr[p].ls) {
        p = tr[p].ls;
    }
}

```

1.12 FHQ Treap 多重集 · 主体 · 续

```

    if(p) {
        res = tr[p].val;
    }
    root = merge(x, y);
    return p != 0;
}
};

```

1.13 隐式 Treap 序列

位置从 1 开始, `insert(root,k,t)` 在前 k 个元素后插入子树; `erase(root,l,r)` 返回被剪下的子树, 可再插回。 `kth` 返回节点编号。 `Policy` 的 `merge` 必须按序, `reverseInfo` 必须真实反转有方向的信息; 本例的区间加和反转互相可交换。

`split(rt,k)` 拆成前 k 个和余下; `merge` 只按顺序拼接。若维护字符串哈希, `Info` 要存正反两个哈希, `reverseInfo` 交换它们。

1.13 隐式 Treap 序列 · 主体

```

using namespace std;
template <class Policy> struct Treap {
    using Val = typename Policy::Val;
    using Info = typename Policy::Info;
    using Tag = typename Policy::Tag;
    struct Node {
        int ls = 0, rs = 0, sz = 1;
        uint32_t pri = 0;
        Val val{};
        Info info{};
        Tag tag{};
        bool rev = false;
    };
};
vector<Node> tr;
mt19937 rng;
Treap(int q = 0,
      uint32_t seed = uint32_t(chrono::steady_clock::now().time_since_epoch().count()))
: tr(1), rng(seed) {
    tr.reserve(q + 1);
    tr[0].sz = 0;
    tr[0].info = Policy::identity();
    tr[0].tag = Policy::tagIdentity();
}
int size(int p) const {
    return tr[p].sz;
}
Info info(int p) const {
    return p ? tr[p].info : Policy::identity();
}
int newNode(const Val& v) {
    tr.push_back({});
    int p = int(tr.size()) - 1;
    tr[p].pri = rng();
    tr[p].val = v;
    tr[p].info = Policy::make(v);
    tr[p].tag = Policy::tagIdentity();
    return p;
}
void pull(int p) {
    tr[p].sz = size(tr[p].ls) + size(tr[p].rs) + 1;
    tr[p].info = Policy::merge(Policy::merge(info(tr[p].ls), Policy::make(tr[p].val)),
                              info(tr[p].rs));
}
void applyTag(int p, const Tag& tag) {
    if(!p || Policy::tagEmpty(tag)) {
        return;
    }
}

```

1.13 隐式 Treap 序列 · 主体 · 续

```

    Policy::apply(tr[p].val, tr[p].info, tr[p].sz, tag);
    Policy::compose(tr[p].tag, tag);
}
void applyRev(int p) {
    if(!p) {
        return;
    }
    swap(tr[p].ls, tr[p].rs);
    tr[p].rev ^= 1;
    Policy::reverseInfo(tr[p].info);
}
void push(int p) {
    if(tr[p].rev) {
        applyRev(tr[p].ls);
        applyRev(tr[p].rs);
        tr[p].rev = false;
    }
    if(!Policy::tagEmpty(tr[p].tag)) {
        applyTag(tr[p].ls, tr[p].tag);
        applyTag(tr[p].rs, tr[p].tag);
        tr[p].tag = Policy::tagIdentity();
    }
}
pair<int, int> split(int p, int k) {
    if(!p) {
        return {0, 0};
    }
    push(p);
    if(size(tr[p].ls) >= k) {
        auto [x, y] = split(tr[p].ls, k);
        tr[p].ls = y;
        pull(p);
        return {x, p};
    }
    auto [x, y] = split(tr[p].rs, k - size(tr[p].ls) - 1);
    tr[p].rs = x;
    pull(p);
    return {p, y};
}
int merge(int x, int y) {
    if(!x || !y) {
        return x | y;
    }
    if(tr[x].pri > tr[y].pri) {
        push(x);
        tr[x].rs = merge(tr[x].rs, y);
        pull(x);
        return x;
    }
    push(y);
    tr[y].ls = merge(x, tr[y].ls);
    pull(y);
    return y;
}
int build(const vector<Val>& a) {
    int root = 0;
    for(const Val& v : a) {
        root = merge(root, newNode(v));
    }
    return root;
}
int kth(int root, int k) {
    int p = root;
    while(p) {
        push(p);
        int s = size(tr[p].ls);
        if(k == s + 1) {

```

1.13 隐式 Treap 序列 · 主体 · 续

```

        return p;
    }
    if(k <= s) {
        p = tr[p].ls;
    } else {
        k -= s + 1;
        p = tr[p].rs;
    }
}
return 0;
}
tuple<int, int, int> splitRange(int root, int l, int r) {
    auto [a, bc] = split(root, l - 1);
    auto [b, c] = split(bc, r - l + 1);
    return {a, b, c};
}
Info query(int& root, int l, int r) {
    auto [a, b, c] = splitRange(root, l, r);
    Info res = info(b);
    root = merge(a, merge(b, c));
    return res;
}
void apply(int& root, int l, int r, const Tag& tag) {
    auto [a, b, c] = splitRange(root, l, r);
    applyTag(b, tag);
    root = merge(a, merge(b, c));
}
void rangeReverse(int& root, int l, int r) {
    auto [a, b, c] = splitRange(root, l, r);
    applyRev(b);
    root = merge(a, merge(b, c));
}
int erase(int& root, int l, int r) {
    auto [a, b, c] = splitRange(root, l, r);
    root = merge(a, c);
    return b;
}
void insert(int& root, int pos, int mid) {
    auto [a, b] = split(root, pos);
    root = merge(merge(a, mid), b);
}
};

```

完整 Policy: 区间加、反转、求和

1.13 · 完整 Policy: 区间加、反转、求和

```

struct Policy {
    using Val = long long;
    using Info = long long;
    using Tag = long long;
    static Info identity() {
        return 0;
    }
    static Tag tagIdentity() {
        return 0;
    }
    static bool tagEmpty(Tag t) {
        return t == 0;
    }
    static Info make(Val v) {
        return v;
    }
    static Info merge(Info a, Info b) {
        return a + b;
    }
    static void apply(Val& v, Info& sum, int len, Tag t) {
        v += t;
        sum += len * t;
    }
    static void compose(Tag& a, Tag b) {
        a += b;
    }
    static void reverseInfo(Info&) {}
    // 和与方向无关, 才能留空。
};

int main() {
    Treap<Policy> s;
    int rt = s.build(vector<long long>{1, 2, 3, 4}); // build为0-based.
    s.apply(rt, 2, 4, 10);
    s.rangeReverse(rt, 1, 3);
    assert(s.query(rt, 1, 2) == 25);
    int t = s.erase(rt, 2, 3);
    s.insert(rt, 0, t);
    assert(s.size(rt) == 4);
}

```

1.14 PBDS 可重多重集

GNU 扩展; 用 (值, 唯一编号) 处理重复。orderOfKey(v) 返回严格小于 v 的个数, kth(k, ans) 从 1 开始, 越界返回 false。仅支持元素级操作, 不能替代带区间信息的平衡树。

1.14 PBDS 可重多重集 · 主体

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template <class T> struct OrderedSet {
    using Key = pair<T, int>;
    tree<Key, null_type, less<Key>, rb_tree_tag, tree_order_statistics_node_update> tr;
    int uid = 0;
    int size() const {
        return int(tr.size());
    }
    void insert(const T& x) {
        tr.insert({x, uid++});
    }
    bool erase(const T& x) {
        auto it = tr.lower_bound({x, numeric_limits<int>::min()});
        if(it == tr.end() || it->first != x) {

```

1.14 PBDS 可多重集 · 主体 · 续

```

        return false;
    }
    tr.erase(it);
    return true;
}
int orderOfKey(const T& x) const {
    return int(tr.order_of_key({x, numeric_limits<int>::min()}));
}
int count(const T& x) const {
    return int(tr.order_of_key({x, numeric_limits<int>::max()})) - orderOfKey(x);
}
bool kth(int k, T& ans) const {
    if(k < 1 || k > size()) {
        return false;
    }
    ans = tr.find_by_order(k - 1)->first;
    return true;
}
};

```

1.15 李超线段树

维护整数 $x \in [L, R]$ 上的最小 $kx+b$, $\text{addLine}(k,b)$, $\text{query}(x)$ 返回 $i128$; 空集合返回 inf 。插入与查询 $O(\log(R-L+1))$ 。无需斜率或查询顺序单调。值域中点及乘法范围要符合本版类型。

1.15 李超线段树 · 主体

```

using namespace std;
using i64 = long long;
struct LiChao {
    using i128 = __int128;
    static constexpr i128 inf = i128(1) << 120;
    struct Line {
        i64 k = 0, b = 0;
        i128 get(i64 x) const {
            return i128(k) * x + b;
        }
    };
};
struct Node {
    int ls = 0, rs = 0;
    Line line;
    bool has = false;
};
i64 L, R;
vector<Node> tr;
int root = 0;
LiChao(i64 l, i64 r) : L(l), R(r), tr(1) {}
int newNode(Line line) {
    tr.push_back({0, 0, line, true});
    return int(tr.size()) - 1;
}
int addLine(int p, i64 l, i64 r, Line line) {
    if(!p) {
        return newNode(line);
    }
    i64 m = l + (r - l) / 2;
    bool left = line.get(l) < tr[p].line.get(l);
    bool mid = line.get(m) < tr[p].line.get(m);
    if(mid) {
        swap(line, tr[p].line);
    }
    if(l == r) {
        return p;
    }
    if(left != mid) {

```

1.15 李超线段树 · 主体 · 续

```

        int child = addLine(tr[p].ls, l, m, line);
        tr[p].ls = child;
    } else {
        int child = addLine(tr[p].rs, m + 1, r, line);
        tr[p].rs = child;
    }
    return p;
}
void addLine(i64 k, i64 b) {
    root = addLine(root, L, R, {k, b});
}
i128 query(int p, i64 l, i64 r, i64 x) const {
    if(!p) {
        return inf;
    }
    i128 res = tr[p].line.get(x);
    if(l == r) {
        return res;
    }
    i64 m = l + (r - l) / 2;
    if(x <= m) {
        return min(res, query(tr[p].ls, l, m, x));
    }
    return min(res, query(tr[p].rs, m + 1, r, x));
}
i128 query(i64 x) const {
    return query(root, L, R, x);
}
};

```

斜率任意顺序：最小直线值

1.15 · 斜率任意顺序：最小直线值

```

int main() {
    LiChao s(-1000000, 1000000);
    s.addLine(3, 5);
    s.addLine(-2, 1);
    assert(s.query(7) == -13);
}

```

1.16 凸包优化 DP

CHT 保存点 (x,y) ，按 x 非递减插入， $\text{query}(\{a,b\})$ 求最小 $ax+by$ ，要求 $b>0$ 。点积/叉积用 $i128$ 。把转移写成这个点积再代入；若条件不满足改用李超树。插入均摊 $O(1)$ ，查询 $O(\log n)$ 。

把候选 j 写成点 $(j, dp[j]+j^2)$ ，查询向量 $(-2i, 1)$ ，常数 i^2 最后加；插入点横坐标递增。

1.16 凸包优化 DP · 主体

```

using namespace std;
using i64 = long long;
struct CHT {
    using i128 = __int128;
    struct Point {
        i64 x, y;
        Point operator-(const Point& p) const {
            return {x - p.x, y - p.y};
        }
    };
    vector<Point> hull;
    static i128 cross(Point a, Point b) {
        return i128(a.x) * b.y - i128(a.y) * b.x;
    }
    static i128 dot(Point a, Point b) {
        return i128(a.x) * b.x + i128(a.y) * b.y;
    }
};

```

1.16 凸包优化 DP · 主体 · 续

```

    }
    void addPoint(Point p) {
        if(!hull.empty() && hull.back().x == p.x) {
            if(hull.back().y <= p.y) {
                return;
            }
            hull.pop_back();
        }
        while(hull.size() >= 2 &&
            cross(hull.back() - hull[hull.size() - 2], p - hull.back()) <= 0) {
            hull.pop_back();
        }
        hull.push_back(p);
    }
    i128 query(Point q) const {
        int l = 0, r = int(hull.size()) - 1;
        while(l < r) {
            int m = (l + r) >> 1;
            if(dot(hull[m], q) <= dot(hull[m + 1], q)) {
                r = m;
            } else {
                l = m + 1;
            }
        }
        return dot(hull[l], q);
    }
};

```

转移 $dp[i]=\min(dp[j]+(i-j)^2)$ 1.16 · 转移 $dp[i]=\min(dp[j]+(i-j)^2)$

```

int main() {
    CHT s;
    vector<long long> dp(9);
    s.addPoint({0, 0});
    for(int i = 1; i <= 8; i++) {
        dp[i] = (long long)(s.query({-2LL * i, 1}) + 1LL * i * i);
        s.addPoint({i, dp[i] + 1LL * i * i});
    }
    assert(dp[8] == 8);
}

```

1.17 笛卡尔树

输入含下标 0，占位不参与；小根堆，等值按本栈条件决定归属。root、left、right 描述树；中序遍历就是原下标顺序。O(n)，递归遍历它时要注意单调数组会退化成链。

1.17 笛卡尔树 · 主体

```

using namespace std;
template <class T, class Compare = less<T>> struct Cartesian {
    int root = 0;
    vector<int> parent, left, right;
    Cartesian(const vector<T>& a, Compare cmp = Compare())
        : parent(a.size()), left(a.size()), right(a.size()) {
        vector<int> st;
        for(int i = 1; i < int(a.size()); i++) {
            int last = 0;
            while(!st.empty() && cmp(a[i], a[st.back()])) {
                last = st.back();
                st.pop_back();
            }
            if(!st.empty()) {
                parent[i] = st.back();
                right[st.back()] = i;
            }
        }
    }
};

```

1.17 笛卡尔树 · 主体 · 续

```

    }
    if(last) {
        parent[last] = i;
        left[i] = last;
    }
    st.push_back(i);
}
if(!st.empty()) {
    root = st.front();
}
}
};

```

1.18 Link-Cut Tree 动态森林

本版信息为点权异或，setValue、link、cut、connected、pathXor。link 拒绝成环，cut 只删除直接相邻的边；查询 pathXor 前先确认连通。makeroot 会改变代表根，不能把 findRoot 当固定根 LCA。均摊 $O(\log n)$ 。

1.18 Link-Cut Tree 动态森林 · 主体

```

using namespace std;
using i64 = long long;
struct LCT {
    struct Node {
        int ch[2]{}, fa = 0;
        i64 val = 0, sum = 0;
        bool rev = false;
    };
    vector<Node> tr;
    LCT(int n = 0) : tr(n + 1) {}
    bool isRoot(int x) const {
        int f = tr[x].fa;
        return !f || (tr[f].ch[0] != x && tr[f].ch[1] != x);
    }
    void pull(int x) {
        tr[x].sum = tr[tr[x].ch[0]].sum ^ tr[x].val ^ tr[tr[x].ch[1]].sum;
    }
    void applyRev(int x) {
        if(!x) {
            return;
        }
        swap(tr[x].ch[0], tr[x].ch[1]);
        tr[x].rev ^= 1;
    }
    void push(int x) {
        if(!tr[x].rev) {
            return;
        }
        applyRev(tr[x].ch[0]);
        applyRev(tr[x].ch[1]);
        tr[x].rev = false;
    }
    void rotate(int x) {
        int y = tr[x].fa, z = tr[y].fa;
        int k = tr[y].ch[1] == x;
        if(!isRoot(y)) {
            tr[z].ch[tr[z].ch[1] == y] = x;
        }
        tr[x].fa = z;
        tr[y].ch[k] = tr[x].ch[k ^ 1];
        if(tr[x].ch[k ^ 1]) {
            tr[tr[x].ch[k ^ 1]].fa = y;
        }
        tr[x].ch[k ^ 1] = y;
        tr[y].fa = x;
    }

```

1.18 Link-Cut Tree 动态森林 · 主体 · 续

```

    pull(y);
    pull(x);
}
void splay(int x) {
    vector<int> st{x};
    for(int y = x; !isRoot(y); y = tr[y].fa) {
        st.push_back(tr[y].fa);
    }
    while(!st.empty()) {
        push(st.back());
        st.pop_back();
    }
    while(!isRoot(x)) {
        int y = tr[x].fa, z = tr[y].fa;
        if(!isRoot(y)) {
            if((tr[y].ch[1] == x) == (tr[z].ch[1] == y)) {
                rotate(y);
            } else {
                rotate(x);
            }
        }
        rotate(x);
    }
}
int access(int x) {
    int y = 0;
    for(int z = x; z; z = tr[z].fa) {
        splay(z);
        tr[z].ch[1] = y;
        pull(z);
        y = z;
    }
    splay(x);
    return y;
}
void makeRoot(int x) {
    access(x);
    applyRev(x);
}
int findRoot(int x) {
    access(x);
    while(tr[x].ch[0]) {
        push(x);
        x = tr[x].ch[0];
    }
    splay(x);
    return x;
}
bool connected(int x, int y) {
    return x == y || findRoot(x) == findRoot(y);
}
bool link(int x, int y) {
    makeRoot(x);
    if(findRoot(y) == x) {
        return false;
    }
    tr[x].fa = y;
    return true;
}
bool cut(int x, int y) {
    makeRoot(x);
    access(y);
    if(tr[y].ch[0] != x || tr[x].ch[1]) {
        return false;
    }
    tr[y].ch[0] = tr[x].fa = 0;
    pull(y);
}

```

1.18 Link-Cut Tree 动态森林 · 主体 · 续

```

    return true;
}
void setValue(int x, i64 value) {
    access(x);
    tr[x].val = value;
    pull(x);
}
i64 pathXor(int x, int y) {
    makeRoot(x);
    access(y);
    return tr[y].sum;
}
};

```

动态树点权异或

1.18 · 动态树点权异或

```

int main() {
    LCT s(3);
    for(int i = 1; i <= 3; i++) {
        s.setValue(i, i);
    }
    assert(s.link(1, 2) && s.link(2, 3));
    assert(s.pathXor(1, 3) == (1 ^ 2 ^ 3));
    assert(!s.link(1, 3));
    assert(s.cut(1, 2));
    assert(!s.connected(1, 3));
}

```

1.19 按类别维护 Top-K

同类别只保留最大值；get(ban1,ban2) 排除至多两个类别。默认 -1 表示没有这个排除条件，因此类别不能使用 -1。
K 至少为排除类别数+1；空答案为 -INF。可用于换根时排除来源子树。

1.19 按类别维护 Top-K · 主体

```

using namespace std;
using i64 = long long;
// 不同 key 的前 K 大值；空槽 key = -1、val = -INF (key 可能为 -1 时需改哨兵)
template <int K> struct TopK {
    static constexpr i64 INF = numeric_limits<i64>::max() / 4;
    struct Node {
        i64 val = -INF;
        int key = -1;
    };
    array<Node, K> a;
    void norm() {
        sort(a.begin(), a.end(),
            [](const Node& x, const Node& y) { return x.val > y.val; });
    }
    // 插入 (key, val): 同 key 保留较大值；否则尝试替换第 K 小
    void add(int key, i64 val) {
        for(auto& x : a) {
            if(x.key == key) {
                x.val = max(x.val, val);
                norm();
                return;
            }
        }
        if(val > a[K - 1].val) {
            a[K - 1] = {val, key};
            norm();
        }
    }
    // 返回未被禁用 key 的最大 val；无候选时返回 -INF

```

1.19 按类别维护 Top-K · 主体 · 续

```
    i64 get(int ban1 = -1, int ban2 = -1) const {  
        for(const auto& x : a) {  
            if(x.key != ban1 && x.key != ban2) {  
                return x.val;  
            }  
        }  
        return -INF;  
    }  
};
```

2. 图论与建模

2.1 Dijkstra

1-based, 有向 addEdge; 无向边添加两次。边权必须非负, run(s) 返回距离数组, 不可达为 inf。堆实现 $O((n+m)\log n)$; 所有有限路径和必须小于 inf, 不能拿 inf 直接参与答案运算。

2.1 Dijkstra · 主体

```
using namespace std;
using i64 = long long;
struct Dijkstra {
    struct Edge {
        int to;
        i64 w;
    };
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;
    vector<vector<Edge>> adj;
    Dijkstra(int n = 0) : adj(n + 1) {}
    void addEdge(int u, int v, i64 w) {
        adj[u].push_back({v, w});
    }
    vector<i64> run(int s) const {
        vector<i64> dis(adj.size(), inf);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
        dis[s] = 0;
        q.push({0, s});
        while(!q.empty()) {
            auto [du, u] = q.top();
            q.pop();
            if(du != dis[u]) {
                continue;
            }
            for(auto [v, w] : adj[u]) {
                if(dis[v] > du + w) {
                    dis[v] = du + w;
                    q.push({dis[v], v});
                }
            }
        }
        return dis;
    }
};
```

2.2 0-1 BFS

边权只能为 0 或 1; 0 放队首、1 放队尾, $O(n+m)$ 。接口同 Dijkstra, 不能推广为任意小整数边权。

2.2 0-1 BFS · 主体

```
using namespace std;
struct BFS01 {
    struct Edge {
        int to, w;
    };
    static constexpr int inf = numeric_limits<int>::max() / 4;
    vector<vector<Edge>> adj;
    BFS01(int n = 0) : adj(n + 1) {}
    void addEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
    }
    vector<int> run(int s) const {
        vector<int> dis(adj.size(), inf);
        deque<int> q;
```

2.2 0-1 BFS · 主体 · 续

```

    dis[s] = 0;
    q.push_back(s);
    while(!q.empty()) {
        int u = q.front();
        q.pop_front();
        for(auto [v, w] : adj[u]) {
            if(dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                if(w) {
                    q.push_back(v);
                } else {
                    q.push_front(v);
                }
            }
        }
    }
    return dis;
}
};

```

2.3 Floyd

d 为 1-based 方阵，初值对角 0，重边取 $\min$ ，不可达 inf ；支持负边但最短路意义要求无负环。 $O(n^3)$ 。若 $d[k][k] < 0$ ，则经 k 可达的点对没有有限最短路，需另行标记。

2.3 Floyd · 主体

```

using namespace std;
using i64 = long long;
void floyd(vector<vector<i64>>& dis, i64 inf) {
    int n = int(dis.size()) - 1;
    for(int k = 1; k <= n; k++) {
        for(int i = 1; i <= n; i++) {
            if(dis[i][k] == inf) {
                continue;
            }
            for(int j = 1; j <= n; j++) {
                if(dis[k][j] == inf) {
                    continue;
                }
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
            }
        }
    }
}

```

2.4 拓扑排序

输入 1-based 邻接表；返回全体点的一个拓扑序，空返回值在 $n > 0$ 时表示存在环。重复边计入入度也必须逐条减回。 $O(n+m)$ ；要字典序最小则队列换小根堆。

2.4 拓扑排序 · 主体

```

using namespace std;
vector<int> toposort(const vector<vector<int>>& adj) {
    vector<int> deg(adj.size());
    int n = int(adj.size()) - 1;
    for(int u = 1; u <= n; u++) {
        for(int v : adj[u]) {
            deg[v]++;
        }
    }
    queue<int> q;
    for(int i = 1; i <= n; i++) {

```

2.4 拓扑排序 · 主体 · 续

```

        if(!deg[i]) {
            q.push(i);
        }
    }
    vector<int> order;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for(int v : adj[u]) {
            if(--deg[v] == 0) {
                q.push(v);
            }
        }
    }
    if(int(order.size()) != n) {
        return {};
    }
    return order;
}

```

2.5 Kruskal 最小生成森林

`run(n,e)` 返回权重和与选边；不连通也返回森林。选边数等于 $n-1$ 才是生成树。支持负边、重边，自环自动跳过； $O(m \log m)$ 。

2.5 Kruskal 最小生成森林 · 主体

```

using namespace std;
using i64 = long long;
struct Kruskal {
    struct Edge {
        int u, v;
        i64 w;
    };
    static pair<i64, vector<int>> run(int n, const vector<Edge>& edges) {
        vector<int> p(n + 1), sz(n + 1, 1), id(edges.size());
        iota(p.begin(), p.end(), 0);
        iota(id.begin(), id.end(), 0);
        sort(id.begin(), id.end(), [&](int i, int j) { return edges[i].w < edges[j].w; });
        auto find = [&](int x) {
            while(x != p[x]) {
                x = p[x] = p[p[x]];
            }
            return x;
        };
        i64 sum = 0;
        vector<int> chosen;
        for(int i : id) {
            int x = find(edges[i].u);
            int y = find(edges[i].v);
            if(x == y) {
                continue;
            }
            if(sz[x] < sz[y]) {
                swap(x, y);
            }
            p[y] = x;
            sz[x] += sz[y];
            sum += edges[i].w;
            chosen.push_back(i + 1);
        }
        return {sum, chosen};
    }
};

```

2.6 二分图度数序列构造

输入左右度数 1-based; $\text{run}(n,m,a,b,c)$ 返回可行性与 0/1 邻接矩阵。度数非负，左不超过 m 、右不超过 n ，总和必须相等。构造矩阵本身需 $O(nm)$ 空间，稠密输出时使用。

2.6 二分图度数序列构造 · 主体

```
using namespace std;
using i64 = long long;
// Gale-Ryser 构造: 逐行取剩余度数最大的若干列 (按剩余度分组, 从组内右侧取)
struct BiDegree {
    static bool run(int n, int m, const vector<int>& row, const vector<int>& col,
        vector<vector<int>>& mat) {
        mat.assign(n + 1, vector<int>(m + 1, 0));
        vector<int> csum(m + 1);
        i64 total = 0;
        for(int i = 1; i <= n; i++) {
            if(row[i] < 0 || row[i] > m) {
                return false;
            }
            total += row[i];
        }
        for(int j = 1; j <= m; j++) {
            if(col[j] < 0 || col[j] > n) {
                return false;
            }
            csum[j] = col[j];
            total -= col[j];
        }
        if(total != 0) {
            return false;
        }
        vector<int> ord(m);
        iota(ord.begin(), ord.end(), 1);
        sort(ord.begin(), ord.end(), [&](int a, int b) { return csum[a] > csum[b]; });
        for(int i = 1; i <= n; i++) {
            int cnt = row[i];
            for(int lo = 0, hi = 0; lo < m && cnt > 0; lo = hi) {
                if(csum[ord[lo]] == 0) {
                    break;
                }
                while(hi < m && csum[ord[hi]] == csum[ord[lo]]) {
                    hi++;
                }
                for(int k = hi - 1; k >= lo && cnt > 0; k--) {
                    mat[i][ord[k]] = 1;
                    csum[ord[k]]--;
                    cnt--;
                }
            }
            if(cnt > 0) {
                return false;
            }
        }
        for(int j = 1; j <= m; j++) {
            if(csum[j] != 0) {
                return false;
            }
        }
        return true;
    }
};
```

2.7 增量 APSP

$\text{APSP}(n)$, $\text{addDirected}(u,v,w)$ 在已有最短路上加入一条边, $O(n^2)$, 只支持加边。要求每次加入后无负环, 距离在 INF 范围内; 无向边用 addUndirected 。删除/加大权值不能这样更新。

2.7 增量 APSP · 主体

```

using namespace std;
using i64 = long long;
struct APSP {
    static constexpr i64 INF = numeric_limits<i64>::max() / 4;
    int n;
    vector<vector<i64>> dis;
    APSP(int n_) : n(n_), dis(n_ + 1, vector<i64>(n_ + 1, INF)) {
        for(int i = 1; i <= n; i++) {
            dis[i][i] = 0;
        }
    }
    void addEdge(int u, int v, i64 w) {
        relax(u, v, w);
        relax(v, u, w);
    }
    void addDirected(int u, int v, i64 w) {
        relax(u, v, w);
    }
    void relax(int u, int v, i64 w) {
        for(int i = 1; i <= n; i++) {
            if(dis[i][u] == INF) {
                continue;
            }
            for(int j = 1; j <= n; j++) {
                if(dis[v][j] == INF) {
                    continue;
                }
                if(dis[i][j] > dis[i][u] + w + dis[v][j]) {
                    dis[i][j] = dis[i][u] + w + dis[v][j];
                }
            }
        }
    }
};

```

2.8 强连通分量 SCC

SCC(g) 接收 1-based 有向邻接表; id 从 0 起, comp 按 Tarjan 出栈顺序, 跨分量边 $u \rightarrow v$ 满足 $\text{id}[u] > \text{id}[v]$ 。缩点后去重可选; 时间 $O(n+m)$, 遍历为显式栈, 支持深链。

2.8 强连通分量 SCC · 主体

```

using namespace std;
struct SCC {
    int n, timer = 0, count = 0;
    const vector<vector<int>>& adj;
    vector<int> dfn, low, stack, id;
    vector<bool> inStack;
    vector<vector<int>> comp;
    SCC(const vector<vector<int>>& graph)
        : n(int(graph.size()) - 1), adj(graph), dfn(graph.size()), low(graph.size()),
          id(graph.size(), -1), inStack(graph.size()) {
        for(int u = 1; u <= n; u++) {
            if(!dfn[u]) {
                dfs(u);
            }
        }
    }
    void dfs(int s) {
        vector<pair<int, int>> cs{{s, 0}};
        dfn[s] = low[s] = ++timer;
        stack.push_back(s);
        inStack[s] = true;
        while(!cs.empty()) {
            int u = cs.back().first;
            int& i = cs.back().second;

```

2.8 强连通分量 SCC · 主体 · 续

```

        if(i < int(adj[u].size())) {
            int v = adj[u][i++];
            if(!dfn[v]) {
                dfn[v] = low[v] = ++timer;
                stack.push_back(v);
                inStack[v] = true;
                cs.push_back({v, 0});
            } else if(inStack[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        } else {
            cs.pop_back();
            if(low[u] == dfn[u]) {
                comp.push_back({});
                while(true) {
                    int v = stack.back();
                    stack.pop_back();
                    inStack[v] = false;
                    id[v] = count;
                    comp.back().push_back(v);
                    if(v == u) {
                        break;
                    }
                }
                count++;
            }
            if(!cs.empty()) {
                int p = cs.back().first;
                low[p] = min(low[p], low[u]);
            }
        }
    }
}
};

```

2.9 2-SAT

变量 1..n; addClause(u,a,v,b) 表示 $(xu=a)$ 或 $(xv=b)$, setValue 固定变量。solve() 返回可行性, answer[1..n] 给一组解; 支持加约束后重新 solve。不能把单条蕴含误当一个完整析取, 否定与逆否必须成对。

2.9 2-SAT · 主体

```

using namespace std;
struct TwoSAT {
    int n;
    vector<vector<int>>> adj;
    vector<bool> answer;
    vector<int> dfn, low, stack, id;
    vector<bool> inStack;
    int timer = 0, count = 0;
    TwoSAT(int n_ = 0)
        : n(n_), adj(2 * n_ + 2), answer(n_ + 1), dfn(2 * n_ + 2), low(2 * n_ + 2),
          id(2 * n_ + 2, -1), inStack(2 * n_ + 2) {}
    int literal(int x, bool value) const {
        return 2 * x + int(value);
    }
    void imply(int a, int b) {
        adj[a].push_back(b);
    }
    void addClause(int x, bool xValue, int y, bool yValue) {
        int a = literal(x, xValue);
        int b = literal(y, yValue);
        imply(a ^ 1, b);
        imply(b ^ 1, a);
    }
}

```

2.9 2-SAT · 主体 · 续

```

void setValue(int x, bool value) {
    addClause(x, value, x, value);
}

bool solve() {
    timer = count = 0;
    stack.clear();
    fill(dfn.begin(), dfn.end(), 0);
    fill(low.begin(), low.end(), 0);
    fill(inStack.begin(), inStack.end(), false);
    for(int u = 2; u <= 2 * n + 1; u++) {
        if(!dfn[u]) {
            dfs(u);
        }
    }
    for(int i = 1; i <= n; i++) {
        if(id[2 * i] == id[2 * i + 1]) {
            return false;
        }
        answer[i] = id[2 * i] > id[2 * i + 1];
    }
    return true;
}

private:
void dfs(int s) {
    vector<pair<int, int>> cs{{s, 0}};
    dfn[s] = low[s] = ++timer;
    stack.push_back(s);
    inStack[s] = true;
    while(!cs.empty()) {
        int u = cs.back().first;
        int& i = cs.back().second;
        if(i < int(adj[u].size())) {
            int v = adj[u][i++];
            if(!dfn[v]) {
                dfn[v] = low[v] = ++timer;
                stack.push_back(v);
                inStack[v] = true;
                cs.push_back({v, 0});
            } else if(inStack[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        } else {
            cs.pop_back();
            if(low[u] == dfn[u]) {
                while(true) {
                    int v = stack.back();
                    stack.pop_back();
                    inStack[v] = false;
                    id[v] = count;
                    if(v == u) {
                        break;
                    }
                }
                count++;
            }
        }
        if(!cs.empty()) {
            int p = cs.back().first;
            low[p] = min(low[p], low[u]);
        }
    }
}
};

```

析取与固定变量

2.9 · 析取与固定变量

```
int main() {
    TwoSAT s(2);
    s.addClause(1, true, 2, false); // x1 或 非x2.
    s.setValue(1, false);
    assert(s.solve());
    assert(!s.answer[1] && !s.answer[2]);
}
```

2.10 桥与割点 Lowlink

无向图，addEdge 返回 1-based 边编号，work 后 bridge[id]、cut[u]。用父边编号而非父点跳过，正确处理平行边；自环不影响桥割点。O(n+m)，支持不连通与深链。

2.10 桥与割点 Lowlink · 主体

```
using namespace std;
struct Lowlink {
    int n, timer = 0;
    vector<pair<int, int>> edges;
    vector<vector<pair<int, int>>> adj;
    vector<int> dfn, low;
    vector<bool> bridge, cut;
    Lowlink(int n_ = 0)
        : n(n_), edges(1), adj(n_ + 1), dfn(n_ + 1), low(n_ + 1), cut(n_ + 1) {}
    int addEdge(int u, int v) {
        int id = int(edges.size());
        edges.push_back({u, v});
        adj[u].push_back({v, id});
        adj[v].push_back({u, id});
        return id;
    }
    void work() {
        bridge.assign(edges.size(), false);
        for(int u = 1; u <= n; u++) {
            if(!dfn[u]) {
                dfs(u, 0);
            }
        }
    }
    void dfs(int s, int e) {
        vector<array<int, 4>> cs{{s, e, 0, 0}};
        dfn[s] = low[s] = ++timer;
        while(!cs.empty()) {
            int u = cs.back()[0], pe = cs.back()[1];
            int& i = cs.back()[2];
            if(i < int(adj[u].size())) {
                auto [v, id] = adj[u][i++];
                if(id == pe) {
                    continue;
                }
                if(!dfn[v]) {
                    cs.back()[3]++;
                    dfn[v] = low[v] = ++timer;
                    cs.push_back({v, id, 0, 0});
                } else {
                    low[u] = min(low[u], dfn[v]);
                }
            } else {
                int child = cs.back()[3];
                cs.pop_back();
                if(!pe && child > 1) {
                    cut[u] = true;
                }
            }
        }
    }
};
```

2.10 桥与割点 Lowlink · 主体 · 续

```

        if(cs.empty()) {
            continue;
        }
        int p = cs.back()[0];
        low[p] = min(low[p], low[u]);
        if(low[u] > dfn[p]) {
            bridge[pe] = true;
        }
        if(cs.back()[1] && low[u] >= dfn[p]) {
            cut[p] = true;
        }
    }
}
};

```

2.11 边双连通分量

EdgeBCC 内含 Lowlink; work 后 $id[u]$ 从 1 起、 $comp[1..]$ 是删去所有桥后的连通块。缩成森林时每条桥连接其端点所属块; 包含孤立点。

主体依赖: 2.10 桥与割点 Lowlink。先复制这些基础定义, 再复制下列代码。

2.11 边双连通分量 · 主体

```

struct EdgeBCC {
    Lowlink bridge;
    vector<int> id;
    vector<vector<int>>> comp;
    EdgeBCC(int n = 0) : bridge(n), id(n + 1), comp(1) {}
    int addEdge(int u, int v) {
        return bridge.addEdge(u, v);
    }
    void work() {
        bridge.work();
        for(int s = 1; s <= bridge.n; s++) {
            if(id[s] != 0) {
                continue;
            }
            int bel = int(comp.size());
            comp.push_back({});
            vector<int> stack{s};
            id[s] = bel;
            while(!stack.empty()) {
                int u = stack.back();
                stack.pop_back();
                comp.back().push_back(u);
                for(auto [v, edge] : bridge.adj[u]) {
                    if(!bridge.bridge[edge] && id[v] == 0) {
                        id[v] = bel;
                        stack.push_back(v);
                    }
                }
            }
        }
    }
};

```

2.12 点双连通分量与圆方树

VertexBCC.work(), $comp[1..]$ 为块, 原点 $1..n$, 方点从 $n+1$ 开始, $tree$ 为圆方森林。桥本身也形成二点块, 孤立点单独成块; 自环不产生额外块。本版只构建一次, $O(n+m)$ 。

2.12 点双连通分量与圆方树 · 主体

```

using namespace std;
struct VertexBCC {
    int n, timer = 0;
    vector<pair<int, int>> edges;
    vector<vector<pair<int, int>>> adj;
    vector<int> dfn, low, stk;
    vector<bool> cut;
    vector<vector<int>>> comp, tree;
    VertexBCC(int n_ = 0)
        : n(n_), edges(1), adj(n_ + 1), dfn(n_ + 1), low(n_ + 1), cut(n_ + 1), comp(1) {}
    int addEdge(int u, int v) {
        int id = int(edges.size());
        edges.push_back({u, v});
        adj[u].push_back({v, id});
        adj[v].push_back({u, id});
        return id;
    }
    void work() {
        for(int u = 1; u <= n; u++) {
            if(dfn[u]) {
                continue;
            }
            if(adj[u].empty()) {
                dfn[u] = low[u] = ++timer;
                comp.push_back({u});
            } else {
                dfs(u, 0);
            }
        }
        tree.assign(n + comp.size(), {});
        for(int i = 1; i < int(comp.size()); i++) {
            int v = n + i;
            for(int u : comp[i]) {
                tree[u].push_back(v);
                tree[v].push_back(u);
            }
        }
    }
    void dfs(int s, int e) {
        vector<array<int, 4>> cs{{s, e, 0, 0}};
        dfn[s] = low[s] = ++timer;
        while(!cs.empty()) {
            int u = cs.back()[0], pe = cs.back()[1];
            int& i = cs.back()[2];
            if(i < int(adj[u].size())) {
                auto [v, id] = adj[u][i++];
                if(id == pe) {
                    continue;
                }
                if(!dfn[v]) {
                    cs.back()[3]++;
                    stk.push_back(id);
                    dfn[v] = low[v] = ++timer;
                    cs.push_back({v, id, 0, 0});
                } else {
                    low[u] = min(low[u], dfn[v]);
                    if(dfn[v] < dfn[u]) {
                        stk.push_back(id);
                    }
                }
            } else {
                int child = cs.back()[3];
                cs.pop_back();
                if(!pe && child > 1) {
                    cut[u] = true;
                }
            }
        }
    }
};

```

2.12 点双连通分量与圆方树 · 主体 · 续

```

        if(cs.empty()) {
            continue;
        }
        int p = cs.back()[0];
        low[p] = min(low[p], low[u]);
        if(low[u] >= dfn[p]) {
            if(cs.back()[1] || cs.back()[3] > 1) {
                cut[p] = true;
            }
            vector<int> v;
            while(true) {
                int e = stk.back();
                stk.pop_back();
                v.push_back(edges[e].first);
                v.push_back(edges[e].second);
                if(e == pe) {
                    break;
                }
            }
            sort(v.begin(), v.end());
            v.erase(unique(v.begin(), v.end()), v.end());
            comp.push_back(move(v));
        }
    }
}
};

```

2.13 欧拉通路

`solve(n,e,dir,ans)`: `dir=true` 有向, `false` 无向; 边允许重边、自环。返回 `bool`, `ans` 为 $m+1$ 个点; 度数条件与最终耗尽所有边同时检验。无边时 $n>0$ 返回单点; $n=0$ 返回空路径。 $O(n+m)$ 。

2.13 欧拉通路 · 主体

```

using namespace std;
struct EulerTrail {
    // dir=true: directed; dir=false: undirected. Vertices are 1..n.
    static bool solve(int n, const vector<pair<int, int>>& e, bool dir, vector<int>& ans) {
        ans.clear();
        if(!n) {
            return e.empty();
        }
        vector<vector<pair<int, int>>> g(n + 1);
        vector<int> in(n + 1), out(n + 1), it(n + 1);
        for(int i = 0; i < int(e.size()); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            ++out[u];
            ++in[v];
            if(!dir) {
                g[v].push_back({u, i}), ++out[v], ++in[u];
            }
        }
        int s = 1, plus = 0, minus = 0, odd = 0;
        for(int u = 1; u <= n; u++) {
            if(out[u]) {
                s = u;
                break;
            }
        }
        for(int u = 1; u <= n; u++) {
            if(dir) {
                int d = out[u] - in[u];
                if(abs(d) > 1) {
                    return false;
                }
            }
        }
    }
};

```

2.13 欧拉通路 · 主体 · 续

```

    }
    if(d == 1) {
        ++plus, s = u;
    }
    if(d == -1) {
        ++minus;
    }
} else if(out[u] & 1) {
    ++odd, s = u;
}
}
if(dir && !((plus == 0 && minus == 0) || (plus == 1 && minus == 1))) {
    return false;
}
if(!dir && odd != 0 && odd != 2) {
    return false;
}
vector<bool> used(e.size());
vector<int> stk(1, s);
while(!stk.empty()) {
    int u = stk.back();
    while(it[u] < int(g[u].size()) && used[g[u][it[u]].second]) {
        ++it[u];
    }
    if(it[u] == int(g[u].size())) {
        ans.push_back(u), stk.pop_back();
    } else {
        auto [v, id] = g[u][it[u]++];
        used[id] = true;
        stk.push_back(v);
    }
}
if(ans.size() != e.size() + 1) {
    ans.clear();
    return false;
}
reverse(ans.begin(), ans.end());
return true;
}
};

```

2.14 二分图最大匹配 Hopcroft-Karp

HK(n, m), 左右点各自从 1 编号, $\text{addEdge}(u, v)$, $\text{solve}()$ 返回匹配数; left/right 为匹配对象, 0 表示未匹配。 $O(E/V)$, 替换朴素增广。solve 后 $\text{minVertexCover}()$ 返回左右最小点覆盖; 补集为最大独立集。增广使用显式栈, 支持深交替链。

最小点覆盖: 从未匹配左点出发, 走未匹配边到右边, 再走匹配边回左边; 取未到达左点+到达右点。

2.14 二分图最大匹配 Hopcroft-Karp · 主体

```

using namespace std;
struct HK {
    int n, m, shortest;
    vector<vector<int>>> g;
    vector<int> left, right, d, it;
    HK(int n_, int m_)
        : n(n_), m(m_), g(n + 1), left(n + 1), right(m + 1), d(n + 1), it(n + 1) {}
    void addEdge(int u, int v) {
        g[u].push_back(v);
    }
    bool bfs() {
        queue<int> q;
        fill(d.begin(), d.end(), -1);
    }
};

```

2.14 二分图最大匹配 Hopcroft-Karp · 主体 · 续

```

    shortest = n + 1;
    for(int u = 1; u <= n; u++) {
        if(!left[u]) {
            d[u] = 0, q.push(u);
        }
    }
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        if(d[u] >= shortest) {
            continue;
        }
        for(int v : g[u]) {
            int x = right[v];
            if(!x) {
                shortest = d[u] + 1;
            } else if(d[x] < 0) {
                d[x] = d[u] + 1, q.push(x);
            }
        }
    }
    return shortest <= n;
}

bool dfs(int s) {
    vector<int> a{s}, b;
    while(!a.empty()) {
        int u = a.back();
        bool go = false;
        while(it[u] < int(g[u].size())) {
            int v = g[u][it[u]++], x = right[v];
            if(!x && d[u] + 1 == shortest) {
                b.push_back(v);
                for(int i = 0; i < int(a.size()); i++) {
                    left[a[i]] = b[i], right[b[i]] = a[i];
                }
                return true;
            }
            if(x && d[x] == d[u] + 1) {
                b.push_back(v);
                a.push_back(x);
                go = true;
                break;
            }
        }
        if(!go) {
            d[u] = -1;
            a.pop_back();
            if(!b.empty()) {
                b.pop_back();
            }
        }
    }
    return false;
}

int solve() {
    int ans = 0;
    for(int u = 1; u <= n; u++) {
        ans += left[u] != 0;
    }
    while(bfs()) {
        fill(it.begin(), it.end(), 0);
        for(int u = 1; u <= n; u++) {
            if(!left[u] && dfs(u)) {
                ++ans;
            }
        }
    }
}

```

2.14 二分图最大匹配 Hopcroft-Karp · 主体 · 续

```

    return ans;
}
// Call after solve(). Return vertices in a minimum vertex cover.
pair<vector<int>, vector<int>> minVertexCover() const {
    vector<bool> a(n + 1), b(m + 1);
    queue<int> q;
    for(int u = 1; u <= n; u++) {
        if(!left[u]) {
            a[u] = true, q.push(u);
        }
    }
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        for(int v : g[u]) {
            if(left[u] != v && !b[v]) {
                b[v] = true;
                if(right[v] && !a[right[v]]) {
                    a[right[v]] = true, q.push(right[v]);
                }
            }
        }
    }
    vector<int> A, B;
    for(int u = 1; u <= n; u++) {
        if(!a[u]) {
            A.push_back(u);
        }
    }
    for(int v = 1; v <= m; v++) {
        if(b[v]) {
            B.push_back(v);
        }
    }
    return {A, B};
}
};

```

最大匹配与最小点覆盖

2.14 · 最大匹配与最小点覆盖

```

int main() {
    HK s(3, 3);
    s.addEdge(1, 1);
    s.addEdge(1, 2);
    s.addEdge(2, 2);
    s.addEdge(3, 2);
    assert(s.solve() == 2);
    auto [a, b] = s.minVertexCover();
    assert(a.size() + b.size() == 2);
    for(int u = 1; u <= 3; u++) {
        if(s.left[u]) {
            cout << u << ' ' << s.left[u] << '\n';
        }
    }
}

```

2.15 二分图最大权匹配 Hungarian

maximum(w) 接受 1-based 矩阵, 返回权重与行匹配, 较小一侧全部匹配; $n > m$ 时内部转置, 原行中未匹配的为 0。
复杂度 $O(\min(n, m)^2 \max(n, m))$ 。负权允许; 缺边不能用 0 代替, 可设足够负值并检查是否选中缺边。

2.15 二分图最大权匹配 Hungarian · 主体

```

using namespace std;
using i64 = long long;
struct Hungarian {
    static pair<i64, vector<int>> maximum(const vector<vector<i64>>& weight) {
        int n = int(weight.size()) - 1;
        int m = n ? int(weight[1].size()) - 1 : 0;
        if(n == 0 || m == 0) {
            return {0, vector<int>(n + 1)};
        }
        if(n <= m) {
            return solve(weight);
        }
        vector<vector<i64>> transposed(m + 1, vector<i64>(n + 1));
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                transposed[j][i] = weight[i][j];
            }
        }
        auto [value, columnToRow] = solve(transposed);
        vector<int> rowToColumn(n + 1);
        for(int column = 1; column <= m; column++) {
            rowToColumn[columnToRow[column]] = column;
        }
        return {value, rowToColumn};
    }
private:
    static pair<i64, vector<int>> solve(const vector<vector<i64>>& weight) {
        int n = int(weight.size()) - 1;
        int m = int(weight[1].size()) - 1;
        const i64 inf = numeric_limits<i64>::max() / 4;
        vector<i64> u(n + 1), v(m + 1);
        vector<int> match(m + 1, previous(m + 1));
        for(int row = 1; row <= n; row++) {
            match[0] = row;
            int column = 0;
            vector<i64> slack(m + 1, inf);
            vector<bool> used(m + 1);
            do {
                used[column] = true;
                int row = match[column];
                i64 delta = inf;
                int nxt = 0;
                for(int j = 1; j <= m; j++) {
                    if(used[j]) {
                        continue;
                    }
                    i64 current = -weight[row][j] - u[row] - v[j];
                    if(current < slack[j]) {
                        slack[j] = current;
                        previous[j] = column;
                    }
                    if(slack[j] < delta) {
                        delta = slack[j];
                        nxt = j;
                    }
                }
            } while(match[nxt] != 0);
            for(int j = 0; j <= m; j++) {
                if(used[j]) {
                    u[match[j]] += delta;
                    v[j] -= delta;
                } else {
                    slack[j] -= delta;
                }
            }
            column = nxt;
        } while(match[column] != 0);
    }
};

```

2.15 二分图最大权匹配 Hungarian · 主体 · 续

```

        do {
            int nxt = previous[column];
            match[column] = match[nxt];
            column = nxt;
        } while(column != 0);
    }
    vector<int> mate(n + 1);
    for(int column = 1; column <= m; column++) {
        if(match[column]) {
            mate[match[column]] = column;
        }
    }
    i64 value = 0;
    for(int row = 1; row <= n; row++) {
        value += weight[row][mate[row]];
    }
    return {value, mate};
}
};

```

2.16 最大流 Dinic

与其他图模板不同：点为 $0..n-1$ 。addEdge 返回正向边在 $g[u]$ 内的下标，cap 为剩余容量，初始容量需自己保存。maxFlow(s,t,limit) 返回本次新增流， $s!=t$ ；可在残量图上继续加边再增广。minCut(s) 返回残量可达集合。一般 $O(V^2E)$ ，单位网络可更快；增广使用显式栈，支持深链。

2.16 最大流 Dinic · 主体

```

using namespace std;
using i64 = long long;
struct Dinic {
    struct Edge {
        int to, rev;
        i64 cap;
    };
    int n;
    vector<vector<Edge>> g;
    vector<int> dep, it;
    Dinic(int n_ = 0) : n(n_), g(n_), dep(n_), it(n_) {}
    int addEdge(int u, int v, i64 cap) {
        int id = int(g[u].size());
        int rev = int(g[v].size()) + (u == v);
        g[u].push_back({v, rev, cap});
        g[v].push_back({u, id, 0});
        return id;
    }
    i64 maxFlow(int s, int t, i64 limit = numeric_limits<i64>::max() / 4) {
        assert(s != t);
        i64 flow = 0;
        while(flow < limit && bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while(flow < limit) {
                i64 pushed = dfs(s, t, limit - flow);
                if(!pushed) {
                    break;
                }
                flow += pushed;
            }
        }
        return flow;
    }
    vector<bool> minCut(int s) const {
        vector<bool> visit(n);
        queue<int> q;

```

2.16 最大流 Dinic · 主体 · 续

```

    visit[s] = true;
    q.push(s);
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        for(const auto& edge : g[u]) {
            if(edge.cap && !visit[edge.to]) {
                visit[edge.to] = true;
                q.push(edge.to);
            }
        }
    }
    return visit;
}

private:
bool bfs(int s, int t) {
    fill(dep.begin(), dep.end(), -1);
    queue<int> q;
    dep[s] = 0;
    q.push(s);
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        for(const auto& edge : g[u]) {
            if(edge.cap && dep[edge.to] == -1) {
                dep[edge.to] = dep[u] + 1;
                q.push(edge.to);
            }
        }
    }
    return dep[t] != -1;
}

i64 dfs(int s, int t, i64 limit) {
    vector<int> st{s};
    vector<pair<int, int>> path;
    vector<i64> val{limit};
    while(!st.empty()) {
        int u = st.back();
        if(u == t) {
            i64 f = val.back();
            for(auto [v, id] : path) {
                auto& e = g[v][id];
                e.cap -= f;
                g[e.to][e.rev].cap += f;
            }
            return f;
        }
        int& i = it[u];
        while(i < int(g[u].size()) && (!g[u][i].cap || dep[g[u][i].to] != dep[u] + 1)) {
            i++;
        }
        if(i == int(g[u].size())) {
            dep[u] = -1;
            st.pop_back();
            val.pop_back();
            if(!path.empty()) {
                int p = path.back().first;
                path.pop_back();
                it[p]++;
            }
        } else {
            path.push_back({u, i});
            st.push_back(g[u][i].to);
            val.push_back(min(val.back(), g[u][i].cap));
        }
    }
}

```

2.16 最大流 Dinic · 主体 · 续

```

    }
    return 0;
}
};

```

读回边流量

2.16 · 读回边流量

```

int main() {
    Dinic s(3);
    int id = s.addEdge(0, 1, 5);
    s.addEdge(1, 2, 3);
    assert(s.maxFlow(0, 2) == 3);
    assert(5 - s.g[0][id].cap == 3);
    s.addEdge(1, 2, 2);
    assert(s.maxFlow(0, 2) == 2); // 本次新增量。
}

```

2.17 最小费用流 MCF

点为 $0..n-1$ ，容量和费用 $i64$ ； $\text{flow}(s,t,\text{limit})$ 返回 (本次新增流,费用)。初始 Bellman-Ford 势能 ($O(V)$) 允许负边，但从源可达处不能有负费用环；否则这个接口不适用。后续用势能 Dijkstra，约 $O(\text{增广次数} \cdot E \log V)$ ，费用总和不能溢出。

要“必须发送 K 流”就检查 $f==K$ ；要最大流最小费用则不传 limit 。仅需负费用增广直到不划算的建模，不能直接把该接口当最大收益可选流。

2.17 最小费用流 MCF · 主体

```

using namespace std;
using i64 = long long;
struct MCF {
    struct Edge {
        int to, rev;
        i64 cap, cost;
    };
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;
    int n;
    vector<vector<Edge>> g;
    MCF(int n_ = 0) : n(n_), g(n_) {}
    void addEdge(int u, int v, i64 cap, i64 cost) {
        int uIndex = int(g[u].size());
        int vIndex = int(g[v].size()) + (u == v);
        g[u].push_back({v, vIndex, cap, cost});
        g[v].push_back({u, uIndex, 0, -cost});
    }
    pair<i64, i64> flow(int s, int t, i64 limit = inf) {
        assert(s != t);
        vector<i64> h = initPot(s);
        vector<i64> dis(n);
        vector<int> pv(n), pe(n);
        i64 flowSum = 0;
        i64 costSum = 0;
        while(flowSum < limit) {
            fill(dis.begin(), dis.end(), inf);
            priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
            dis[s] = 0;
            q.push({0, s});
            while(!q.empty()) {
                auto [du, u] = q.top();
                q.pop();
                if(du != dis[u]) {
                    continue;

```

2.17 最小费用流 MCF · 主体 · 续

```

    }
    for(int i = 0; i < int(g[u].size()); i++) {
        const Edge& edge = g[u][i];
        if(!edge.cap) {
            continue;
        }
        i64 nd = du + edge.cost + h[u] - h[edge.to];
        if(nd < dis[edge.to]) {
            dis[edge.to] = nd;
            pv[edge.to] = u;
            pe[edge.to] = i;
            q.push({nd, edge.to});
        }
    }
}
if(dis[t] == inf) {
    break;
}
for(int u = 0; u < n; u++) {
    if(dis[u] != inf) {
        h[u] += dis[u];
    }
}
i64 pushed = limit - flowSum;
for(int v = t; v != s; v = pv[v]) {
    const Edge& edge = g[pv[v]][pe[v]];
    pushed = min(pushed, edge.cap);
}
for(int v = t; v != s; v = pv[v]) {
    Edge& edge = g[pv[v]][pe[v]];
    costSum += pushed * edge.cost;
    edge.cap -= pushed;
    g[v][edge.rev].cap += pushed;
}
flowSum += pushed;
}
return {flowSum, costSum};
}

private:
vector<i64> initPot(int s) const {
    vector<i64> dis(n, inf);
    dis[s] = 0;
    for(int round = 1; round < n; round++) {
        bool changed = false;
        for(int u = 0; u < n; u++) {
            if(dis[u] == inf) {
                continue;
            }
            for(const Edge& edge : g[u]) {
                if(edge.cap && dis[edge.to] > dis[u] + edge.cost) {
                    dis[edge.to] = dis[u] + edge.cost;
                    changed = true;
                }
            }
        }
        if(!changed) {
            break;
        }
    }
    for(i64& value : dis) {
        if(value == inf) {
            value = 0;
        }
    }
}

```

2.17 最小费用流 MCF · 主体 · 续

```

    }
    return dis;
}
};

```

指定流量的最小费用

2.17 · 指定流量的最小费用

```

int main() {
    MCF s(3);
    s.addEdge(0, 1, 2, -3);
    s.addEdge(1, 2, 2, 5);
    auto [f, c] = s.flow(0, 2, 2);
    assert(f == 2 && c == 4);
}

```

2.18 上下界网络流

BoundFlow(n), 点 $0..n-1$, addEdge(u,v,lo,hi), $0 \leq lo \leq hi$ 。circulation(f) 判断无源汇可行环流;
 maxFlow(s,t,res) 求非负流值的最大可行 $s \rightarrow t$ 流, 返回值表示是否存在该类流。res.flow 按输入边顺序含原下界。两个求解均重新建图, 可多次调用。

下界预流导致 $balance[u] -= lo, balance[v] += lo$; $balance > 0$ 接 $SS \rightarrow u$, < 0 接 $u \rightarrow TT$ 。源汇版本加 $t \rightarrow s$ 的 INF 边, 再跑 $SS \rightarrow TT$ 。可行后读人工边流量为 base, 删掉人工边的正反残量与超级点关联边; 最大再跑 $s \rightarrow t$, 最小跑 $t \rightarrow s$ 且 $limit = base$, 以保持非负流值。总需求和与最大流值均应小于 inf。

minFlow(s,t,res) 同样限制非负流值, 最大/最小结果都返回每条输入边的实际流量。

主体依赖: 2.16 最大流 Dinic。先复制这些基础定义, 再复制下列代码。

2.18 上下界网络流 · 主体

```

struct BoundFlow {
    struct Edge {
        int from, to;
        i64 lo, hi;
    };
    struct Result {
        i64 value;
        vector<i64> flow;
    };
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;
    int n;
    vector<Edge> edges;
    BoundFlow(int n_ = 0) : n(n_) {}
    int addEdge(int from, int to, i64 lo, i64 hi) {
        assert(0 <= lo && lo <= hi);
        int id = int(edges.size());
        edges.push_back({from, to, lo, hi});
        return id;
    }
    bool circulation(vector<i64>& ans) const {
        auto net = build(false, -1, -1);
        if(net.dinic.maxFlow(net.S, net.T) != net.demand) {
            return false;
        }
        ans = recover(net);
        return true;
    }
    bool maxFlow(int s, int t, Result& ans) const {
        auto net = build(true, s, t);
        if(net.dinic.maxFlow(net.S, net.T) != net.demand) {
            return false;
        }
    }
}

```

2.18 上下界网络流 · 主体 · 续

```

    }
    auto& ex = net.dinic.g[t][net.extra];
    i64 base = inf - ex.cap;
    net.dinic.g[ex.to][ex.rev].cap = 0;
    ex.cap = 0;
    delSuper(net);
    i64 extra = net.dinic.maxFlow(s, t);
    ans = Result{base + extra, recover(net)};
    return true;
}

bool minFlow(int s, int t, Result& ans) const {
    assert(s != t);
    auto net = build(true, s, t);
    if(net.dinic.maxFlow(net.S, net.T) != net.demand) {
        return false;
    }
    auto& edge = net.dinic.g[t][net.extra];
    i64 base = inf - edge.cap;
    net.dinic.g[edge.to][edge.rev].cap = 0;
    edge.cap = 0;
    delSuper(net);
    i64 delta = net.dinic.maxFlow(t, s, base);
    ans = {base - delta, recover(net)};
    return true;
}

private:
    struct Net {
        Dinic dinic;
        int S, T;
        i64 demand = 0;
        vector<pair<int, int>> pos;
        int extra = -1;
    };

    Net build(bool addExtra, int s, int t) const {
        Net net;
        net.dinic = Dinic(n + 2);
        net.S = n;
        net.T = n + 1;
        vector<i64> balance(n);
        for(int i = 0; i < int(edges.size()); i++) {
            const Edge& edge = edges[i];
            int index = net.dinic.addEdge(edge.from, edge.to, edge.hi - edge.lo);
            net.pos.push_back({edge.from, index});
            balance[edge.from] -= edge.lo;
            balance[edge.to] += edge.lo;
        }
        if(addExtra) {
            net.extra = net.dinic.addEdge(t, s, inf);
        }
        for(int u = 0; u < n; u++) {
            if(balance[u] > 0) {
                net.dinic.addEdge(net.S, u, balance[u]);
                net.demand += balance[u];
            } else if(balance[u] < 0) {
                net.dinic.addEdge(u, net.T, -balance[u]);
            }
        }
        return net;
    }

    vector<i64> recover(const Net& net) const {
        vector<i64> result(edges.size());
        for(int i = 0; i < int(edges.size()); i++) {
            auto [u, index] = net.pos[i];
            const auto& edge = net.dinic.g[u][index];
            result[i] = edges[i].hi - edge.cap;
        }
    }

```

2.18 上下界网络流 · 主体 · 续

```

        return result;
    }
    static void delSuper(Net& net) {
        for(int u = 0; u < net.dinic.n; u++) {
            for(auto& edge : net.dinic.g[u]) {
                if(u == net.S || u == net.T || edge.to == net.S || edge.to == net.T) {
                    edge.cap = 0;
                }
            }
        }
    }
};

```

可行环流 / 最大流 / 最小流

2.18 · 可行环流 / 最大流 / 最小流

```

int main() {
    BoundFlow s(3);
    s.addEdge(0, 1, 1, 3);
    s.addEdge(1, 2, 1, 2);
    BoundFlow::Result a, b;
    assert(s.maxFlow(0, 2, a) && a.value == 2);
    assert(s.minFlow(0, 2, b) && b.value == 1);
    assert(a.flow[0] == 2 && b.flow[1] == 1);
    vector<long long> f;
    assert(!s.circulation(f)); // 此图没有回边，无法形成环流。
}

```

2.19 差分约束

$le(u,v,w)$ 就是 $x_v - x_u \leq w$ ，建 $u \rightarrow v$ 权 w ； $x_v - x_u \geq w$ 改为 $le(v,u,-w)$ ，等式 $eq(u,v,w)$ 。整数严格小于 w 改成 $\leq w-1$ 。所有点初距为 0 等价超级源，能查出任意分量负环； $solve(d)$ 返回 bool，解用 $i128$ 。SPFA 最坏 $O(VE)$ ， $solve(d,false)$ 为确定轮次的 Bellman-Ford。

得到的是一组可行势能；若要让 $x_1=0$ ，可把全部值减 $d[1]$ ，不改变差。要求最小/最大某变量时必须按边方向推导界，不能把任意可行解当最优解。

2.19 差分约束 · 主体

```

using namespace std;
struct Diff {
    using i64 = long long;
    using i128 = __int128_t;
    struct Edge {
        int u, v;
        i64 w;
    };
    int n;
    vector<Edge> edges;
    vector<vector<int>>> g;
    Diff(int n_) : n(n_), g(n_ + 1) {}
    //  $x[v] - x[u] \leq w$ 
    void le(int u, int v, i64 w) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w});
    }
    void eq(int u, int v, i64 w) {
        assert(w != LLONG_MIN);
        le(u, v, w);
        le(v, u, -w);
    }
    bool solve(vector<i128>& d, bool spfa = true) const {
        d.assign(n + 1, 0); // Implicit super s to every vertex.
    }
};

```

2.19 差分约束 · 主体 · 续

```

    if(!spfa) {
        for(int round = 1; round <= n; round++) {
            bool changed = false;
            for(auto [u, v, w] : edges) {
                if(d[v] > d[u] + w) {
                    d[v] = d[u] + w;
                    changed = true;
                    if(round == n) {
                        return false;
                    }
                }
            }
            if(!changed) {
                break;
            }
        }
        return true;
    }
    queue<int> q;
    vector<bool> in(n + 1, true);
    vector<int> len(n + 1);
    for(int u = 1; u <= n; u++) {
        q.push(u);
    }
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        in[u] = false;
        for(int id : g[u]) {
            auto [from, v, w] = edges[id];
            if(d[v] <= d[u] + w) {
                continue;
            }
            d[v] = d[u] + w;
            len[v] = len[u] + 1;
            if(len[v] >= n) {
                return false;
            }
            if(!in[v]) {
                in[v] = true, q.push(v);
            }
        }
    }
    return true;
}
};

```

把不等式翻译成边

2.19 · 把不等式翻译成边

```

int main() {
    Diff s(3);
    s.le(2, 1, -3); //  $x_2 \geq x_1 + 3$ 。
    s.le(2, 3, 2); //  $x_3 \leq x_2 + 2$ 。
    s.eq(1, 3, 4); //  $x_3 = x_1 + 4$ 。
    vector<__int128> d;
    assert(s.solve(d));
    assert(d[2] - d[1] >= 3 && d[3] - d[1] == 4);
    s.le(1, 3, 3);
    assert(!s.solve(d));
}

```

2.20 函数图

输入 $to[1..n]$ 且每点恰好一条出边。FuncGraph 预处理 $O(64n)$, $jump(u,k)$ 支持完整 $uint64_t$ 步数; $dis(u,v)$ 为沿出边到 v 的最少步数, 不可达 -1 。区分树上祖先、入环点和同环偏移, 不能套无向距离。

2.20 函数图 · 主体

```
using namespace std;
struct FuncGraph {
    int n;
    vector<int> to, cycle, depth, entry, pos;
    vector<vector<int>> rings, up;
    FuncGraph(const vector<int>& next)
        : n(next.size() - 1), to(next), cycle(n + 1, -1), depth(n + 1), entry(n + 1),
          pos(n + 1), up(64, next) {
        vector<int> deg(n + 1), order;
        for(int u = 1; u <= n; u++) {
            ++deg[to[u]];
        }
        queue<int> q;
        for(int u = 1; u <= n; u++) {
            if(!deg[u]) {
                q.push(u);
            }
        }
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);
            if(--deg[to[u]]) {
                q.push(to[u]);
            }
        }
        for(int u = 1; u <= n; u++) {
            if(deg[u] && cycle[u] < 0) {
                int id = rings.size(), v = u;
                rings.push_back({});
                do {
                    cycle[v] = id;
                    entry[v] = v;
                    pos[v] = rings.back().size();
                    rings.back().push_back(v);
                    v = to[v];
                } while(v != u);
            }
        }
        reverse(order.begin(), order.end());
        for(int u : order) {
            int v = to[u];
            cycle[u] = cycle[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        for(int j = 1; j < 64; j++) {
            for(int u = 1; u <= n; u++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
        }
    }
    int jump(int u, uint64_t k) const {
        for(int j = 0; j < 64; j++) {
            if(k >> j & 1) {
                u = up[j][u];
            }
        }
        return u;
    }
    // Minimum number of directed steps from u to v; -1 if unreachable.
    long long dis(int u, int v) const {
```

2.20 函数图 · 主体 · 续

```

    if(cycle[u] != cycle[v]) {
        return -1;
    }
    if(depth[v]) {
        if(depth[u] < depth[v]) {
            return -1;
        }
        int k = depth[u] - depth[v];
        return jump(u, k) == v ? k : -1;
    }
    int c = cycle[u], len = rings[c].size();
    return depth[u] + (pos[v] - pos[entry[u]] + len) % len;
}
};

```

2.21 线段树优化建图

RangeGraph(n) 中原点始终为 1..n; pointRange、rangePoint、rangeRange 均为闭区间，有向边且权非负。辅助点内部创建，distances(s) 只返回原点距离。出树父→子、入树子→父，叶子共用原点，区间到区间用一个中转点只支付一次权值。边 $O(n+q \log n)$ ，点 $O(n+q)$ 。

在原点周围走辅助零边只代表集合覆盖；入树与出树方向相反，写错会凭空把一个区间里的点互相连通。

2.21 线段树优化建图 · 主体

```

using namespace std;
struct RangeGraph {
    using i64 = long long;
    static constexpr i64 INF = LLONG_MAX / 4;
    int n;
    vector<int> down, up;
    vector<vector<pair<int, i64>>> g;
    RangeGraph(int n_) : n(n_), down(4 * max(1, n_)), up(down.size()), g(n_ + 1) {
        assert(n > 0);
        build(1, 1, n);
    }
    int node() {
        g.emplace_back();
        return int(g.size()) - 1;
    }
    void edge(int u, int v, i64 w) {
        g[u].push_back({v, w});
    }
    void build(int p, int l, int r) {
        if(l == r) {
            down[p] = up[p] = 1;
            return;
        }
        down[p] = node();
        up[p] = node();
        int m = (l + r) / 2;
        build(p * 2, l, m);
        build(p * 2 + 1, m + 1, r);
        for(int c : {p * 2, p * 2 + 1}) {
            edge(down[p], down[c], 0), edge(up[c], up[p], 0);
        }
    }
    void link(int p, int l, int r, int x, int y, int v, i64 w, bool fromPoint) {
        if(x <= l && r <= y) {
            if(fromPoint) {
                edge(v, down[p], w);
            } else {
                edge(up[p], v, w);
            }
        }
        return;
    }
};

```

2.21 线段树优化建图 · 主体 · 续

```

    }
    int m = (l + r) / 2;
    if(x <= m) {
        link(p * 2, l, m, x, y, v, w, fromPoint);
    }
    if(y > m) {
        link(p * 2 + 1, m + 1, r, x, y, v, w, fromPoint);
    }
}
void pointRange(int v, int l, int r, i64 w) {
    link(1, 1, n, l, r, v, w, true);
}
void rangePoint(int l, int r, int v, i64 w) {
    link(1, 1, n, l, r, v, w, false);
}
void rangeRange(int l1, int r1, int l2, int r2, i64 w) {
    int v = node();
    rangePoint(l1, r1, v, 0);
    pointRange(v, l2, r2, w);
}
vector<i64> distances(int s) const {
    vector<i64> d(g.size(), INF);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while(!q.empty()) {
        auto [du, u] = q.top();
        q.pop();
        if(du != d[u]) {
            continue;
        }
        for(auto [v, w] : g[u]) {
            if(d[v] > du + w) {
                d[v] = du + w, q.push({d[v], v});
            }
        }
    }
    d.resize(n + 1);
    return d;
}
};

```

点 / 区间到区间的有向边

2.21 · 点 / 区间到区间的有向边

```

int main() {
    RangeGraph s(6);
    s.pointRange(1, 2, 3, 5); // 1→[2,3], 每条权5。
    s.rangePoint(2, 3, 4, 2); // [2,3]→4, 权2。
    s.rangeRange(4, 4, 5, 6, 1);
    auto d = s.distances(1);
    assert(d[2] == 5 && d[4] == 7 && d[6] == 8);
}

```

2.22 增量最大流 [速查]

见 Dinic 的残量图接口：保留同一对象，加入新边或增加正向剩余容量后，再调用 `maxFlow` 得到新增量，累计到原答案。容量减小或删除边可能破坏可行流，不能直接复用。每轮遍历游标由算法重置。

2.23 严格次小生成树

`SecondMST::solve(n,e,mst,second);` `false` 表示图不连通，`second==INF` 表示不存在权重严格更大的生成树。树路径维护最大与严格次大边：候选边等于最大时要删次大。负权、重边允许； $O(m \log m + (n+m) \log n)$ 。

2.23 严格次小生成树 · 主体

```

using namespace std;
struct SecondMST {
    typedef long long ll;
    static const ll INF = LLONG_MAX / 4;
    struct Edge {
        int u, v;
        ll w;
    };
    struct Two {
        ll a, b;
        Two(ll x = -INF, ll y = -INF) : a(x), b(y) {}
    };
};
static Two join(Two x, Two y) {
    ll v[4] = {x.a, x.b, y.a, y.b};
    Two z;
    for(int i = 0; i < 4; i++) {
        if(v[i] > z.a) {
            z.b = z.a, z.a = v[i];
        } else if(v[i] < z.a && v[i] > z.b) {
            z.b = v[i];
        }
    }
    return z;
}
// false: disconnected; second=INF: no strictly heavier spanning tree.
static bool solve(int n, vector<Edge> e, ll& mst, ll& second) {
    vector<int> p(n + 1), id(e.size()), used(e.size());
    vector<vector<pair<int, ll>>> g(n + 1);
    iota(p.begin(), p.end(), 0);
    iota(id.begin(), id.end(), 0);
    function<int(int)> find = [&](int x) { return p[x] == x ? x : p[x] = find(p[x]); };
    sort(id.begin(), id.end(), [&](int x, int y) { return e[x].w < e[y].w; });
    mst = 0;
    second = INF;
    int cnt = 0;
    for(int k : id) {
        int x = find(e[k].u), y = find(e[k].v);
        if(x == y) {
            continue;
        }
        p[x] = y;
        used[k] = 1;
        mst += e[k].w;
        ++cnt;
        g[e[k].u].push_back(make_pair(e[k].v, e[k].w));
        g[e[k].v].push_back(make_pair(e[k].u, e[k].w));
    }
    if(cnt != n - 1) {
        return false;
    }
    int lg = 1;
    while((1LL << lg) <= n) {
        ++lg;
    }
    vector<int> dep(n + 1), ord(1, 1);
    vector<vector<int>> fa(lg, vector<int>(n + 1));
    vector<vector<Two>> mx(lg, vector<Two>(n + 1));
    for(size_t i = 0; i < ord.size(); i++) {
        int u = ord[i];
        for(size_t j = 0; j < g[u].size(); j++) {
            int v = g[u][j].first;
            if(v == fa[0][u]) {
                continue;
            }
            fa[0][v] = u;
            dep[v] = dep[u] + 1;

```

2.23 严格次小生成树 · 主体 · 续

```

        mx[0][v] = Two(g[u][j].second);
        ord.push_back(v);
    }
}
for(int j = 1; j < lg; j++) {
    for(int u = 1; u <= n; u++) {
        fa[j][u] = fa[j - 1][fa[j - 1][u]];
        mx[j][u] = join(mx[j - 1][u], mx[j - 1][fa[j - 1][u]]);
    }
}
for(size_t k = 0; k < e.size(); k++) {
    if(!used[k] && e[k].u != e[k].v) {
        int u = e[k].u, v = e[k].v;
        Two z;
        if(dep[u] < dep[v]) {
            swap(u, v);
        }
        int d = dep[u] - dep[v];
        for(int j = 0; j < lg; j++) {
            if(d >> j & 1) {
                z = join(z, mx[j][u]), u = fa[j][u];
            }
        }
        if(u != v) {
            for(int j = lg - 1; j >= 0; j--) {
                if(fa[j][u] != fa[j][v]) {
                    z = join(z, join(mx[j][u], mx[j][v]));
                    u = fa[j][u];
                    v = fa[j][v];
                }
            }
            z = join(z, join(mx[0][u], mx[0][v]));
        }
        ll rem = e[k].w > z.a ? z.a : z.b;
        if(rem != -INF) {
            second = min(second, mst + e[k].w - rem);
        }
    }
}
return true;
}
};

```

2.24 隐式 XOR 完全图 MST

XorMST::run(a) 返回 (权重, 边列表), a 为 0-based uint32_t, 边端点同为 0-based。Boruvka 每轮 Trie 保存两个来自不同分量的代表, 查询最小外部异或边。重复值与第 31 位均支持; $O(32n \log n)$, 避免枚举 n^2 条边。

2.24 隐式 XOR 完全图 MST · 主体

```

using namespace std;
// Complete graph on values a[0..n-1], edge weight a[u] XOR a[v].
struct XorMST {
    struct Node {
        int ch[2]{-1, -1};
        int a = -1, b = -1;
    };
    static pair<long long, vector<pair<int, int>>> run(const vector<uint32_t>& a) {
        int n = a.size();
        vector<int> p(n), sz(n, 1), color(n);
        iota(p.begin(), p.end(), 0);
        auto find = [&](int u) {
            while(u != p[u]) {
                u = p[u] = p[p[u]];
            }
            return u;
        };
    };
};

```

2.24 隐式 XOR 完全图 MST · 主体 · 续

```

};
long long ans = 0;
int groups = n;
vector<pair<int, int>> chosen;
while(groups > 1) {
    for(int u = 0; u < n; u++) {
        color[u] = find(u);
    }
    vector<Node> t(1);
    auto mark = [&](int k, int u) {
        if(t[k].a < 0) {
            t[k].a = u;
        } else if(color[t[k].a] != color[u]) {
            t[k].b = u;
        }
    };
    for(int u = 0; u < n; u++) {
        int k = 0;
        mark(k, u);
        for(int b = 31; b >= 0; b--) {
            int c = a[u] >> b & 1;
            if(t[k].ch[c] < 0) {
                int v = t.size();
                t.emplace_back();
                t[k].ch[c] = v;
            }
            k = t[k].ch[c];
            mark(k, u);
        }
    }
    auto good = [&](int k, int u) {
        return k >= 0 && ((t[k].a >= 0 && color[t[k].a] != color[u]) ||
            (t[k].b >= 0 && color[t[k].b] != color[u]));
    };
    vector<pair<int, int>> best(n, {-1, -1});
    for(int u = 0; u < n; u++) {
        int k = 0;
        for(int b = 31; b >= 0; b--) {
            int c = a[u] >> b & 1;
            k = t[k].ch[good(t[k].ch[c], u) ? c : c ^ 1];
        }
        int v = color[t[k].a] != color[u] ? t[k].a : t[k].b;
        auto& e = best[color[u]];
        if(e.first < 0 || (a[u] ^ a[v]) < (a[e.first] ^ a[e.second])) {
            e = {u, v};
        }
    }
    for(auto [u, v] : best) {
        if(u >= 0) {
            int x = find(u), y = find(v);
            if(x == y) {
                continue;
            }
            if(sz[x] < sz[y]) {
                swap(x, y);
            }
            p[y] = x;
            sz[x] += sz[y];
            --groups;
            ans += (a[u] ^ a[v]);
            chosen.push_back({u, v});
        }
    }
}
return {ans, chosen};
};

```

2.25 最大权闭合子图

权重 w 为 1-based, 依赖 (u,v) 表示选 u 必须选 v 。正权连 $S \rightarrow u$, 负权连 $u \rightarrow T$, 依赖 $u \rightarrow v$ 连 INF; 答案=正权和-最小割。maxClosure 返回权重与选点。INF=正权和+1, 不能随便取小常数; 本版要求正权和 $< \text{LLONG_MAX}/4$ 。

主体依赖: 2.16 最大流 Dinic。先复制这些基础定义, 再复制下列代码。

2.25 最大权闭合子图 · 主体

```
pair<i64, vector<int>> maxClosure(const vector<i64>& w, const vector<pair<int, int>>& dep) {
    int n = int(w.size()) - 1, s = 0, t = n + 1;
    Dinic f(n + 2);
    i64 sum = 0;
    for(int u = 1; u <= n; u++) {
        if(w[u] > 0) {
            sum += w[u];
        }
    }
    assert(sum < LLONG_MAX / 4);
    for(int u = 1; u <= n; u++) {
        if(w[u] > 0) {
            f.addEdge(s, u, w[u]);
        }
        if(w[u] < 0) {
            f.addEdge(u, t, -w[u]);
        }
    }
    for(auto [u, v] : dep) {
        f.addEdge(u, v, sum + 1);
    }
    i64 ans = sum - f.maxFlow(s, t);
    auto vis = f.minCut(s);
    vector<int> chosen;
    for(int u = 1; u <= n; u++) {
        if(vis[u]) {
            chosen.push_back(u);
        }
    }
    return {ans, chosen};
}
```

2.26 Boruvka 最小生成森林

Boruvka::run(n,e), 点 $1..n$, 返回 weight、edges (1-based 输入边编号)、comp。每轮按轮初分量选最便宜外连边, 再统一合并; 平权用编号稳定比较, 合并前再次判同根防重复。显式图 $O(m \log n)$ 。隐式图的关键是实现每分量最便宜外连边查询, 不能只打印一个 findBest 空函数。

2.26 Boruvka 最小生成森林 · 主体

```
using namespace std;
struct Boruvka {
    using i64 = long long;
    struct Edge {
        int u, v;
        i64 w;
    };
    struct Result {
        i64 weight = 0;
        vector<int> edges;
        int comp = 0;
    };
    static Result run(int n, const vector<Edge>& e) {
        vector<int> p(n + 1), sz(n + 1, 1), best(n + 1);
        iota(p.begin(), p.end(), 0);
        auto find = [&](int u) {
            while(u != p[u]) {

```

2.26 Boruvka 最小生成森林 · 主体 · 续

```

        u = p[u] = p[p[u]];
    }
    return u;
};
Result ans;
ans.comp = n;
auto improve = [&](int r, int i) {
    if(best[r] < 0 || pair{e[i].w, i} < pair{e[best[r]].w, best[r]}) {
        best[r] = i;
    }
};
while(ans.comp > 1) {
    fill(best.begin(), best.end(), -1);
    // Component partition is frozen throughout candidate selection.
    for(int i = 0; i < (int)e.size(); i++) {
        int u = find(e[i].u), v = find(e[i].v);
        if(u != v) {
            improve(u, i), improve(v, i);
        }
    }
    int merged = 0;
    for(int r = 1; r <= n; r++) {
        if(best[r] >= 0) {
            int i = best[r], u = find(e[i].u), v = find(e[i].v);
            if(u == v) {
                continue;
            }
            if(sz[u] < sz[v]) {
                swap(u, v);
            }
            p[v] = u;
            sz[u] += sz[v];
            ans.weight += e[i].w;
            ans.edges.push_back(i + 1);
            --ans.comp;
            ++merged;
        }
    }
    if(!merged) {
        break;
    }
}
return ans;
}
};

```

森林返回值也要检查

2.26 · 森林返回值也要检查

```

int main() {
    vector<Boruvka::Edge> e = {{1, 2, 4}, {2, 3, 1}, {1, 3, 2}};
    auto a = Boruvka::run(4, e);
    assert(a.weight == 3 && a.comp == 2);
    assert(a.edges.size() == 2); // 第4点孤立, 不能当生成树。
}

```

3. 树上算法

3.1 树的直径

非负边权的连通树，TreeDiameter::run(g) 两次找最远点，返回 a、b、dis，O(n)。允许零边权；负权时两次最远点法可能失败，需树形 DP。遍历用显式栈。

3.1 树的直径 · 主体

```
using namespace std;
using i64 = long long;
struct TreeDiameter {
    struct Edge {
        int to;
        i64 w;
    };
    struct Result {
        int a, b;
        i64 dis;
    };
    static Result run(const vector<vector<Edge>>& g) {
        if(g.size() <= 1) {
            return {0, 0, 0};
        }
        auto farthest = [&](int s) {
            pair<i64, int> best{0, s};
            vector<tuple<int, int, i64>> stk{{s, 0, 0}};
            while(!stk.empty()) {
                auto [u, p, d] = stk.back();
                stk.pop_back();
                best = max(best, pair{d, u});
                for(auto [v, w] : g[u]) {
                    if(v != p) {
                        stk.push_back({v, u, d + w});
                    }
                }
            }
            return best;
        };
        int a = farthest(1).second;
        auto [d, b] = farthest(a);
        return {a, b, d};
    }
};
```

3.2 树的重心

删除后每块大小 $\leq n/2$ ，至多两个；centroids(g) 返回全部重心，O(n)。这是点数重心，不是直径中心，也不是按边权定义的中心。

3.2 树的重心 · 主体

```
using namespace std;
vector<int> centroids(const vector<vector<int>>& g) {
    int n = int(g.size()) - 1;
    if(n <= 0) {
        return {};
    }
    vector<int> p(n + 1), sz(n + 1, 1), order{1}, ans;
    for(int i = 0; i < int(order.size()); i++) {
        int u = order[i];
        for(int v : g[u]) {
            if(v != p[u]) {
                p[v] = u, order.push_back(v);
            }
        }
    }
}
```

3.2 树的重心 · 主体 · 续

```

    }
    for(int i = n - 1; i >= 0; i--) {
        int u = order[i], mx = 0;
        for(int v : g[u]) {
            if(v != p[u]) {
                sz[u] += sz[v], mx = max(mx, sz[v]);
            }
        }
        mx = max(mx, n - sz[u]);
        if(mx * 2 <= n) {
            ans.push_back(u);
        }
    }
    return ans;
}

```

3.3 Kruskal 重构树

KRT(n, e), 原点为叶子 $1..n$, 合并点编号 $> n$, 权为合并边权。bottle(u, v, ans) 查询原图 $u-v$ 路径最小可能的最大边; 跨分量 false, $u=v$ 约定 0。构造 $O(m \log m + n \log n)$, 查询 $O(\log n)$, 树高可为 n , 预处理已用显式栈。

3.3 Kruskal 重构树 · 主体

```

using namespace std;
using i64 = long long;
struct KRT {
    struct Edge {
        int u, v;
        i64 w;
    };
    int n, tot, log;
    vector<i64> weight;
    vector<array<int, 2>> child;
    vector<int> depth, bel;
    vector<vector<int>> up;
    KRT(int n_, vector<Edge> edges) : n(n_) {
        int cap = max(2, 2 * n + 1);
        vector<int> dsu(cap), root(cap);
        iota(dsu.begin(), dsu.end(), 0);
        iota(root.begin(), root.end(), 0);
        weight.assign(cap, 0);
        child.assign(cap, {0, 0});
        auto find = [&](int x) {
            int r = x;
            while(r != dsu[r]) {
                r = dsu[r];
            }
            while(x != r) {
                int y = dsu[x];
                dsu[x] = r;
                x = y;
            }
            return r;
        };
        sort(edges.begin(), edges.end(),
            [](const auto& a, const auto& b) { return a.w < b.w; });
        tot = n;
        for(auto [u, v, w] : edges) {
            int x = find(u);
            int y = find(v);
            if(x == y) {
                continue;
            }
            int z = ++tot;
            weight[z] = w;
            child[z] = {root[x], root[y]};
        }
    }
};

```

3.3 Kruskal 重构树 · 主体 · 续

```

        dsu[x] = dsu[y] = dsu[z] = z;
        root[z] = z;
    }
    weight.resize(tot + 1);
    child.resize(tot + 1);
    log = (32 - __builtin_clz((unsigned)max(1, tot)));
    depth.assign(tot + 1, 0);
    bel.assign(tot + 1, 0);
    up.assign(log, vector<int>(tot + 1, 0));
    for(int i = 1; i <= n; i++) {
        int r = find(i);
        if(bel[r] != 0) {
            continue;
        }
        bel[r] = r;
        vector<int> stk{r};
        while(!stk.empty()) {
            int u = stk.back();
            stk.pop_back();
            bel[u] = r;
            for(int j = 1; j < log; j++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
            for(int v : child[u]) {
                if(v) {
                    up[0][v] = u;
                    depth[v] = depth[u] + 1;
                    stk.push_back(v);
                }
            }
        }
    }
}

int lca(int u, int v) const {
    if(depth[u] < depth[v]) {
        swap(u, v);
    }
    int d = depth[u] - depth[v];
    for(int j = 0; j < log; j++) {
        if(d >> j & 1) {
            u = up[j][u];
        }
    }
    if(u == v) {
        return u;
    }
    for(int j = log - 1; j >= 0; j--) {
        if(up[j][u] != up[j][v]) {
            u = up[j][u];
            v = up[j][v];
        }
    }
    return up[0][u];
}

bool bottle(int u, int v, i64& ans) const {
    if(bel[u] != bel[v]) {
        return false;
    }
    if(u == v) {
        ans = 0;
        return true;
    }
    ans = weight[lca(u, v)];
    return true;
}
};

```

路径瓶颈查询

3.3 · 路径瓶颈查询

```
int main() {
    KRT s(4, {{1, 2, 4}, {2, 3, 7}, {1, 3, 10}});
    long long w;
    assert(s.bottle(1, 3, w) && w == 7);
    assert(!s.bottle(1, 4, w));
}
```

3.4 倍增 LCA

连通无向树邻接表 1-based, LCA(g, root); lca、jump、dis、isAnc. tin/tout 为 0-based 半开子树区间 [tin, tout), order 是 DFS 先序; jump 超过根深度返回 0. 预处理 $O(n \log n)$, 查询 $O(\log n)$, 支持深链。

3.4 倍增 LCA · 主体

```
using namespace std;
struct LCA {
    int n, log;
    vector<int> depth, tin, tout, parent, order;
    vector<vector<int>>> up;
    LCA(const vector<vector<int>>& g, int root = 1)
        : n(int(g.size()) - 1), log(1), depth(n + 1), tin(n + 1), tout(n + 1),
          parent(n + 1) {
        while((1LL << log) <= n) {
            log++;
        }
        up.assign(log, vector<int>(n + 1));
        if(!n) {
            return;
        }
        vector<int> stk{root}, sz(n + 1, 1);
        while(!stk.empty()) {
            int u = stk.back();
            stk.pop_back();
            tin[u] = order.size();
            order.push_back(u);
            up[0][u] = parent[u];
            for(int j = 1; j < log; j++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
            for(int i = int(g[u].size()) - 1; i >= 0; i--) {
                int v = g[u][i];
                if(v == parent[u]) {
                    continue;
                }
                parent[v] = u;
                depth[v] = depth[u] + 1;
                stk.push_back(v);
            }
        }
        for(int i = int(order.size()) - 1; i >= 0; i--) {
            int u = order[i];
            tout[u] = tin[u] + sz[u];
            if(parent[u]) {
                sz[parent[u]] += sz[u];
            }
        }
    }
    bool isAnc(int u, int v) const {
        return tin[u] <= tin[v] && tin[v] < tout[u];
    }
    int jump(int u, int k) const {
        if(k > depth[u]) {
            return 0;
        }
    }
```

3.4 倍增 LCA · 主体 · 续

```

    for(int j = 0; j < log; j++) {
        if(k >> j & 1) {
            u = up[j][u];
        }
    }
    return u;
}
int lca(int u, int v) const {
    if(depth[u] < depth[v]) {
        swap(u, v);
    }
    u = jump(u, depth[u] - depth[v]);
    if(u == v) {
        return u;
    }
    for(int j = log - 1; j >= 0; j--) {
        if(up[j][u] != up[j][v]) {
            u = up[j][u], v = up[j][v];
        }
    }
    return parent[u];
}
int dis(int u, int v) const {
    return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}
};

```

3.5 重链剖分 HLD

HLD(n) 的 n 就是点数；addEdge 后仅调用一次 work(root)。in 从 1 起，out 为半开端点，子树闭区间是 $[in[u], out[u]-1]$ ；seq 把 DFS 下标映回点。根固定于 work，rootedParent/Size/Lca 支持临时换根查询。路径拆链 $O(\log n)$ ，接线段树为 $O(\log^2 n)$ 。

边权存较深端点，最后同链段改为 $[in[lca]+1, in[另一点]]$ ，空段跳过；跨链仍包含链头。非交换信息需要分别积累 u/v 两端并反转方向，不能像求和这样随意交换。

3.5 重链剖分 HLD · 主体

```

using namespace std;
struct HLD {
    int n, cur;
    vector<int> siz, top, dep, parent, in, out, seq;
    vector<vector<int>>> adj;
    HLD() {}
    HLD(int n) {
        init(n);
    }
    void init(int n) {
        this->n = n;
        siz.assign(n + 1, 0);
        top.assign(n + 1, 0);
        dep.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        in.assign(n + 1, 0);
        out.assign(n + 1, 0);
        seq.assign(n + 1, 0);
        adj.assign(n + 1, {});
        cur = 1;
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void work(int root = 1) {

```

3.5 重链剖分 HLD · 主体 · 续

```

vector<int> order{root};
parent[root] = 0;
dep[root] = 0;
cur = 1;
for(int i = 0; i < int(order.size()); i++) {
    int u = order[i];
    if(parent[u]) {
        adj[u].erase(find(adj[u].begin(), adj[u].end(), parent[u]));
    }
    for(int v : adj[u]) {
        parent[v] = u, dep[v] = dep[u] + 1, order.push_back(v);
    }
}
for(int i = int(order.size()) - 1; i >= 0; i--) {
    int u = order[i];
    siz[u] = 1;
    int best = -1;
    for(int j = 0; j < int(adj[u].size()); j++) {
        int v = adj[u][j];
        siz[u] += siz[v];
        if(best == -1 || siz[v] > siz[adj[u][best]]) {
            best = j;
        }
    }
    if(best >= 0) {
        swap(adj[u][0], adj[u][best]);
    }
}
vector<int> stk{root};
top[root] = root;
while(!stk.empty()) {
    int u = stk.back();
    stk.pop_back();
    in[u] = cur;
    seq[cur++] = u;
    out[u] = in[u] + siz[u];
    for(int i = int(adj[u].size()) - 1; i >= 0; i--) {
        int v = adj[u][i];
        top[v] = i == 0 ? top[u] : v;
        stk.push_back(v);
    }
}
}
int lca(int u, int v) const {
    while(top[u] != top[v]) {
        if(dep[top[u]] > dep[top[v]]) {
            u = parent[top[u]];
        } else {
            v = parent[top[v]];
        }
    }
    return dep[u] < dep[v] ? u : v;
}
int dist(int u, int v) const {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
}
int jump(int u, int k) const {
    if(dep[u] < k) {
        return 0;
    }
    int d = dep[u] - k;
    while(dep[top[u]] > d) {
        u = parent[top[u]];
    }
    return seq[in[u] - dep[u] + d];
}
bool isAnc(int u, int v) const {

```

3.5 重链剖分 HLD · 主体 · 续

```

    return in[u] <= in[v] && in[v] < out[u];
}
int rootedParent(int root, int u) const {
    if(root == u) {
        return u;
    }
    if(!isAnc(u, root)) {
        return parent[u];
    }
    auto it = upper_bound(adj[u].begin(), adj[u].end(), root,
        [&](int x, int y) { return in[x] < in[y]; });
    return *prev(it);
}
int rootedSize(int root, int u) const {
    if(root == u) {
        return n;
    }
    if(!isAnc(u, root)) {
        return siz[u];
    }
    return n - siz[rootedParent(root, u)];
}
int rootedLca(int a, int b, int root) const {
    return lca(a, b) ^ lca(b, root) ^ lca(root, a);
}
};

```

点权路径求和：连接 Seg

例子还需要：1.7 线段树 Seg。

3.5 · 点权路径求和：连接 Seg

```

struct Info {
    long long sum = 0;
    Info operator+(Info b) const {
        return {sum + b.sum};
    }
};
int main() {
    HLD h(4);
    h.addEdge(1, 2);
    h.addEdge(1, 3);
    h.addEdge(2, 4);
    h.work();
    vector<Info> a(5);
    for(int u = 1; u <= 4; u++) {
        a[h.in[u]] = {u};
    }
    Seg<Info> s(a);
    auto query = [&](int u, int v) {
        long long ans = 0;
        while(h.top[u] != h.top[v]) {
            if(h.dep[h.top[u]] < h.dep[h.top[v]]) {
                swap(u, v);
            }
            ans += s.query(h.in[h.top[u]], h.in[u]).sum;
            u = h.parent[h.top[u]];
        }
        if(h.dep[u] > h.dep[v]) {
            swap(u, v);
        }
        return ans + s.query(h.in[u], h.in[v]).sum;
    };
    assert(query(4, 3) == 10);
}

```

3.6 树上差分

TreeDiff 基于 LCA。addVertexPath(d,u,v,w) 用 $d[u] += w, d[v] += w, d[lca] -= w, d[parent(lca)] -= w$ ；边差分在 lca 减 $2w$ 。所有操作完成后 accumulate(d,parent,order) 自底向上一次汇总。边权存在较深端点；传给 vector<i64> 的权值用 i64 或 7LL。

主体依赖：3.4 倍增 LCA。先复制这些基础定义，再复制下列代码。

3.6 树上差分 · 主体

```
struct TreeDiff : LCA {
    TreeDiff(const vector<vector<int>>& adj, int root = 1) : LCA(adj, root) {}
}
template <class T> void addVertexPath(vector<T>& diff, int u, int v, T w) {
    int p = lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[p] -= w;
    if(parent[p] != 0) {
        diff[parent[p]] -= w;
    }
}
template <class T> void addEdgePath(vector<T>& diff, int u, int v, T w) {
    int p = lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[p] -= w + w;
}
template <class T>
static void accumulate(vector<T>& diff, const vector<int>& fa,
                       const vector<int>& order) {
    for(int i = int(order.size()) - 1; i > 0; i--) {
        diff[fa[order[i]]] += diff[order[i]];
    }
}
};
```

3.7 虚树

原树 LCA 只预处理一次；build(keys) 返回 (根,父子边)。先按 DFS 序排序，加相邻 LCA，再去重，点数 $\leq 2k-1$ ；空集根0。边表示原树上一整段路径，无权距离为 $dis(u,v)$ ，加权距离用根距差，边上 min/max 要另查原树。每次只清理触及点，不能每询问清空 n 个位置。

本例算连接所有关键点的最小树边数；若原题必须从固定根出发，就把固定根也加入 keys。做DP时把虚树边建成父→子，反向处理 DFS 序；先确认关键点标记不能被加入的 LCA 混淆。

主体依赖：3.4 倍增 LCA。先复制这些基础定义，再复制下列代码。

3.7 虚树 · 主体

```
struct VirtualTree : LCA {
    VirtualTree(const vector<vector<int>>& adj, int root = 1) : LCA(adj, root) {}
}
pair<int, vector<pair<int, int>>> build(vector<int> nodes) const {
    if(nodes.empty()) {
        return {0, {}};
    }
    auto cmp = [&](int u, int v) { return tin[u] < tin[v]; };
    sort(nodes.begin(), nodes.end(), cmp);
    int k = int(nodes.size());
    for(int i = 1; i < k; i++) {
        nodes.push_back(lca(nodes[i - 1], nodes[i]));
    }
    sort(nodes.begin(), nodes.end(), cmp);
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
    vector<pair<int, int>> edges;
    vector<int> st;
```

3.7 虚树 · 主体 · 续

```

    for(int u : nodes) {
        while(!st.empty() && lca(st.back(), u) != st.back()) {
            st.pop_back();
        }
        if(!st.empty()) {
            edges.push_back({st.back(), u});
        }
        st.push_back(u);
    }
    return {nodes.front(), edges};
}
};

```

虚树边权与 Steiner 子树长度

3.7 · 虚树边权与 Steiner 子树长度

```

int main() {
    vector<vector<int>> g(6);
    for(auto [u, v] : vector<pair<int, int>>{{1, 2}, {1, 3}, {2, 4}, {2, 5}}) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    VirtualTree s(g);
    auto [root, e] = s.build({4, 5, 3, 4});
    long long ans = 0;
    for(auto [u, v] : e) {
        ans += s.dis(u, v);
    }
    assert(root == 1 && ans == 4);
}

```

3.8 带信息倍增

`Doubling<T,Op>(to,w,maxStep,identity)`, 信息描述从点 x 跳一次的权值。Op 为有单位元的结合运算，按路径先后合并；允许非交换。0 为吸收哨兵，信息已置单位元。jump(x,k) 返回终点与累计信息， $0 \leq k \leq \text{maxStep}$ ；超范围高位不能静默使用。

3.8 带信息倍增 · 主体

```

using namespace std;
using i64 = long long;
template <class T, class Op> struct Doubling {
    int log;
    T identity;
    Op op;
    vector<vector<int>> next;
    vector<vector<T>> value;
    Doubling(const vector<int>& to, const vector<T>& weight, i64 maxStep, T identity_,
              Op op_ = Op())
        : log(1), identity(identity_), op(op_) {
        for(i64 x = maxStep; x > 1; x >>= 1) {
            log++;
        }
        next.assign(log, vector<int>(to.size()));
        value.assign(log, vector<T>(to.size()));
        next[0] = to;
        value[0] = weight;
        for(int j = 0; j < log; j++) {
            value[j][0] = identity;
        }
        for(int j = 1; j < log; j++) {
            for(int x = 1; x < int(to.size()); x++) {
                next[j][x] = next[j - 1][next[j - 1][x]];
                value[j][x] = op(value[j - 1][x], value[j - 1][next[j - 1][x]]);
            }
        }
    }
};

```

3.8 带信息倍增 · 主体 · 续

```

    }
}
pair<int, T> jump(int x, i64 k) const {
    T res = identity;
    for(int j = 0; j < log; j++) {
        if(k >> j & 1) {
            res = op(res, value[j][x]);
            x = next[j][x];
        }
    }
    return {x, res};
}
};

```

3.9 DSU on Tree

固定根子树统计，轻子树算完清空，重子树保留，再加轻子树与当前点。run(add,answer) 的 add(u,+1/-1) 必须可逆，answer(u) 读取当前整个子树。tin/tout 是1-based闭区间；总操作 $O(n \log n)$ ，本版控制流为显式栈，run 结束清空统计。

3.9 DSU on Tree · 主体

```

using namespace std;
struct DSUonTree {
    const vector<vector<int>>& adj;
    vector<int> parent, size, heavy, tin, tout, euler;
    int root, timer = 0;
    DSUonTree(const vector<vector<int>>& graph, int root_ = 1)
        : adj(graph), parent(adj.size()), size(adj.size()), heavy(adj.size()),
          tin(adj.size()), tout(adj.size()), euler(adj.size()), root(root_), timer(1) {
        if(adj.size() > 1) {
            dfsSize(root, 0);
        }
    }
    void dfsSize(int root, int p) {
        vector<int> stk{root}, order;
        parent[root] = p;
        while(!stk.empty()) {
            int u = stk.back();
            stk.pop_back();
            tin[u] = timer;
            euler[timer++] = u;
            order.push_back(u);
            for(int i = int(adj[u].size()) - 1; i >= 0; i--) {
                int v = adj[u][i];
                if(v == parent[u]) {
                    continue;
                }
                parent[v] = u;
                stk.push_back(v);
            }
        }
        for(int i = int(order.size()) - 1; i >= 0; i--) {
            int u = order[i];
            size[u] = 1;
            for(int v : adj[u]) {
                if(v != parent[u]) {
                    size[u] += size[v];
                    if(!heavy[u] || size[v] > size[heavy[u]]) {
                        heavy[u] = v;
                    }
                }
            }
            tout[u] = tin[u] + size[u] - 1;
        }
    }
}

```

3.9 DSU on Tree · 主体 · 续

```

    }
    template <class Add, class Answer> void run(Add add, Answer answer) {
        if(adj.size() <= 1) {
            return;
        }
        vector<tuple<int, int, bool>> stk{{root, 0, false}};
        while(!stk.empty()) {
            auto [u, kind, keep] = stk.back();
            stk.pop_back();
            if(kind == 0) {
                stk.push_back({u, 3, keep});
                stk.push_back({u, 2, keep});
                stk.push_back({u, 1, keep});
                if(heavy[u]) {
                    stk.push_back({heavy[u], 0, true});
                }
                for(int v : adj[u]) {
                    if(v != parent[u] && v != heavy[u]) {
                        stk.push_back({v, 0, false});
                    }
                }
            }
            else if(kind == 1) {
                for(int v : adj[u]) {
                    if(v != parent[u] && v != heavy[u]) {
                        for(int i = tin[v]; i <= tout[v]; i++) {
                            add(euler[i], 1);
                        }
                    }
                }
                add(u, 1);
            }
            else if(kind == 2) {
                answer(u);
            }
            else if(!keep) {
                for(int i = tin[u]; i <= tout[u]; i++) {
                    add(euler[i], -1);
                }
            }
        }
    }
};

```

子树不同颜色数

3.9 · 子树不同颜色数

```

int main() {
    vector<vector<int>> g(5);
    for(int u = 2; u <= 4; u++) {
        g[1].push_back(u), g[u].push_back(1);
    }
    vector<int> color = {0, 1, 1, 2, 2}, cnt(3), ans(5);
    int cur = 0;
    DSUOnTree s(g);
    s.run(
        [&](int u, int d) {
            if(cnt[color[u]]) {
                cur--;
            }
            cnt[color[u]] += d;
            if(cnt[color[u]]) {
                cur++;
            }
        },
        [&](int u) { ans[u] = cur; });
    assert(ans[1] == 2 && ans[2] == 1 && cur == 0);
}

```

3.10 点分治与点分树

Centroid(g) 返回点分树 parent、level, $O(n \log n)$; build 后 blocked 全为 true, 只是构建时的内部标记。点分祖先链长度 $O(\log n)$ 。下面给完整 “染色点最近距离” 用法; 若做路径计数, 分治中心时逐子树查询、再插入, 避免同子树对被重复计算。

每个点更新/查询所有点分祖先。能保证某个分治中心落在原树查询点与最近染色点的路径上, 故取最小正确。此实现结合倍增LCA为 $O(\log^2 n)$ 每次; 若预存各点到点分祖先的距离可做到 $O(\log n)$ 。只支持添加染色点, 删除要把每个中心 best 改成可删多重集。

3.10 点分治与点分树 · 主体

```
using namespace std;
struct Centroid {
    const vector<vector<int>>& adj;
    vector<int> parent, level, size, fa;
    vector<bool> blocked;
    Centroid(const vector<vector<int>>& graph)
        : adj(graph), parent(adj.size()), level(adj.size()), size(adj.size()),
          fa(adj.size()), blocked(adj.size()) {}
    if(adj.size() > 1) {
        build(1, 0, 0);
    }
}
int getSize(int u, int p) {
    vector<int> order{u};
    fa[u] = p;
    for(int i = 0; i < int(order.size()); i++) {
        int x = order[i];
        for(int v : adj[x]) {
            if(v != fa[x] && !blocked[v]) {
                fa[v] = x, order.push_back(v);
            }
        }
    }
    for(int i = int(order.size()) - 1; i >= 0; i--) {
        int x = order[i];
        size[x] = 1;
        for(int v : adj[x]) {
            if(v != fa[x] && !blocked[v]) {
                size[x] += size[v];
            }
        }
    }
    return size[u];
}
int getCentroid(int u, int p, int n) {
    while(true) {
        int v = 0;
        for(int x : adj[u]) {
            if(x != p && !blocked[x] && size[x] * 2 > n) {
                v = x;
                break;
            }
        }
        if(!v) {
            return u;
        }
        p = u;
        u = v;
    }
}
void build(int entry, int p, int dep) {
    int c = getCentroid(entry, 0, getSize(entry, 0));
    parent[c] = p;
    level[c] = dep;
    blocked[c] = true;
    for(int v : adj[c]) {

```

3.10 点分治与点分树 · 主体 · 续

```

        if(!blocked[v]) {
            build(v, c, dep + 1);
        }
    }
};
};

```

点分树：动态染色后的最近点

例子还需要：3.4 倍增 LCA。

3.10 · 点分树：动态染色后的最近点

```

int main() {
    vector<vector<int>> g(6);
    for(int u = 2; u <= 5; u++) {
        g[u - 1].push_back(u), g[u].push_back(u - 1);
    }
    LCA l(g);
    Centroid c(g);
    const int INF = 1000000000;
    vector<int> best(6, INF);
    auto add = [&](int u) {
        for(int x = u; x; x = c.parent[x]) {
            best[x] = min(best[x], l.dis(u, x));
        }
    };
    auto query = [&](int u) {
        int ans = INF;
        for(int x = u; x; x = c.parent[x]) {
            ans = min(ans, best[x] + l.dis(u, x));
        }
        return ans;
    };
    add(1);
    add(5);
    assert(query(3) == 2 && query(4) == 1);
}

```

3.11 换根 DP 速查〔速查〕

两遍：子→父计算 down，父→子计算 up。若合并可逆可减掉该子贡献；不可逆用前后缀合并，保持原顺序。根上的单位元、点权加入次数、边方向各自定义，不能复用在不说明状态的空模板。

3.12 Small-to-Large〔速查〕

把较小容器逐元素并入较大容器，元素所在容器大小至少翻倍，移动 $O(n \log n)$ 次；若每次插入 $O(\log n)$ ，总 $O(n \log^2 n)$ 。合并后旧容器应释放或复用，持久保存所有中间大容器会额外耗内存。

3.13 长链剖分〔速查〕

选深度最大的儿子，把其深度 DP 数组向父偏移一格复用；轻子数组逐项合并。只有合并代价与轻子数组长度成正比等条件下能得 $O(n)$ 摊还。按子树大小选重儿子是重链剖分，不等价；数组缓冲区扩容会使引用失效。

4. DP: 实用代码与条件速查

4.1 LIS / 最长不下降子序列

`lisLength(a,true)` 严格上升用 `lower_bound`; `false` 为不下降用 `upper_bound`。输入 0-based, $O(n \log n)$, 仅返回长度; 恢复方案需记录前驱与每个长度结尾下标。

4.1 LIS / 最长不下降子序列 · 主体

```
using namespace std;
int lisLength(const vector<int>& a, bool strict = true) {
    vector<int> d;
    d.reserve(a.size());
    for(int x : a) {
        auto it = strict ? lower_bound(d.begin(), d.end(), x)
                        : upper_bound(d.begin(), d.end(), x);
        if(it == d.end()) {
            d.push_back(x);
        } else {
            *it = x;
        }
    }
    return int(d.size());
}
```

4.2 多重背包: 单调队列

物品 1-based, $w>0, m>=0$; `multiKnapsack` 返回总重量不超过 `capacity` 的最大价值, 允许不选, 因此初值0。按重量模 w 的同余类维护 `pre[j]-j/w*v` 的滑窗, $O(nV)$, i64 价值。要求“恰好装满”需把不可达设负无穷并跳过不可达转移。

4.2 多重背包: 单调队列 · 主体

```
using namespace std;
using i64 = long long;
i64 multiKnapsack(int n, int capacity, const vector<int>& w, const vector<i64>& v,
                  const vector<int>& m) {
    vector<vector<i64>> dp(2, vector<i64>(capacity + 1, 0));
    for(int i = 1; i <= n; i++) {
        auto& pre = dp[(i - 1) % 2];
        auto& cur = dp[i % 2];
        for(int rem = 0; rem < w[i]; rem++) {
            deque<int> q;
            for(int j = rem; j <= capacity; j += w[i]) {
                while(!q.empty() &&
                    pre[j] >= pre[q.back()] + (j - q.back()) / w[i] * v[i]) {
                    q.pop_back();
                }
                q.push_back(j);
                while(!q.empty() && q.front() < (j - 1LL * m[i] * w[i])) {
                    q.pop_front();
                }
                cur[j] = max(pre[j], pre[q.front()] + (j - q.front()) / w[i] * v[i]);
            }
        }
    }
    return dp[n % 2][capacity];
}
```

4.3 数位 DP 速查 [速查]

处理 $[0,R]$, 区间答案 $F(R)-F(L-1)$, $L=0$ 单独处理。状态常含 `pos`、`lead`、性质状态; `tight` 依上界变化, 通常只缓存非 `tight` 状态。必须明确 0 是否计入、前导0是否影响性质; 多次查询要清理依赖上界/长度的缓存。

4.4 轮廓线 DP〔速查〕

状态保留扫描线上的连通性/占用，每格更新并移动边界。插头连通标号需规范化后合并状态；行末移位不可丢掉未闭合连通块。一般状态数随宽度指数增长，先把较短边作为宽度。

4.5 中位数阈值变换〔速查〕

判断中位数 $> x$ ： $a_i > x$ 映为 +1，否则 -1。奇数长度要求和 > 0 ；偶数取上中位数要求和 > 0 ，取下中位数要求和 > 0 （等价 > 2 ）。前缀和把区间条件转成两前缀比较，须同时满足长度限制。

4.6 排列 LCS 转 LIS〔速查〕

仅当映射序列中每个值唯一时，记录第一序列 pos，把第二序列映成 pos 后求严格 LIS。一般重复元素 LCS 不能这样做；可用 Hunt-Szymanski 对每次匹配位置倒序展开，规模取决于匹配对数。

4.7 背包循环方向〔速查〕

01 背包容量降序；完全背包升序；多重可二进制拆分或用前述单调队列。至多容量初值 0，恰好容量 $dp[0]=0$ 、其余 -INF；计数背包的循环顺序决定计算排列还是组合。

4.8 单调队列优化 DP〔速查〕

把转移写为 $dp[i]=A(i)+\min/\max\{B(j)\}$ 且 j 的合法窗口左右端点单调移动。入队按 B 优劣弹尾，窗口过期弹头。平值弹不弹会影响方案选取；没有固定窗口或候选比较随 i 改变时不能直接套。

4.9 区间 DP〔速查〕

常见 $dp[l][r]=\min_k(dp[l][k]+dp[k+1][r]+w(l,r))$ ，按长度递增。明确空区间、单点、INF 与合法分割点。三重循环只是问题骨架，此处只留状态与条件速查。

4.10 树上背包〔速查〕

合并儿子 v ： $new[i+j]=\max(new[i+j], dp[u][i]+dp[v][j]+边贡献)$ ，从合法容量与可达状态转移；原数组若就地更新必须防重复使用儿子。复杂度由子树容量和截断方式分析，不能把任意合并笼统写成 $O(nK)$ 。

4.11 SOS 子集和与反演〔速查〕

$F[S]=\sum_{T\subseteq S} f[T]$ ：每位 b ，对含 b 的 S 执行 $F[S] += F[S \setminus \{b\}]$ ， $O(2^B \cdot B)$ 。反演把加改减；超集和条件改为不含 b 。普通 SOS 不是按子集大小分层的子集卷积。

4.12 分治 DP 优化〔速查〕

上一层已固定， $dp[i]=\min_{j<i}(pre[j]+w(j,i))$ ，最优决策 $opt[i]$ 单调时递归求中点并缩小候选范围，一层 $O(n \log n)$ 次转移。需证明决策单调或 Monge 条件；当前层内互相依赖的转移不能照搬。

4.13 状态压缩与 Gray code〔速查〕

$Gray(i)=i \oplus (i >> 1)$ ，相邻只改变一位；可用 $_builtin_ctz(diff)$ 定位 ($diff \neq 0$)。只适合能 $O(1)$ 增删单元元素贡献的枚举。uint64_t 全位移时避免 $1 < 64$ ；一般子集枚举 $sub=(sub-1)\&S$ 的终止要显式处理 0。

4.14 Knuth 优化〔速查〕

$dp[l][r] = \min_k (dp[l][k] + dp[k+1][r] + w(l, r))$; 要求区间代价满足包含单调性和四边形不等式, 推出 $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$ 。只检查四边形不等式不足以自动成立。单点 opt 初始化, 候选截到 $[l, r-1]$, $O(n^2)$ 。

4.15 WQS / Aliens trick〔速查〕

给每段/每选项加惩罚 λ , 求 $\min(F(k) + \lambda k)$ 并同时记录 k ; λ 增大时最优 k 非增, 平值偏向统一方向。只有目标点在相应凸包上等条件才可从惩罚答案减 λK 恢复恰好 K 的值。计数跳过 K 时不能无证明输出这个式子。

4.16 bitset 优化〔速查〕

非负01背包: $bs[0]=1$, 每物品 $bs |= bs \ll w$ 。bitset<N> 的 N 是编译期常量, 越界位自动截断; 想用大范围时先计算可达和上界。移位/位运算约 $O(N/\text{机器字长})$, 负权不能直接使用左移。

5. 数论与组合

5.1 模整数 MInt

MInt<P> 保证值在 $[0,P)$ ；加减乘支持 $2 \leq P \leq \text{INT_MAX}$ 。除法/逆元使用费马，要求 P 为素数且分母非0；非素数改用 exgcd，不能认为构造类型就自动保证可除。power 指数非负。

5.1 模整数 MInt · 主体

```
using namespace std;
using i64 = long long;
template <int MOD> struct MInt {
    int x;
    constexpr MInt(i64 v = 0) : x(norm(v)) {
    }
    constexpr int val() const {
        return x;
    }
    constexpr static int getMod() {
        return MOD;
    }
    constexpr int norm(i64 v) const {
        v %= MOD;
        if(v < 0) {
            v += MOD;
        }
        return int(v);
    }
    constexpr MInt operator-() const {
        return MInt(x ? MOD - x : 0);
    }
    constexpr MInt& operator+=(MInt rhs) {
        i64 sum = i64(x) + rhs.x;
        x = int(sum >= MOD ? sum - MOD : sum);
        return *this;
    }
    constexpr MInt& operator-=(MInt rhs) {
        x -= rhs.x;
        if(x < 0) {
            x += MOD;
        }
        return *this;
    }
    constexpr MInt& operator*=(MInt rhs) {
        x = int(1LL * x * rhs.x % MOD);
        return *this;
    }
    constexpr MInt& operator/=(MInt rhs) {
        return *this *= rhs.inv();
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return power(*this, MOD - 2);
    }
    friend constexpr MInt operator+(MInt a, MInt b) {
        return a += b;
    }
    friend constexpr MInt operator-(MInt a, MInt b) {
        return a -= b;
    }
    friend constexpr MInt operator*(MInt a, MInt b) {
        return a *= b;
    }
    friend constexpr MInt operator/(MInt a, MInt b) {
        return a /= b;
    }
    friend constexpr bool operator==(MInt a, MInt b) {
```

5.1 模整数 MInt · 主体 · 续

```

        return a.x == b.x;
    }
    friend constexpr bool operator!=(MInt a, MInt b) {
        return a.x != b.x;
    }
    friend constexpr bool operator<(MInt a, MInt b) {
        return a.x < b.x;
    }
};
template <int MOD> constexpr MInt<MOD> power(MInt<MOD> a, i64 b) {
    MInt<MOD> res = 1;
    while(b > 0) {
        if(b & 1) {
            res *= a;
        }
        a *= a;
        b >>= 1;
    }
    return res;
}

```

5.2 exgcd / 扩展 CRT

exgcd 用 i128 输出 x, y , 满足 $ax+by=\gcd(a,b)$, 常用非负 a, b . $\text{invMod}(a, m, \text{res})$ 仅 $\gcd=1$ 可逆. crt 的每个元素是 (余数, 正模数); 返回 1 成功、0 不相容、-1 合并模数超 i64, 输出标准余数 $r \in [0, \text{mod})$. 不是所有同余都要求模数互素.

5.2 exgcd / 扩展 CRT · 主体

```

using namespace std;
using i64 = long long;
using i128 = __int128;
i128 exgcd(i128 a, i128 b, i128& x, i128& y) {
    if(!b) {
        x = 1;
        y = 0;
        return a;
    }
    i128 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
bool invMod(i64 a, i64 m, i64& ans) {
    if(m <= 0) {
        return false;
    }
    a = (i128(a) % m + m) % m;
    i128 x, y;
    if(exgcd(a, m, x, y) != 1) {
        return false;
    }
    ans = (x % m + m) % m;
    return true;
}
// Return 0: inconsistent, 1: success, -1: resulting modulus exceeds i64.
int crtMerge(i64 a, i64 m, i64 b, i64 n, i64& r, i64& mod) {
    assert(m > 0 && n > 0);
    a = (i128(a) % m + m) % m;
    b = (i128(b) % n + n) % n;
    i128 x, y, g = exgcd(m, n, x, y), d = i128(b) - a;
    if(d % g) {
        return 0;
    }
    i128 q = n / g, k = (d / g * x % q + q) % q, M = i128(m) * q;
    if(M > LLONG_MAX) {

```

5.2 exgcd / 扩展 CRT · 主体 · 续

```

        return -1;
    }
    r = (a + i128(m) * k) % M;
    mod = M;
    return 1;
}
int crt(const vector<pair<i64, i64>>& e, i64& r, i64& mod) {
    r = 0;
    mod = 1;
    for(auto [a, m] : e) {
        int z = crtMerge(r, mod, a, m, r, mod);
        if(z != 1) {
            return z;
        }
    }
    return 1;
}

```

非互素 CRT

5.2 · 非互素 CRT

```

int main() {
    long long r, m;
    int ok = crt({{2, 6}, {5, 9}}, r, m);
    assert(ok == 1 && r == 14 && m == 18);
    assert(crt({{1, 2}, {0, 4}}, r, m) == 0);
}

```

5.3 阶乘组合数

Combination(N,P), P 素数, 预处理到 $\min(N, P-1)$, 查询要求 $0 \leq n \leq N$ 且 $n < P$; 非法 k 返回 0。 $n \geq P$ 时要用 Lucas 或其他算法, 本版不会静默返回错误 0。 $O(N)$ 预处理、 $O(1)$ 查询。

5.3 阶乘组合数 · 主体

```

using namespace std;
using i64 = long long;
struct Combination {
    int mod;
    vector<int> fac, ifac;
    Combination(int n, int p) : mod(p) {
        int limit = min(n, p - 1);
        fac.assign(limit + 1, 1);
        for(int i = 1; i <= limit; i++) {
            fac[i] = int(1LL * fac[i - 1] * i % p);
        }
        ifac.assign(limit + 1, 1);
        ifac[limit] = powerMod(fac[limit], p - 2, p);
        for(int i = limit; i >= 1; i--) {
            ifac[i - 1] = int(1LL * ifac[i] * i % p);
        }
    }
    static int powerMod(int a, i64 e, int p) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
                res = int(1LL * res * a % p);
            }
            a = int(1LL * a * a % p);
            e >>= 1;
        }
        return res;
    }
}
int C(i64 n, i64 k) const {
    if(k < 0 || k > n) {

```

5.3 阶乘组合数 · 主体 · 续

```

        return 0;
    }
    assert(n < mod && n < (i64)fac.size());
    int nn = int(n);
    int kk = int(k);
    return int(1LL * fac[nn] * ifac[kk] % mod * ifac[nn - kk] % mod);
}
};

```

5.4 Lucas

P 为小素数，预处理 $0..P-1$ $O(P)$ ； $C(n,k)$ 分 P 进制相乘 $O(\log_P n)$ 。若 $P \approx 1e9$ 无法开这么大的表；非素数不能直接用 Lucas，应分解素数幂处理。

5.4 Lucas · 主体

```

using namespace std;
using i64 = long long;
struct Lucas {
    int p;
    vector<int> fac, ifac;
    Lucas(int p_) : p(p_), fac(p_), ifac(p_) {
        fac[0] = ifac[0] = 1;
        for(int i = 1; i < p; i++) {
            fac[i] = int(1LL * fac[i - 1] * i % p);
        }
        ifac[p - 1] = powerMod(fac[p - 1], p - 2, p);
        for(int i = p - 2; i >= 1; i--) {
            ifac[i] = int(1LL * ifac[i + 1] * (i + 1) % p);
        }
    }
    static int powerMod(int a, i64 e, int p) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
                res = int(1LL * res * a % p);
            }
            a = int(1LL * a * a % p);
            e >>= 1;
        }
        return res;
    }
    int smallC(int n, int k) const {
        if(k < 0 || k > n) {
            return 0;
        }
        return int(1LL * fac[n] * ifac[k] % p * ifac[n - k] % p);
    }
    int C(i64 n, i64 k) const {
        if(k < 0 || k > n) {
            return 0;
        }
        if(n < p) {
            return smallC(int(n), int(k));
        }
        return int(1LL * smallC(int(n % p), int(k % p)) * C(n / p, k / p) % p);
    }
};

```

5.5 连续点拉格朗日插值

$y[i]=f(i), i=0..d$, 次数 $\leq d$; $\text{lagrange}(y,x,P)$ $O(d)$, P 素数且 $d < P$, y 已规范化。 x 可负或大于 P , 先模 P ; 任意不连续横坐标需换通式, 本版不适用。

5.5 连续点拉格朗日插值 · 主体

```

using namespace std;
using i64 = long long;
int powerMod(int a, i64 e, int p) {
    int res = 1;
    while(e > 0) {
        if(e & 1) {
            res = int(1LL * res * a % p);
        }
        a = int(1LL * a * a % p);
        e >>= 1;
    }
    return res;
}
int lagrange(const vector<int>& y, i64 x, int p) {
    int n = int(y.size()) - 1;
    if(n < 0) {
        return 0;
    }
    if(0 <= x && x <= n) {
        return y[int(x)];
    }
    i64 v = x % p;
    if(v < 0) {
        v += p;
    }
    vector<int> fac(n + 1, 1), ifac(n + 1);
    for(int i = 1; i <= n; i++) {
        fac[i] = int(1LL * fac[i - 1] * i % p);
    }
    ifac[n] = powerMod(fac[n], p - 2, p);
    for(int i = n; i >= 1; i--) {
        ifac[i - 1] = int(1LL * ifac[i] * i % p);
    }
    vector<int> pre(n + 1, 1), suf(n + 2, 1);
    for(int i = 1; i <= n; i++) {
        int d = int(v - (i - 1));
        if(d < 0) {
            d += p;
        }
        pre[i] = int(1LL * pre[i - 1] * d % p);
    }
    for(int i = n - 1; i >= 0; i--) {
        int d = int(v - (i + 1));
        if(d < 0) {
            d += p;
        }
        suf[i] = int(1LL * suf[i + 1] * d % p);
    }
    i64 res = 0;
    for(int i = 0; i <= n; i++) {
        int cur = int(1LL * y[i] * pre[i] % p * suf[i] % p * ifac[i] % p * ifac[n - i] % p);
        if((n - i) & 1) {
            res -= cur;
            if(res < 0) {
                res += p;
            }
        } else {
            res += cur;
            if(res >= p) {
                res -= p;
            }
        }
    }
    return res;
}

```

5.6 线性筛

LinearSieve(N): primes、minp、phi、mu; phi[1]=mu[1]=1, 0处为占位。O(N) 时间与空间。合数由最小质因子唯一筛出; 分解小整数可反复除 minp。

5.6 线性筛 · 主体

```
using namespace std;
using i64 = long long;
struct LinearSieve {
    int n;
    vector<int> primes, minp, phi, mu;
    LinearSieve(int n_ = 0) : n(n_), minp(n_ + 1), phi(n_ + 1), mu(n_ + 1) {
        if(n < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        for(int i = 2; i <= n; i++) {
            if(minp[i] == 0) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > n) {
                    break;
                }
                int vv = int(v);
                minp[vv] = p;
                if(i % p == 0) {
                    phi[vv] = phi[i] * p;
                    mu[vv] = 0;
                    break;
                }
                phi[vv] = phi[i] * (p - 1);
                mu[vv] = -mu[i];
            }
        }
    }
};
```

5.7 φ / μ 前缀与反演

MobiusPhi(N) 线性预处理, phiPrefix、muPrefix 查询至 N。 $\sum(d|n)\varphi(d)=n$, $\sum(d|n)\mu(d)=[n=1]$ 。互质对数= $\sum d \mu(d)[A/d][B/d]$, 结合整除分块; 矩形与有序/无序定义要单独处理。

5.7 φ / μ 前缀与反演 · 主体

```
using namespace std;
using i64 = long long;
struct MobiusPhi {
    int n;
    vector<int> primes, minp, phi, mu;
    vector<i64> prefPhi, prefMu;
    MobiusPhi(int n_ = 0) : n(n_), minp(n_ + 1), phi(n_ + 1), mu(n_ + 1), prefPhi(n_ + 1), prefMu(n_ + 1) {
        if(n < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        for(int i = 2; i <= n; i++) {
            if(minp[i] == 0) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > n) {
                    break;
                }
                int vv = int(v);
                minp[vv] = p;
                if(i % p == 0) {
                    phi[vv] = phi[i] * p;
                    mu[vv] = 0;
                    break;
                }
                phi[vv] = phi[i] * (p - 1);
                mu[vv] = -mu[i];
            }
        }
        for(int i = 1; i <= n; i++) {
            prefPhi[i] = prefPhi[i-1] + phi[i];
            prefMu[i] = prefMu[i-1] + mu[i];
        }
    }
};
```

5.7 φ / μ 前缀与反演 · 主体 · 续

```

        primes.push_back(i);
        phi[i] = i - 1;
        mu[i] = -1;
    }
    for(int p : primes) {
        i64 v = 1LL * i * p;
        if(v > n) {
            break;
        }
        int vv = int(v);
        minp[vv] = p;
        if(i % p == 0) {
            phi[vv] = phi[i] * p;
            mu[vv] = 0;
            break;
        }
        phi[vv] = phi[i] * (p - 1);
        mu[vv] = -mu[i];
    }
}
for(int i = 1; i <= n; i++) {
    prefPhi[i] = prefPhi[i - 1] + phi[i];
    prefMu[i] = prefMu[i - 1] + mu[i];
}
}
i64 phiPrefix(int x) const {
    return prefPhi[x];
}
i64 muPrefix(int x) const {
    return prefMu[x];
}
};

```

5.8 Miller-Rabin 64 位素性

isPrime(u64) 使用确定性 64 位底数集合，mulMod 以 u128 乘法防溢出；0、1 为非素数。不能减少成只测 2、3、5，也不要浮点乘法取模替进去。

5.8 Miller-Rabin 64 位素性 · 主体

```

using namespace std;
using u64 = unsigned long long;
using u128 = __uint128_t;
u64 mulMod(u64 a, u64 b, u64 m) {
    return u64(u128(a) * b % m);
}
u64 powerMod(u64 a, u64 e, u64 m) {
    u64 res = 1 % m;
    while(e > 0) {
        if(e & 1) {
            res = mulMod(res, a, m);
        }
        a = mulMod(a, a, m);
        e >>= 1;
    }
    return res;
}
bool isPrime(u64 n) {
    if(n < 2) {
        return false;
    }
    for(u64 p :
        {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if(n % p == 0) {
            return n == p;
        }
    }
}

```

5.8 Miller-Rabin 64 位素性 · 主体 · 续

```

    }
    u64 d = n - 1;
    int s = 0;
    while((d & 1) == 0) {
        d >>= 1;
        s++;
    }
    for(u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if(a % n == 0) {
            continue;
        }
        u64 x = powerMod(a % n, d, n);
        if(x == 1 || x == n - 1) {
            continue;
        }
        bool composite = true;
        for(int r = 1; r < s; r++) {
            x = mulMod(x, x, n);
            if(x == n - 1) {
                composite = false;
                break;
            }
        }
        if(composite) {
            return false;
        }
    }
    return true;
}

```

5.9 Pollard-Rho 分解

PollardRho::factorize(n) 返回排好序、带重复的质因数；支持正 i64, $n \leq 1$ 返回空。随机重试，不承诺最坏多项式时间；可传 seed 复现。对超出 i64 的无符号数，此分解接口不适用，虽素性判定支持 u64。

5.9 Pollard-Rho 分解 · 主体

```

using namespace std;
using i64 = long long;
using u64 = unsigned long long;
using u128 = __uint128_t;
struct PollardRho {
    static u64 mulMod(u64 a, u64 b, u64 m) {
        return u64(u128(a) * b % m);
    }
    static u64 powerMod(u64 a, u64 e, u64 m) {
        u64 res = 1 % m;
        while(e > 0) {
            if(e & 1) {
                res = mulMod(res, a, m);
            }
            a = mulMod(a, a, m);
            e >>= 1;
        }
        return res;
    }
    static bool isPrime(u64 n) {
        if(n < 2) {
            return false;
        }
        for(u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL,
                    31ULL, 37ULL}) {
            if(n % p == 0) {
                return n == p;
            }
        }
    }
}

```

5.9 Pollard-Rho 分解 · 主体 · 续

```

    u64 d = n - 1;
    int s = 0;
    while((d & 1) == 0) {
        d >>= 1;
        s++;
    }
    for(u64 a :
        {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if(a % n == 0) {
            continue;
        }
        u64 x = powerMod(a % n, d, n);
        if(x == 1 || x == n - 1) {
            continue;
        }
        bool composite = true;
        for(int r = 1; r < s; r++) {
            x = mulMod(x, x, n);
            if(x == n - 1) {
                composite = false;
                break;
            }
        }
        if(composite) {
            return false;
        }
    }
    return true;
}
static i64 f(i64 x, i64 c, i64 n) {
    return i64((u128(x) * x + u64(c)) % u64(n));
}
static i64 pollard(i64 n, mt19937_64& rng) {
    if(n % 2 == 0) {
        return 2;
    }
    while(true) {
        i64 c = i64(rng() % (n - 1)) + 1;
        i64 x = i64(rng() % (n - 1)) + 1;
        i64 y = x;
        i64 d = 1;
        while(d == 1) {
            x = f(x, c, n);
            y = f(f(y, c, n), c, n);
            i64 diff = x > y ? x - y : y - x;
            d = std::gcd(diff, n);
        }
        if(d != n) {
            return d;
        }
    }
}
static void factorRec(i64 n, mt19937_64& rng, vector<i64>& out) {
    if(n <= 1) {
        return;
    }
    if(isPrime(u64(n))) {
        out.push_back(n);
        return;
    }
    if(n % 2 == 0) {
        out.push_back(2);
        factorRec(n / 2, rng, out);
        return;
    }
    i64 d = pollard(n, rng);
    factorRec(d, rng, out);
}

```

5.9 Pollard-Rho 分解 · 主体 · 续

```

    factorRec(n / d, rng, out);
}
static vector<i64> factorize(i64 n) {
    vector<i64> out;
    if(n <= 1) {
        return out;
    }
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    factorRec(n, rng, out);
    sort(out.begin(), out.end());
    return out;
}
static vector<i64> factorize(i64 n, u64 seed) {
    vector<i64> out;
    if(n <= 1) {
        return out;
    }
    mt19937_64 rng(seed);
    factorRec(n, rng, out);
    sort(out.begin(), out.end());
    return out;
}
};

```

5.10 整除分块

$\text{sumFloorDiv}(n,k)=\sum_{i=1..n}[k/i]$, n,k 非负, 返回值需在 i64 内。同值块右端 $r=\min(n,k/(k/l))$; 商0时余下全为0。双商问题 r 同时取两者右端最小值, 复杂度 $O(\sqrt{k})$ 。

5.10 整除分块 · 主体

```

using namespace std;
using i64 = long long;
using i128 = __int128_t;
i64 sumFloorDiv(i64 n, i64 k) {
    i128 ans = 0;
    for(i64 l = 1, r; l <= n; l = r + 1) {
        i64 v = k / l;
        if(v == 0) {
            r = n;
        } else {
            r = min(n, k / v);
        }
        ans += i128(r - l + 1) * v;
    }
    return i64(ans);
}

```

5.11 类欧几里得 floor sum

$\text{floorSum}(n,a,b,c)$, $c>0$, i 从0到 n (包含 n)。返回 $f=\sum q_i$, $g=\sum i*q_i$, $h=\sum q_i^2$, 其中 $q_i=\text{floor}((ai+b)/c)$ 。支持负 a,b , 用向下取整而非 C++ 截断; 中间 i128, 输出三项需在 i64 内。 $O(\log \max(|a|,c))$ 。

5.11 类欧几里得 floor sum · 主体

```

using namespace std;
using i64 = long long;
using i128 = __int128_t;
struct FSResult {
    i64 f, g, h;
};
FSResult floorSumCalc(i128 n, i128 a, i128 b, i128 c) {
    if(n < 0) {
        return {0, 0, 0};
    }
}

```

5.11 类欧几里得 floor sum · 主体 · 续

```

    }
    i128 A = a / c;
    if(a % c != 0 && a < 0) {
        A--;
    }
    i128 B = b / c;
    if(b % c != 0 && b < 0) {
        B--;
    }
    i128 ra = a - A * c;
    i128 rb = b - B * c;
    if(A != 0 || B != 0) {
        FSResult t = floorSumCalc(n, ra, rb, c);
        i128 s0 = n + 1;
        i128 s1 = n * (n + 1) / 2;
        i128 s2 = n * (n + 1) * (2 * n + 1) / 6;
        return {i64(A * s1 + B * s0 + t.f), i64(A * s2 + B * s1 + t.g),
                i64(A * A * s2 + 2 * A * B * s1 + B * B * s0 + 2 * A * t.g + 2 * B * t.f +
                    t.h)};
    }
    if(a == 0) {
        return {0, 0, 0};
    }
    i128 m = (a * n + b) / c;
    if(m == 0) {
        return {0, 0, 0};
    }
    FSResult t = floorSumCalc(m - 1, c, c - b - 1, a);
    return {i64(n * m - t.f), i64(m * n * (n + 1) / 2 - (t.f + t.h) / 2),
            i64(n * m * m - t.f - 2 * t.g)};
}
FSResult floorSum(i64 n, i64 a, i64 b, i64 c) {
    if(c <= 0) {
        return {0, 0, 0};
    }
    return floorSumCalc(n, a, b, c);
}

```

5.12 扩展 BSGS

BSGS::solve(a,b,m) 求 $a^x \equiv b \pmod m$ 的最小 $x \geq 0$, 无解 -1; $m > 0$ 。处理不互素、 $a=0$ 、 $m=1$, 模乘 i128; 时间空间 $O(\sqrt{m})$, m 很大时内存仍可能不可用。不能把哈希表重复值覆盖导致最小解丢失。

5.12 扩展 BSGS · 主体

```

using namespace std;
using i64 = long long;
using i128 = __int128;
struct BSGS {
    static i64 mul(i64 a, i64 b, i64 m) {
        return i128(a) * b % m;
    }
    static i64 power(i64 a, i64 b, i64 m) {
        i64 r = 1 % m;
        for(; b >= 1, a = mul(a, a, m)) {
            if(b & 1) {
                r = mul(r, a, m);
            }
        }
        return r;
    }
    static i64 inv(i64 a, i64 m) {
        i128 x = 1, y = 0, u = 0, v = 1;
        i64 b = m;
        while(b) {
            i64 q = a / b, r = a % b;

```

5.12 扩展 BSGS · 主体 · 续

```

        a = b;
        b = r;
        i128 z = x - q * u;
        x = u;
        u = z;
        z = y - q * v;
        y = v;
        v = z;
    }
    return (x % m + m) % m;
}
// Smallest x >= 0 with a^x = b (mod m); -1 if impossible.
static i64 solve(i64 a, i64 b, i64 m) {
    assert(m > 0);
    if(m == 1) {
        return 0;
    }
    a = (i128(a) % m + m) % m;
    b = (i128(b) % m + m) % m;
    if(b == 1) {
        return 0;
    }
    i64 k = 1, cnt = 0, g;
    while((g = gcd(a, m)) > 1) {
        if(b % g) {
            return -1;
        }
        b /= g;
        m /= g;
        k = mul(k, a / g, m);
        ++cnt;
        if(k == b) {
            return cnt;
        }
    }
    b = mul(b, inv(k, m), m);
    i64 n = sqrtl(m) + 1, cur = 1 % m;
    unordered_map<i64, i64> mp;
    mp.reserve(n * 2);
    for(i64 j = 0; j < n; j++) {
        if(!mp.count(cur)) {
            mp[cur] = j;
        }
        cur = mul(cur, a, m);
    }
    i64 step = inv(power(a, n, m), m);
    cur = b;
    for(i64 i = 0; i <= n; i++) {
        auto it = mp.find(cur);
        if(it != mp.end()) {
            return cnt + i * n + it->second;
        }
        cur = mul(cur, step, m);
    }
    return -1;
}
};

```

不互素与最小指数

5.12 · 不互素与最小指数

```

int main() {
    assert(BSGS::solve(2, 4, 8) == 2);
    assert(BSGS::solve(2, 3, 8) == -1);
    assert(BSGS::solve(0, 0, 7) == 1);
}

```

5.13 模矩阵与快速幂

下标1-based, `Matrix(row,col)`, `operator()(i,j)`; 乘法 $O(rkc)$, 幂要求方阵与非负指数。固定 $MOD=998244353$, 可换正模数, 不做逆元时不要求素数。 `Matrix*vector` 使用列向量: 先 A 后 B 的总转移是 $B*A$; 元素需在 $[0,MOD)$ 。

5.13 模矩阵与快速幂 · 主体

```
using namespace std;
using i64 = long long;
constexpr int MOD = 998244353;
struct Matrix {
    int row, col;
    vector<int> mat;
    Matrix(int row_ = 0, int col_ = 0)
        : row(row_), col(col_), mat((row_ + 1) * (col_ + 1)) {}
    int& operator()(int i, int j) {
        return mat[i * (col + 1) + j];
    }
    const int& operator()(int i, int j) const {
        return mat[i * (col + 1) + j];
    }
    static Matrix identity(int n) {
        Matrix res(n, n);
        for(int i = 1; i <= n; i++) {
            res(i, i) = 1;
        }
        return res;
    }
};
Matrix operator*(const Matrix& a, const Matrix& b) {
    assert(a.col == b.row);
    Matrix res(a.row, b.col);
    for(int i = 1; i <= a.row; i++) {
        for(int k = 1; k <= a.col; k++) {
            int x = a(i, k);
            if(x == 0) {
                continue;
            }
            for(int j = 1; j <= b.col; j++) {
                res(i, j) = int((res(i, j) + 1LL * x * b(k, j)) % MOD);
            }
        }
    }
    return res;
}
Matrix power(Matrix a, i64 b) {
    assert(a.row == a.col);
    Matrix res = Matrix::identity(a.row);
    while(b > 0) {
        if(b & 1) {
            res = res * a;
        }
        a = a * a;
        b >>= 1;
    }
    return res;
}
vector<int> operator*(const Matrix& a, const vector<int>& v) {
    assert(a.col + 1 == int(v.size()));
    vector<int> res(a.row + 1, 0);
    for(int i = 1; i <= a.row; i++) {
        for(int j = 1; j <= a.col; j++) {
            res[i] = int((res[i] + 1LL * a(i, j) * v[j]) % MOD);
        }
    }
    return res;
}
```

列向量：斐波那契

5.13 · 列向量：斐波那契

```
int main() {
    Matrix a(2, 2);
    a(1, 1) = a(1, 2) = a(2, 1) = 1;
    auto v = power(a, 10) * vector<int>{0, 1, 0};
    assert(v[2] == 55); // [F(11), F(10)], 下标0占位。
}
```

5.14 杜教筛： φ 与 μ

get(n) 返回 $(\sum \varphi, \sum \mu)$ ，包括 $1..n$ ， $n \leq 0$ 为 0。由 $\varphi * 1 = \text{id}$ 、 $\mu * 1 = \varepsilon$ 得递推，分块减去 $l \geq 2$ 的重复贡献，记忆化不同 n/l 。 $\sum \varphi$ 为 i128， $\sum \mu$ 为 i64；不能直接 cout i128。预筛 limit 按内存与查询规模选，常取 $n^{2/3}$ 量级；不同询问可共享 memo。

递推： $\Phi(n) = n(n+1)/2 - \sum \text{块}(r-l+1)\Phi(n/l)$ ， $M(n) = 1 - \sum \text{块}(r-l+1)M(n/l)$ ，都从 $l=2$ 开始。limit 太小会拖慢，太大耗内存；按最大询问选一次预筛并复用对象。

5.14 杜教筛： φ 与 μ · 主体

```
using namespace std;
using i64 = long long;
using i128 = __int128_t;
struct DuJiaoSieve {
    int limit;
    vector<i64> prefPhi, prefMu;
    unordered_map<i64, pair<i128, i64>> memo;
    DuJiaoSieve(int limit_ = 2000000) : limit(limit_) {
        vector<int> minp(limit + 1), phi(limit + 1), mu(limit + 1);
        prefPhi.assign(limit + 1, 0);
        prefMu.assign(limit + 1, 0);
        if(limit < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        vector<int> primes;
        for(int i = 2; i <= limit; i++) {
            if(!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > limit) {
                    break;
                }
                int vv = int(v);
                minp[vv] = p;
                if(i % p == 0) {
                    phi[vv] = phi[i] * p;
                    mu[vv] = 0;
                    break;
                }
                phi[vv] = phi[i] * (p - 1);
                mu[vv] = -mu[i];
            }
        }
        for(int i = 1; i <= limit; i++) {
            prefPhi[i] = prefPhi[i - 1] + phi[i];
            prefMu[i] = prefMu[i - 1] + mu[i];
        }
    }
}
```

5.14 杜教筛: φ 与 μ · 主体 · 续

```

pair<i128, i64> get(i64 n) {
    if(n <= 0) {
        return {0, 0};
    }
    if(n <= limit) {
        return {prefPhi[n], prefMu[n]};
    }
    auto it = memo.find(n);
    if(it != memo.end()) {
        return it->second;
    }
    i128 sumPhi = i128(n) * (i128(n) + 1) / 2;
    i64 sumMu = 1;
    for(i64 l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        auto [x, y] = get(n / l);
        sumPhi -= i128(x) * (r - l + 1);
        sumMu -= y * (r - l + 1);
    }
    return memo[n] = {sumPhi, sumMu};
}
};

```

查询与 i128 输出

5.14 · 查询与 i128 输出

```

void print(__int128 x) {
    if(x < 0) {
        cout << '-', x = -x;
    }
    if(x >= 10) {
        print(x / 10);
    }
    cout << char('0' + x % 10);
}

int main() {
    DuJiaoSieve s(1000);
    auto [a, b] = s.get(10);
    assert(a == 32 && b == -1);
    print(a);
    cout << ' ' << b << '\n';
}

```

5.15 Min_25 积性函数求和

本版计算 $\sum_{i \leq n} f(i) \bmod P$, $f(1)=1$, P 为素数 >3 ; 默认洛谷 P5325: $f(p^e)=p^e(p^e-1)$ 。支持 $f(p)=c_0+c_1*p+c_2*p^2$ 的积性函数, 不是任意黑盒函数。修改时 fpe 、 $sumFp$ 、 $sumFpPrefix$ 三处必须一致; 不能把积性误当完全积性。空间 $O(\sqrt{n})$, 经典筛阶段约 $O(n^{3/4}/\log n)$, n 还受 $\sqrt{n}$ 数组与递归枚举耗时限制。

筛得到 $g_0/g_1/g_2$ 分别为不超过商值的质数个数、质数和、平方和; $sp_0/sp_1/sp_2$ 是前 j 个小质数相同前缀。 $S(x,j)$ 枚举最小质因子下标 $>=j$ 的非1整数, 先统计质数, 再枚举 p^e 与更大质因子部分; solve 最后补 $f(1)=1$ 。 g/sp 取值本来已模 P 。

5.15 Min_25 积性函数求和 · 主体

```

using namespace std;
using i64 = long long;
struct Min25Sieve {
    i64 n;
    int mod;
    int sq;
    vector<int> primes;
    vector<i64> w;
    vector<int> id1, id2;

```

5.15 Min_25 积性函数求和 · 主体 · 续

```

vector<i64> g0, g1, g2;
vector<i64> sp0, sp1, sp2;
Min25Sieve(i64 n_, int mod_) : n(n_), mod(mod_) {
    assert(n >= 1 && mod > 3);
    sq = int(sqrt((long double)n));
    while(1LL * (sq + 1) * (sq + 1) <= n) {
        ++sq;
    }
    while(1LL * sq * sq > n) {
        --sq;
    }
    id1.assign(sq + 2, -1);
    id2.assign(sq + 2, -1);
    initSieve(sq);
    build();
}
// ---- 按题目修改: f(p^e) ----
i64 fpe(i64 p, int e) const {
    // 默认 f(p^e) = p^e * (p^e - 1) (Luogu P5325)
    i64 pe = powerMod(p % mod, e, mod);
    return pe * ((pe - 1 + mod) % mod) % mod;
}
// f(p) = a0 + a1*p + a2*p^2, 默认 a0=0, a1=-1, a2=1 (即 p^2-p)
i64 sumFp(i64 x) const {
    int k = get(x);
    return (g2[k] - g1[k] + mod) % mod;
}
i64 sumFpPrefix(int j) const {
    return (sp2[j] - sp1[j] + mod) % mod;
}
i64 solve() {
    return (S(n, 0) + 1) % mod;
}

private:
i64 powerMod(i64 a, i64 e, int m) const {
    i64 res = 1 % m;
    a %= m;
    while(e > 0) {
        if(e & 1) {
            res = res * a % m;
        }
        a = a * a % m;
        e >>= 1;
    }
    return res;
}
void initSieve(int lim) {
    vector<int> minp(lim + 1);
    sp0.assign(lim + 2, 0);
    sp1.assign(lim + 2, 0);
    sp2.assign(lim + 2, 0);
    for(int i = 2; i <= lim; i++) {
        if(!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
            int j = int(primes.size());
            sp0[j] = sp0[j - 1] + 1;
            sp1[j] = (sp1[j - 1] + i) % mod;
            sp2[j] = (sp2[j - 1] + 1LL * i * i) % mod;
        }
        for(int p : primes) {
            i64 v = 1LL * i * p;
            if(v > lim) {
                break;
            }
            minp[int(v)] = p;
        }
    }
}

```

5.15 Min_25 积性函数求和 · 主体 · 续

```

        if(minp[i] == p) {
            break;
        }
    }
}

void build() {
    for(i64 l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        i64 v = n / l;
        w.push_back(v);
        if(v <= sq) {
            id1[int(v)] = int(w.size()) - 1;
        } else {
            id2[int(n / v)] = int(w.size()) - 1;
        }
    }
    i64 inv2 = powerMod(2, mod - 2, mod);
    i64 inv6 = powerMod(6, mod - 2, mod);
    g0.assign(w.size(), 0);
    g1.assign(w.size(), 0);
    g2.assign(w.size(), 0);
    for(int i = 0; i < int(w.size()); i++) {
        i64 x = w[i] % mod;
        g0[i] = (x - 1 + mod) % mod;
        g1[i] = (x * ((x + 1) % mod) % mod * inv2 % mod - 1 + mod) % mod;
        i64 t = x * ((x + 1) % mod) % mod * ((2 * x + 1) % mod) % mod;
        g2[i] = (t * inv6 % mod - 1 + mod) % mod;
    }
    for(int j = 0; j < int(primes.size()); j++) {
        i64 p = primes[j];
        i64 p2 = p * p;
        for(int i = 0; i < int(w.size()); i++) {
            if(w[i] < p2) {
                break;
            }
            int k = get(w[i] / p);
            g0[i] = (g0[i] - (g0[k] - sp0[j] + mod) % mod + mod) % mod;
            g1[i] = (g1[i] - p * ((g1[k] - sp1[j] + mod) % mod) % mod + mod) % mod;
            g2[i] =
                (g2[i] - p2 % mod * ((g2[k] - sp2[j] + mod) % mod) % mod + mod) % mod;
        }
    }
}

int get(i64 x) const {
    return x <= sq ? id1[int(x)] : id2[int(n / x)];
}

i64 S(i64 x, int j) {
    if(x <= 1 || (j < int(primes.size()) && primes[j] > x)) {
        return 0;
    }
    i64 ans = (sumFp(x) - sumFpPrefix(j) + mod) % mod;
    for(int i = j; i < int(primes.size()); i++) {
        i64 p = primes[i];
        if(p * p > x) {
            break;
        }
        i64 pe = p;
        int e = 1;
        while(pe <= x / p) {
            ans = (ans + fpe(p, e) * S(x / pe, i + 1)) % mod;
            ans = (ans + fpe(p, e + 1)) % mod;
            pe *= p;
            e++;
        }
    }
}

```

```

5.15 Min_25 积性函数求和 · 主体 · 续
    }
    return ans;
}
};

```

默认积性函数的调用

```

5.15 · 默认积性函数的调用
int main() {
    Min25Sieve s(10, 1000000007);
    assert(s.solve() == 263);
}

```

改成 φ ：替换三处函数

```

5.15 · 改成  $\varphi$ ：替换三处函数
i64 fpe(i64 p, int e) const {
    return powerMod(p % mod, e - 1, mod) * ((p - 1) % mod) % mod;
}
i64 sumFp(i64 x) const {
    int k = get(x);
    return (g1[k] - g0[k] + mod) % mod;
}
i64 sumFpPrefix(int j) const {
    return (sp1[j] - sp0[j] % mod + mod) % mod;
}

```

改成约数个数 τ ：替换三处函数

```

5.15 · 改成约数个数  $\tau$ ：替换三处函数
i64 fpe(i64 p, int e) const {
    return (e + 1) % mod;
}
i64 sumFp(i64 x) const {
    return 2 * g0[get(x)] % mod;
}
i64 sumFpPrefix(int j) const {
    return 2 * sp0[j] % mod;
}

```

改成 μ ：替换三处函数

```

5.15 · 改成  $\mu$ ：替换三处函数
i64 fpe(i64 p, int e) const {
    return e == 1 ? mod - 1 : 0;
}
i64 sumFp(i64 x) const {
    return (mod - g0[get(x)]) % mod;
}
i64 sumFpPrefix(int j) const {
    return (mod - sp0[j] % mod) % mod;
}

```

5.16 半环矩阵【速查】

把普通乘法的 $+$ 、 $*$ 改为对应“加法”和“乘法”，需满足结合、分配与单位元。min-plus：加法=min、乘法=路径长度相加，单位矩阵对角0其余INF；算恰好 k 条边最短路。不可达先判断再相加，避免 INF 溢出。

5.17 扩展欧拉降幂〔速查〕

模 $m > 1$ ，指数 $b \geq \varphi(m)$ 时可用 $b \% \varphi(m) + \varphi(m)$ ；小指数必须保留原值，需额外维护是否达到阈值。只记指数模 $\varphi(m)$ 无法判断是否加 $\varphi(m)$ 。 $m=1$ 立即0，约定 $0^0=1$ ；合数底数不互素时尤其不能直接费马降幂。

6. 多项式

6.1 FFT 整数卷积

`convolutionFft(int向量,int向量)` 返回 `i64`；空输入返回空。长度补到足够的2次幂， $O(n \log n)$ 。浮点误差随长度与系数规模增长，没有统一“ $1e14$ 都准确”保证；需要严格模结果用 NTT，任意模数可用多模 NTT+CRT。

6.1 FFT 整数卷积 · 主体

```
using namespace std;
using i64 = long long;
using db = double;
using cdb = complex<db>;
void fft(vector<cdb>& a, bool invert) {
    int n = int(a.size());
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for(; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <<= 1) {
        db ang = 2 * acos(-1.0) / len * (invert ? -1 : 1);
        cdb wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {
            cdb w(1);
            for(int j = 0; j < len / 2; j++) {
                cdb u = a[i + j];
                cdb v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if(invert) {
        for(cdb& x : a) {
            x /= n;
        }
    }
}
vector<i64> convolutionFft(const vector<int>& a, const vector<int>& b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    int need = int(a.size() + b.size() - 1);
    int n = 1;
    while(n < need) {
        n <<= 1;
    }
    vector<cdb> A(n), B(n);
    for(int i = 0; i < int(a.size()); i++) {
        A[i] = a[i];
    }
    for(int i = 0; i < int(b.size()); i++) {
        B[i] = b[i];
    }
    fft(A, false);
    fft(B, false);
    for(int i = 0; i < n; i++) {
        A[i] *= B[i];
    }
    fft(A, true);
}
```

6.1 FFT 整数卷积 · 主体 · 续

```

vector<i64> res(need);
for(int i = 0; i < need; i++) {
    res[i] = llround(A[i].real());
}
return res;
}

```

6.2 NTT 模 998244353

convolution 支持负系数并归一化；结果长 $a.size()+b.size()-1$ 。大卷积的补齐长度 $\leq 2^{23}$ ，原根3；改模数时必须同时核对原根与2次幂上限。convolutionTrunc 先做足够长的线性卷积再截断，不能只按输出长度补齐。

6.2 NTT 模 998244353 · 主体

```

using namespace std;
using i64 = long long;
constexpr int MOD = 998244353;
constexpr int G = 3;
constexpr int MAX_NTT_SIZE = 1 << 23;
int normalize(i64 x) {
    x %= MOD;
    if(x < 0) {
        x += MOD;
    }
    return int(x);
}
int powerMod(int a, int b) {
    a = normalize(a);
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % MOD);
        }
        a = int(1LL * a * a % MOD);
        b >>= 1;
    }
    return res;
}
void ntt(vector<int>& a, bool invert) {
    int n = int(a.size());
    if(n <= 1) {
        return;
    }
    assert((n & (n - 1)) == 0);
    assert(n <= MAX_NTT_SIZE);
    assert((MOD - 1) % n == 0);
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <<= 1) {
        int wlen = powerMod(G, (MOD - 1) / len);
        if(invert) {
            wlen = powerMod(wlen, MOD - 2);
        }
        int half = len >> 1;
        for(int left = 0; left < n; left += len) {
            i64 w = 1;
            for(int j = 0; j < half; j++) {

```

6.2 NTT 模 998244353 · 主体 · 续

```

        int u = a[left + j];
        int v = int(w * a[left + j + half] % MOD);
        int sum = u + v;
        if(sum >= MOD) {
            sum -= MOD;
        }
        int diff = u - v;
        if(diff < 0) {
            diff += MOD;
        }
        a[left + j] = sum;
        a[left + j + half] = diff;
        w = w * wlen % MOD;
    }
}
}
if(invert) {
    int invN = powerMod(n, MOD - 2);
    for(int& x : a) {
        x = int(1LL * x * invN % MOD);
    }
}
}
vector<int> convolutionNaive(const vector<int>& a, const vector<int>& b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    vector<int> c(a.size() + b.size() - 1);
    for(int i = 0; i < int(a.size()); i++) {
        for(int j = 0; j < int(b.size()); j++) {
            c[i + j] = int((c[i + j] + 1LL * a[i] * b[j]) % MOD);
        }
    }
    return c;
}
vector<int> convolution(vector<int> a, vector<int> b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    for(int& x : a) {
        x = normalize(x);
    }
    for(int& x : b) {
        x = normalize(x);
    }
    int resSize = int(a.size() + b.size() - 1);
    if(1LL * a.size() * b.size() <= 4096) {
        return convolutionNaive(a, b);
    }
    assert(resSize <= MAX_NTT_SIZE);
    int n = 1;
    while(n < resSize) {
        n <<= 1;
    }
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % MOD);
    }
    ntt(a, true);
    a.resize(resSize);
    return a;
}
vector<int> convolutionTrunc(vector<int> a, vector<int> b, int limit) {
    if(limit <= 0) {

```

6.2 NTT 模 998244353 · 主体 · 续

```

    return {};
}
if(int(a.size()) > limit) {
    a.resize(limit);
}
if(int(b.size()) > limit) {
    b.resize(limit);
}
vector<int> c = convolution(a, b);
if(int(c.size()) > limit) {
    c.resize(limit);
}
return c;
}

```

6.3 FWT: OR / AND / XOR

convOr/And/Xor 输出补齐到2次幂的按位卷积，系数模 $P=998244353$ 。OR/AND 的逆变换为加减反演；XOR 每层除2，故要求2可逆。空输入返回空；这些卷积下标是位集合运算，不是普通下标相加。

6.3 FWT: OR / AND / XOR · 主体

```

using namespace std;
using i64 = long long;
constexpr int P = 998244353;
int powerMod(int a, i64 b) {
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % P);
        }
        a = int(1LL * a * a % P);
        b >>= 1;
    }
    return res;
}
// OR 卷积变换: 正变换 y += x, 逆变换 y -= x
void fwtOr(vector<int>& a, bool inv) {
    int n = int(a.size());
    for(int len = 1; len < n; len <= 1) {
        for(int i = 0; i < n; i += len < 1) {
            for(int j = 0; j < len; j++) {
                int& x = a[i + j];
                int& y = a[i + j + len];
                if(!inv) {
                    y += x;
                    if(y >= P) {
                        y -= P;
                    }
                } else {
                    y -= x;
                    if(y < 0) {
                        y += P;
                    }
                }
            }
        }
    }
}
// AND 卷积变换: 正变换 x += y, 逆变换 x -= y
void fwtAnd(vector<int>& a, bool inv) {
    int n = int(a.size());
    for(int len = 1; len < n; len <= 1) {
        for(int i = 0; i < n; i += len < 1) {
            for(int j = 0; j < len; j++) {
                int& x = a[i + j];

```

6.3 FWT: OR / AND / XOR · 主体 · 续

```

        int& y = a[i + j + len];
        if(!inv) {
            x += y;
            if(x >= P) {
                x -= P;
            }
        } else {
            x -= y;
            if(x < 0) {
                x += P;
            }
        }
    }
}

// XOR 卷积变换: (x, y) -> (x + y, x - y), 逆变换同蝴蝶再乘 n^{-1}
void fwtXor(vector<int>& a, bool inv) {
    int n = int(a.size());
    for(int len = 1; len < n; len <= 1) {
        for(int i = 0; i < n; i += len << 1) {
            for(int j = 0; j < len; j++) {
                int x = a[i + j];
                int y = a[i + j + len];
                a[i + j] = x + y >= P ? x + y - P : x + y;
                a[i + j + len] = x - y < 0 ? x - y + P : x - y;
            }
        }
    }
    if(inv) {
        int invN = powerMod(n, P - 2);
        for(int& v : a) {
            v = int(1LL * v * invN % P);
        }
    }
}

vector<int> convOr(vector<int> a, vector<int> b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    for(int& x : a) {
        x = (x % P + P) % P;
    }
    for(int& x : b) {
        x = (x % P + P) % P;
    }
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtOr(a, false);
    fwtOr(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
    fwtOr(a, true);
    return a;
}

vector<int> convAnd(vector<int> a, vector<int> b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    for(int& x : a) {
        x = (x % P + P) % P;
    }
}

```

6.3 FWT: OR / AND / XOR · 主体 · 续

```

    for(int& x : b) {
        x = (x % P + P) % P;
    }
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <<= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtAnd(a, false);
    fwtAnd(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
    fwtAnd(a, true);
    return a;
}
vector<int> convXor(vector<int> a, vector<int> b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    for(int& x : a) {
        x = (x % P + P) % P;
    }
    for(int& x : b) {
        x = (x % P + P) % P;
    }
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <<= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtXor(a, false);
    fwtXor(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
    fwtXor(a, true);
    return a;
}

```

6.4 形式幂级数 inv / ln / exp

模998244353, 输入系数已规范化, limit为保留系数个数。polyInv 要 $a_0! \neq 0$; polyLog 返回常数0的 $\ln(a/a_0)$, 标准 ln 要 $a_0=1$; polyExp 要 $a_0=0$ (空视为0)。limit<P且内部卷积补齐 $\leq 2^{23}$ 。Newton 每轮加倍, $O(n \log n)$; 截断卷积不得发生循环混叠。

6.4 形式幂级数 inv / ln / exp · 主体

```

using namespace std;
using i64 = long long;
constexpr int FPS_MOD = 998244353;
constexpr int FPS_G = 3;
int fpsPower(int a, int b) {
    a %= FPS_MOD;
    if(a < 0) {
        a += FPS_MOD;
    }
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % FPS_MOD);
        }
        a = int(1LL * a * a % FPS_MOD);
    }
}

```

6.4 形式幂级数 $\text{inv} / \ln / \exp$ · 主体 · 续

```

    b >>= 1;
}
return res;
}
void fpsNtt(vector<int>& a, bool invert) {
    int n = int(a.size());
    if(n <= 1) {
        return;
    }
    assert((n & (n - 1)) == 0);
    assert((FPS_MOD - 1) % n == 0);
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <= 1) {
        int wlen = fpsPower(FPS_G, (FPS_MOD - 1) / len);
        if(invert) {
            wlen = fpsPower(wlen, FPS_MOD - 2);
        }
        int half = len >> 1;
        for(int left = 0; left < n; left += len) {
            i64 w = 1;
            for(int j = 0; j < half; j++) {
                int u = a[left + j];
                int v = int(w * a[left + j + half] % FPS_MOD);
                int sum = u + v;
                if(sum >= FPS_MOD) {
                    sum -= FPS_MOD;
                }
                int diff = u - v;
                if(diff < 0) {
                    diff += FPS_MOD;
                }
                a[left + j] = sum;
                a[left + j + half] = diff;
                w = w * wlen % FPS_MOD;
            }
        }
    }
    if(invert) {
        int invN = fpsPower(n, FPS_MOD - 2);
        for(int& x : a) {
            x = int(1LL * x * invN % FPS_MOD);
        }
    }
}
vector<int> fpsConvTrunc(vector<int> a, vector<int> b, int limit) {
    if(limit <= 0) {
        return {};
    }
    if(int(a.size()) > limit) {
        a.resize(limit);
    }
    if(int(b.size()) > limit) {
        b.resize(limit);
    }
    if(a.empty() || b.empty()) {
        return vector<int>(limit, 0);
    }
}

```

6.4 形式幂级数 $\text{inv} / \ln / \exp$ · 主体 · 续

```

    int resSize = int(a.size() + b.size() - 1);
    if(1LL * a.size() * b.size() <= 4096) {
        vector<int> c(min(resSize, limit), 0);
        for(int i = 0; i < int(a.size()); i++) {
            for(int j = 0; j < int(b.size()) && i + j < limit; j++) {
                c[i + j] = int((c[i + j] + 1LL * a[i] * b[j]) % FPS_MOD);
            }
        }
        return c;
    }
    int n = 1;
    while(n < resSize) {
        n <<= 1;
    }
    a.resize(n);
    b.resize(n);
    fpsNtt(a, false);
    fpsNtt(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % FPS_MOD);
    }
    fpsNtt(a, true);
    a.resize(min(resSize, limit));
    return a;
}
//  $a(x)^{-1} \bmod x^{\text{limit}}$ , 要求  $a[0] \neq 0$ 
vector<int> polyInv(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(!a.empty() && a[0] != 0);
    vector<int> b{fpsPower(a[0], FPS_MOD - 2)};
    while(int(b.size()) < limit) {
        int len = min(2 * int(b.size()), limit);
        vector<int> f(min(int(a.size()), len));
        copy(a.begin(), a.begin() + int(f.size()), f.begin());
        vector<int> t = fpsConvTrunc(f, b, len);
        t.resize(len);
        for(int& x : t) {
            if(x != 0) {
                x = FPS_MOD - x;
            }
        }
        t[0] += 2;
        if(t[0] >= FPS_MOD) {
            t[0] -= FPS_MOD;
        }
        b = fpsConvTrunc(b, t, len);
        b.resize(len);
    }
    return b;
}
//  $\ln(a(x)) \bmod x^{\text{limit}}$ , 要求  $a[0] \neq 0$  (通常为 1)
vector<int> polyLog(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(!a.empty() && a[0] != 0);
    vector<int> inv = polyInv(a, limit);
    vector<int> der(min(int(a.size()) - 1, limit));
    for(int i = 0; i < int(der.size()); i++) {
        der[i] = int(1LL * a[i + 1] * (i + 1) % FPS_MOD);
    }
    vector<int> prod = fpsConvTrunc(der, inv, limit);
    prod.resize(limit, 0);
    vector<int> invInt(limit + 1, 1);
    for(int i = 2; i <= limit; i++) {

```

6.4 形式幂级数 $\text{inv} / \ln / \exp$ · 主体 · 续

```

    invInt[i] = int(1LL * (FPS_MOD - FPS_MOD / i) * invInt[FPS_MOD % i] % FPS_MOD);
}
vector<int> res(limit, 0);
for(int i = 1; i < limit; i++) {
    res[i] = int(1LL * prod[i - 1] * invInt[i] % FPS_MOD);
}
return res;
}
// exp(a(x)) mod x^limit, 要求 a[0] == 0
vector<int> polyExp(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(a.empty() || a[0] == 0);
    vector<int> b{1};
    while(int(b.size()) < limit) {
        int len = min(2 * int(b.size()), limit);
        vector<int> lb = polyLog(b, len);
        vector<int> f(len, 0);
        for(int i = 0; i < len && i < int(a.size()); i++) {
            f[i] = a[i];
        }
        for(int i = 0; i < len; i++) {
            f[i] = (f[i] - lb[i] + FPS_MOD) % FPS_MOD;
        }
        f[0] = (f[0] + 1) % FPS_MOD; // 1 + a - ln(b)
        b = fpsConvTrunc(b, f, len);
        b.resize(len);
    }
    return b;
}

```

逆、对数与指数的约定

6.4 · 逆、对数与指数的约定

```

int main() {
    vector<int> a = {1, 2, 3};
    auto b = polyInv(a, 8);
    auto c = fpsConvTrunc(a, b, 8);
    c.resize(8);
    assert(c[0] == 1);
    for(int i = 1; i < 8; i++) {
        assert(c[i] == 0);
    }
    auto d = polyExp(polyLog(a, 8), 8);
    a.resize(8);
    assert(d == a);
}

```

7. 线性代数

7.1 Gauss-Jordan 消元

$a[1..n][1..m]$ 为系数，最后一列默认 RHS；矩形系统也按最后列处理，求纯矩阵秩传 $\text{variables}=\text{列数}$ 。返回秩与主元列对应行；必须另查零系数行非零 RHS 判断无解，秩 < 变量数为多解。mod 版要求素数、输入归一化；double 版采用最大绝对值主元，误差仍依条件数。

仅求秩而不带右端列： $\text{gaussMod}(a,P,w,\text{列数})$ 。浮点判矛盾用 $\text{abs}(\text{rhs}) > \text{eps}$ ，输入条件很差时加大容差也未必可靠。

7.1 Gauss-Jordan 消元 · 主体

```
using namespace std;
using i64 = long long;
using db = double;
constexpr db EPS = 1e-9;
int gaussDouble(vector<vector<db>>& a, vector<int>& pivotCol, db eps = EPS,
                int variables = -1) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    if(n <= 0) {
        return 0;
    }
    int row = 1;
    int limit = variables < 0 ? m - 1 : variables;
    assert(0 <= limit && limit <= m);
    for(int col = 1; col <= limit && row <= n; col++) {
        int best = row;
        for(int i = row + 1; i <= n; i++) {
            if(fabs(a[i][col]) > fabs(a[best][col])) {
                best = i;
            }
        }
        if(fabs(a[best][col]) < eps) {
            continue;
        }
        swap(a[best], a[row]);
        db inv = 1.0 / a[row][col];
        for(int j = col; j <= m; j++) {
            a[row][j] *= inv;
        }
        for(int i = 1; i <= n; i++) {
            if(i == row) {
                continue;
            }
            db fac = a[i][col];
            for(int j = col; j <= m; j++) {
                a[i][j] -= fac * a[row][j];
            }
        }
        pivotCol[col] = row++;
    }
    return row - 1;
}
int gaussMod(vector<vector<int>>& a, int mod, vector<int>& pivotCol, int variables = -1) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    if(n <= 0) {
        return 0;
    }
    int row = 1;
    auto powerMod = [&](int x, i64 e) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
```

7.1 Gauss-Jordan 消元 · 主体 · 续

```

        res = int(1LL * res * x % mod);
    }
    x = int(1LL * x * x % mod);
    e >>= 1;
}
return res;
};
int limit = variables < 0 ? m - 1 : variables;
assert(0 <= limit && limit <= m);
for(int col = 1; col <= limit && row <= n; col++) {
    int best = 0;
    for(int i = row; i <= n; i++) {
        if(a[i][col] != 0) {
            best = i;
            break;
        }
    }
    if(best == 0) {
        continue;
    }
    swap(a[best], a[row]);
    int inv = powerMod(a[row][col], mod - 2);
    for(int j = col; j <= m; j++) {
        a[row][j] = int(1LL * a[row][j] * inv % mod);
    }
    for(int i = 1; i <= n; i++) {
        if(i == row) {
            continue;
        }
        int fac = a[i][col];
        for(int j = col; j <= m; j++) {
            a[i][j] = int((a[i][j] - 1LL * fac * a[row][j] % mod + mod) % mod);
        }
    }
    pivotCol[col] = row++;
}
return row - 1;
}
int gaussXor(vector<vector<int>>& a, vector<int>& pivotCol, int variables = -1) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    if(n <= 0) {
        return 0;
    }
    int row = 1;
    int limit = variables < 0 ? m - 1 : variables;
    assert(0 <= limit && limit <= m);
    for(int col = 1; col <= limit && row <= n; col++) {
        int best = 0;
        for(int i = row; i <= n; i++) {
            if(a[i][col]) {
                best = i;
                break;
            }
        }
        if(best == 0) {
            continue;
        }
        swap(a[best], a[row]);
        for(int i = 1; i <= n; i++) {
            if(i != row && a[i][col]) {
                for(int j = col; j <= m; j++) {
                    a[i][j] ^= a[row][j];
                }
            }
        }
    }
}

```

7.1 Gauss-Jordan 消元 · 主体 · 续

```

        pivotCol[col] = row++;
    }
    return row - 1;
}

```

模素数方程：判无解并读答案

7.1 · 模素数方程：判无解并读答案

```

int main() {
    const int P = 101, n = 3, m = 2;
    vector<vector<int>> a = {{0, 0, 0, 0}, {0, 1, 2, 5}, {0, 2, 100, 0}, {0, 3, 1, 5}};
    vector<int> w;
    int r = gaussMod(a, P, w);
    bool ok = true;
    for(int i = r + 1; i <= n; i++) {
        if(a[i][m + 1]) {
            ok = false;
        }
    }
    assert(ok);
    vector<int> x(m + 1); // 自由变量取0。
    for(int j = 1; j <= m; j++) {
        if(w[j]) {
            x[j] = a[w[j]][m + 1];
        }
    }
    assert(x[1] == 1 && x[2] == 2);
}

```

7.2 64 位异或线性基

insert、decompose、maxXor、maxWith; zero 表示存在非空子集异或0。minXor(ans)、kth(k,ans) 的 bool 区分无答案与合法 ULLONG_MAX; k从1起，枚举非空子集的“不同异或值”，不是带重数第k项。rank64且包含0时共有 2^{64} 个值，最后那个序号无法用 u64 表示。

7.2 64 位异或线性基 · 主体

```

using namespace std;
using u64 = unsigned long long;
struct XorBasis {
    array<u64, 64> b{};
    int num = 0;
    bool zero = false;
    vector<u64> basis;
    bool insert(u64 v) {
        for(int i = 63; i >= 0; i--) {
            if(!(v >> i)) {
                continue;
            }
            if(b[i] == 0) {
                b[i] = v;
                num++;
                basis.clear();
                return true;
            }
            v ^= b[i];
        }
        zero = true;
        return false;
    }
    bool decompose(u64 v) const {
        for(int i = 63; i >= 0; i--) {
            if(!(v >> i)) {
                continue;
            }
        }
    }
}

```

7.2 64 位异或线性基 · 主体 · 续

```

        if(b[i] == 0) {
            return false;
        }
        v ^= b[i];
    }
    return true;
}
u64 maxXor() const {
    u64 res = 0;
    for(int i = 63; i >= 0; i--) {
        res = max(res, res ^ b[i]);
    }
    return res;
}
u64 maxWith(u64 v) const {
    u64 res = v;
    for(int i = 63; i >= 0; i--) {
        res = max(res, res ^ b[i]);
    }
    return res;
}
bool minXor(u64& ans) const {
    if(zero) {
        ans = 0;
        return true;
    }
    for(int i = 0; i < 64; i++) {
        if(b[i]) {
            ans = b[i];
            return true;
        }
    }
    return false;
}
void reduce() {
    if(!basis.empty()) {
        return;
    }
    vector<u64> tmp(b.begin(), b.end());
    for(int i = 0; i < 64; i++) {
        if(tmp[i] == 0) {
            continue;
        }
        for(int j = i - 1; j >= 0; j--) {
            if(tmp[i] >> j & 1) {
                tmp[i] ^= tmp[j];
            }
        }
        basis.push_back(tmp[i]);
    }
}
bool kth(u64 k, u64& res) {
    if(k == 0) {
        return false;
    }
    reduce();
    if(zero) {
        k--;
    }
    if(basis.size() < 64 && k >= (u64(1) << basis.size())) {
        return false;
    }
    res = 0;
    for(int i = 0; i < int(basis.size()); i++) {
        if(k >> i & 1) {
            res ^= basis[i];
        }
    }
}

```

7.2 64 位异或线性基 · 主体 · 续

```

    }
    return true;
}
void merge(const XorBasis& other) {
    zero = zero || other.zero;
    for(int i = 63; i >= 0; i--) {
        if(other.b[i]) {
            insert(other.b[i]);
        }
    }
}
};

```

非空子集与第 k 个不同值

7.2 · 非空子集与第 k 个不同值

```

int main() {
    XorBasis s;
    s.insert(1);
    s.insert(2);
    s.insert(3);
    unsigned long long x;
    assert(s.zero && s.kth(1, x) && x == 0);
    assert(s.kth(4, x) && x == 3);
    assert(!s.kth(5, x));
}

```

7.3 GF(2) bitset 方程组

XorLinearSystem<MAXC>(n,m,p), 行列0-based, m个变量+p个RHS需 $\leq \text{MAXC}$ 。solve 消元, 若某右端无解会整体 false; answer 把自由变量置0, 给每个 RHS 一组解。单个RHS的解数可由自由变量数 $m - \text{rank}$ 得 $2^{(m - \text{rank})}$, 按题目取模或大数。

7.3 GF(2) bitset 方程组 · 主体

```

using namespace std;
// 行向量 a[i]: 低 m 位是变量系数, 第 m+k 位是第 k 个右端项
template <int MAXC> struct XorLinearSystem {
    using B = bitset<MAXC>;
    int n, m, p;
    vector<B> a;
    vector<int> where; // 主元变量所在行, -1 表示自由变量
    XorLinearSystem(int n_, int m_, int p_) : n(n_), m(m_), p(p_), a(n_), where(m_, -1) {}
    void setCoeff(int row, int col, bool v) {
        a[row].set(col, v);
    }
    void setRhs(int row, int k, bool v) {
        a[row].set(m + k, v);
    }
    bool solve() {
        where.assign(m, -1);
        int r = 0;
        for(int c = 0; c < m && r < n; c++) {
            int s = -1;
            for(int i = r; i < n; i++) {
                if(a[i][c]) {
                    s = i;
                    break;
                }
            }
            if(s == -1) {
                continue;
            }
            swap(a[r], a[s]);

```

7.3 GF(2) bitset 方程组 · 主体 · 续

```

        where[c] = r;
        for(int i = 0; i < n; i++) {
            if(i != r && a[i][c]) {
                a[i] ^= a[r];
            }
        }
        r++;
    }
    for(int i = 0; i < n; i++) {
        bool lhs = false;
        bool rhs = false;
        for(int j = 0; j < m; j++) {
            lhs |= a[i][j];
        }
        for(int k = 0; k < p; k++) {
            rhs |= a[i][m + k];
        }
        if(!lhs && rhs) {
            return false;
        }
    }
    return true;
}
// 仅在 solve() 返回 true 后调用; x[var][k] 为第 k 个右端项下该变量的取值
vector<vector<int>> answer() const {
    vector<vector<int>> x(m, vector<int>(p, 0));
    for(int c = 0; c < m; c++) {
        if(where[c] == -1) {
            continue;
        }
        int row = where[c];
        for(int k = 0; k < p; k++) {
            x[c][k] = a[row][m + k] ? 1 : 0;
        }
    }
    return x;
}
};

```

8. 字符串

8.1 KMP

`kmpMatch(text, pattern)` 返回1-based匹配起点，允许重叠；空模式约定返回空列表。`getNext` 是1-based前缀函数， $O(n+m)$ 。字符串若有空格要用 `getline` 读入。

8.1 KMP · 主体

```
using namespace std;
vector<int> getNext(const string& pattern) {
    int m = int(pattern.size());
    string p = " " + pattern;
    vector<int> next(m + 1, 0);
    for(int i = 2; i <= m; i++) {
        int j = next[i - 1];
        while(j && p[i] != p[j + 1]) {
            j = next[j];
        }
        if(p[i] == p[j + 1]) {
            j++;
        }
        next[i] = j;
    }
    return next;
}
vector<int> kmpMatch(const string& text, const string& pattern) {
    vector<int> next = getNext(pattern);
    int n = int(text.size());
    int m = int(pattern.size());
    vector<int> result;
    if(m == 0) {
        return result;
    }
    string t = " " + text;
    string p = " " + pattern;
    int j = 0;
    for(int i = 1; i <= n; i++) {
        while(j && t[i] != p[j + 1]) {
            j = next[j];
        }
        if(t[i] == p[j + 1]) {
            j++;
        }
        if(j == m) {
            result.push_back(i - m + 1);
            j = next[j];
        }
    }
    return result;
}
```

8.2 Z / 扩展 KMP

`zFunction(s)` 为1-based, $z[1]=n$; `exKmp(text, pattern)[i]` 是文本第*i*位置后缀与模式的 LCP，都是 $O(n+m)$ 。空串安全。若用拼接求匹配，分隔符必须保证不在字符集中。

8.2 Z / 扩展 KMP · 主体

```
using namespace std;
vector<int> zFunction(const string& s) {
    string t = " " + s;
    int n = int(s.size());
    vector<int> z(n + 1);
    if(n == 0) {
        return z;
    }
```

8.2 Z / 扩展 KMP · 主体 · 续

```

    }
    z[1] = n;
    for(int i = 2, l = 0, r = 0; i <= n; i++) {
        if(i <= r) {
            z[i] = min(z[i - l + 1], r - i + 1);
        }
        while(i + z[i] <= n && t[1 + z[i]] == t[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
vector<int> exKmp(const string& text, const string& pattern) {
    vector<int> z = zFunction(pattern);
    string t = " " + text;
    string p = " " + pattern;
    int n = int(text.size());
    int m = int(pattern.size());
    vector<int> ext(n + 1);
    for(int i = 1, l = 0, r = 0; i <= n; i++) {
        if(i <= r) {
            ext[i] = min(z[i - l + 1], r - i + 1);
        }
        while(i + ext[i] <= n && ext[i] < m && p[1 + ext[i]] == t[i + ext[i]]) {
            ext[i]++;
        }
        if(i + ext[i] - 1 > r) {
            l = i;
            r = i + ext[i] - 1;
        }
    }
    return ext;
}

```

8.3 64 位滚动哈希

get(l,r) 为 1-based 闭区间，溢出按 unsigned 的 2^{64} 模运算，字节按 unsigned char 处理。两串比较要用同一 base；默认固定 base 不是抗构造保证。哈希有碰撞，要求确定性用 SA/KMP；可随机奇 base、双模或最终原串核验。

8.3 64 位滚动哈希 · 主体

```

using namespace std;
using u64 = unsigned long long;
struct RollingHash {
    u64 base;
    vector<u64> pw, h;
    RollingHash(const string& s, u64 base_ = 19260817) : base(base_) {
        string t = " " + s;
        int n = int(s.size());
        pw.assign(n + 1, 1);
        h.assign(n + 1, 0);
        for(int i = 1; i <= n; i++) {
            pw[i] = pw[i - 1] * base;
            h[i] = h[i - 1] * base + u64((unsigned char)t[i]) + 1;
        }
    }
    u64 get(int l, int r) const {
        return h[r] - h[l - 1] * pw[r - l + 1];
    }
};

```

8.4 子序列自动机

SeqAM(s, sigma, base) 接连续字符集，例如数字 sigma=10, base='0'; 构造 $O(n\sigma)$, check(t) $O(|t|)$, 空串匹配。原串字符必须落在此范围；目标串越界返回 false。它匹配子序列，不是连续子串。

8.4 子序列自动机 · 主体

```
using namespace std;
struct SeqAM {
    int n, sigma;
    char base;
    vector<vector<int>> nxt; // nxt[i][c]: 从位置 i (含) 起第一个字符 c, 无则 n + 1
    SeqAM(const string& s, int sigma_, char offset = 'a')
        : n(int(s.size())), sigma(sigma_), base(offset),
          nxt(n + 2, vector<int>(sigma_, n + 1)) {
        for(int i = n; i >= 1; i--) {
            nxt[i] = nxt[i + 1];
            nxt[i][s[i - 1] - offset] = i;
        }
    }
    bool check(const string& t) const {
        int cur = 0;
        for(char c : t) {
            int x = c - base;
            if(x < 0 || x >= sigma) {
                return false;
            }
            cur = nxt[cur + 1][x];
            if(cur == n + 1) {
                return false;
            }
        }
        return true;
    }
};
```

8.5 Trie 多重字符串集

默认26小写, insert、erase删一份、count、countPrefix; 空串合法。节点0为根, ch的0表示无边。空间 $O(\sigma \cdot \text{总长度})$, 删除不回收节点; 字符范围变大时考虑映射或稀疏边。

8.5 Trie 多重字符串集 · 主体

```
using namespace std;
template <int SIGMA = 26, char BASE = 'a'> struct Trie {
    struct Node {
        array<int, SIGMA> ch{};
        int cnt = 0;
        int end = 0;
    };
    vector<Node> tr;
    Trie(int n = 0) {
        if(n) {
            tr.reserve(n);
        }
        tr.emplace_back();
    }
    int newNode() {
        tr.emplace_back();
        return int(tr.size()) - 1;
    }
    int id(char c) const {
        return c - BASE;
    }
    int size() const {
        return int(tr.size());
    }
};
```

8.5 Trie 多重字符串集 · 主体 · 续

```

void clear() {
    tr.clear();
    tr.emplace_back();
}
int go(int u, char c) {
    int x = id(c);
    if(!tr[u].ch[x]) {
        tr[u].ch[x] = newNode();
    }
    return tr[u].ch[x];
}
int next(int u, char c) const {
    int x = id(c);
    int v = tr[u].ch[x];
    return v ? v : -1;
}
int insert(const string& s) {
    int u = 0;
    tr[u].cnt++;
    for(char c : s) {
        u = go(u, c);
        tr[u].cnt++;
    }
    tr[u].end++;
    return u;
}
int find(const string& s) const {
    int u = 0;
    for(char c : s) {
        int x = id(c);
        if(!tr[u].ch[x]) {
            return -1;
        }
        u = tr[u].ch[x];
    }
    return u;
}
int count(const string& s) const {
    int u = find(s);
    return u == -1 ? 0 : tr[u].end;
}
int countPrefix(const string& s) const {
    int u = find(s);
    return u == -1 ? 0 : tr[u].cnt;
}
bool contains(const string& s) const {
    return count(s) > 0;
}
bool erase(const string& s) {
    int u = find(s);
    if(u == -1 || tr[u].end == 0) {
        return false;
    }
    tr[0].cnt--;
    u = 0;
    for(char c : s) {
        u = tr[u].ch[id(c)];
        tr[u].cnt--;
    }
    tr[u].end--;
    return true;
}
};

```

8.6 AC 自动机

先 add(非空小写模式), 保存终点编号, 再 build 一次; 构建后不能再插入。match(text) 返回各状态计数, 按保存终点读取每个模式次数, 重复模式可共用同一终点。逆 BFS 累加 fail, $O(\sigma \cdot \text{节点} + \text{文本长})$; 支持重叠, 不按 fail 链逐字枚举全部模式。

8.6 AC 自动机 · 主体

```
using namespace std;
struct AC {
    struct Node {
        array<int, 26> ch{};
        int fa = 0;
    };
    vector<Node> t;
    vector<int> ord;
    bool done = false;
    AC() : t(1) {}
    int add(const string& s) {
        assert(!done && !s.empty());
        int u = 0;
        for(char c : s) {
            int x = c - 'a';
            if(!t[u].ch[x]) {
                int v = t.size();
                t.emplace_back();
                t[u].ch[x] = v;
            }
            u = t[u].ch[x];
        }
        return u;
    }
    void build() {
        if(done) {
            return;
        }
        done = true;
        queue<int> q;
        for(int c = 0; c < 26; c++) {
            if(t[0].ch[c]) {
                q.push(t[0].ch[c]);
            }
        }
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            ord.push_back(u);
            for(int c = 0; c < 26; c++) {
                int v = t[u].ch[c];
                if(v) {
                    t[v].fa = t[t[u].fa].ch[c], q.push(v);
                } else {
                    t[u].ch[c] = t[t[u].fa].ch[c];
                }
            }
        }
    }
    // hits[end[i]] is the occurrence count of pattern i, allowing overlap.
    vector<long long> match(const string& s) {
        build();
        vector<long long> hit(t.size());
        int u = 0;
        for(char c : s) {
            u = t[u].ch[c - 'a'], ++hit[u];
        }
        for(int i = int(ord.size()) - 1; i >= 0; i--) {
            hit[t[ord[i]].fa] += hit[ord[i]];
        }
    }
};
```

```

8.6 AC 自动机 · 主体 · 续
    }
    return hit;
}
};

```

重复模式、重叠匹配

```

8.6 · 重复模式、重叠匹配
int main() {
    AC s;
    int x = s.add("aba"), y = s.add("ba"), z = s.add("aba");
    s.build();
    auto cnt = s.match("ababa");
    assert(cnt[x] == 2 && cnt[y] == 2 && cnt[z] == 2);
}

```

8.7 后缀自动机 SAM

顺序 add(c) 构建一个小写串，根0，fa=-1，无转移-1；普通新状态cnt=1，clonecnt=0。count() 按len降序沿fa累加，之后不可再add；可重复调用count而不重复累加。occ(t)、distinct()、lcs(t) 直接使用。状态u代表长度(len[fa[u]],len[u]) 的一组子串，共享endpos，别把状态数当不同子串数。

cnt先按每个前缀终点统计，再向fa传递，正是endpos大小；查一个串沿ch到达u后cnt[u]即次数。lcs在失配时沿fa跳并把当前长度截到新len，不是只换状态不换长度。要多串广义SAM不能每串简单last=0继续套普通extend。

```

8.7 后缀自动机 SAM · 主体
using namespace std;
struct SAM {
    struct Node {
        int len, fa, ch[26];
        long long cnt;
        Node() : len(0), fa(-1), cnt(0) {
            fill(ch, ch + 26, -1);
        }
    };
    vector<Node> t;
    vector<int> ord;
    int last, n;
    bool done;
    SAM() : t(1), last(0), n(0), done(false) {}
    void add(char c) {
        assert(!done);
        int x = c - 'a', u = t.size(), p = last;
        t.push_back(Node());
        t[u].len = ++n;
        t[u].cnt = 1;
        while(p != -1 && t[p].ch[x] == -1) {
            t[p].ch[x] = u, p = t[p].fa;
        }
        if(p == -1) {
            t[u].fa = 0;
        } else {
            int q = t[p].ch[x];
            if(t[p].len + 1 == t[q].len) {
                t[u].fa = q;
            } else {
                int v = t.size();
                t.push_back(t[q]);
                t[v].len = t[p].len + 1;
                t[v].cnt = 0;
                while(p != -1 && t[p].ch[x] == q) {
                    t[p].ch[x] = v, p = t[p].fa;
                }
            }
        }
    }
};

```

8.7 后缀自动机 SAM · 主体 · 续

```

    }
    t[q].fa = t[u].fa = v;
  }
  last = u;
}
void build(const string& s) {
  for(size_t i = 0; i < s.size(); i++) {
    add(s[i]);
  }
}
void count() {
  if(done) {
    return;
  }
  done = true;
  vector<int> c(n + 1);
  ord.resize(t.size());
  for(size_t i = 0; i < t.size(); i++) {
    ++c[t[i].len];
  }
  for(int i = 1; i <= n; i++) {
    c[i] += c[i - 1];
  }
  for(int i = int(t.size()) - 1; i >= 0; i--) {
    ord[--c[t[i].len]] = i;
  }
  for(int i = int(ord.size()) - 1; i > 0; i--) {
    int u = ord[i];
    t[t[u].fa].cnt += t[u].cnt;
  }
}
long long occ(const string& s) {
  count();
  int u = 0;
  for(size_t i = 0; i < s.size(); i++) {
    int x = s[i] - 'a';
    if(x < 0 || x >= 26 || t[u].ch[x] < 0) {
      return 0;
    }
    u = t[u].ch[x];
  }
  return s.empty() ? n + 1 : t[u].cnt;
}
long long distinct() const {
  long long ans = 0;
  for(size_t i = 1; i < t.size(); i++) {
    ans += t[i].len - t[t[i].fa].len;
  }
  return ans;
}
int lcs(const string& s) const {
  int u = 0, len = 0, ans = 0;
  for(size_t i = 0; i < s.size(); i++) {
    int x = s[i] - 'a';
    if(x < 0 || x >= 26) {
      u = len = 0;
      continue;
    }
    while(u && t[u].ch[x] < 0) {
      u = t[u].fa, len = t[u].len;
    }
    if(t[u].ch[x] >= 0) {
      u = t[u].ch[x], ++len;
    } else {
      u = len = 0;
    }
  }
}

```

8.7 后缀自动机 SAM · 主体 · 续

```

        ans = max(ans, len);
    }
    return ans;
};

```

构建 / 出现次数 / 不同子串 / LCS

8.7 · 构建 / 出现次数 / 不同子串 / LCS

```

int main() {
    SAM s;
    for(char c : string("ababa")) {
        s.add(c);
    }
    assert(s.distinct() == 9);
    s.count();
    assert(s.occ("aba") == 2 && s.occ("ba") == 2);
    assert(s.occ("ac") == 0);
    assert(s.lcs("cababd") == 4);
    long long ans = 0;
    for(int u = 1; u < int(s.t.size()); u++) {
        ans += s.t[u].len - s.t[s.t[u].fa].len;
    }
    assert(ans == s.distinct());
}

```

8.8 后缀数组 SA

SA(s) $O(n \log n)$ 倍增计数排序，字节字符；sa/rk/height为1-based， $\text{height}[i] = \text{LCP}(\text{sa}[i-1], \text{sa}[i])$ ， $\text{height}[1] = 0$ 。两后缀LCP是它们排名间 height 的最小值，可接ST；空串支持。

8.8 后缀数组 SA · 主体

```

using namespace std;
struct SA {
    int n;
    vector<int> sa, rk, height;
    SA(const string& str) {
        n = int(str.size());
        sa.assign(n + 1, 0);
        rk.assign(n + 1, 0);
        height.assign(n + 1, 0);
        if(n == 0) {
            return;
        }
        // 追加最小哨兵，使循环移位排序等价于后缀排序
        string t = str + char(0);
        int N = n + 1;
        vector<int> p(N), c(N);
        int alphabet = 256;
        vector<int> cnt(max(alphabet, N), 0);
        for(int i = 0; i < N; i++) {
            cnt[(unsigned char)t[i]]++;
        }
        for(int i = 1; i < alphabet; i++) {
            cnt[i] += cnt[i - 1];
        }
        for(int i = 0; i < N; i++) {
            p[--cnt[(unsigned char)t[i]]] = i;
        }
        c[p[0]] = 0;
        int classes = 1;
        for(int i = 1; i < N; i++) {
            if(t[p[i]] != t[p[i - 1]]) {
                classes++;
            }
        }
    }
};

```

8.8 后缀数组 SA · 主体 · 续

```

    }
    c[p[i]] = classes - 1;
}
vector<int> pn(N), cn(N);
for(int h = 0; (1 << h) < N; h++) {
    int step = 1 << h;
    for(int i = 0; i < N; i++) {
        pn[i] = p[i] - step;
        if(pn[i] < 0) {
            pn[i] += N;
        }
    }
    fill(cnt.begin(), cnt.begin() + classes, 0);
    for(int i = 0; i < N; i++) {
        cnt[c[pn[i]]]++;
    }
    for(int i = 1; i < classes; i++) {
        cnt[i] += cnt[i - 1];
    }
    for(int i = N - 1; i >= 0; i--) {
        p[--cnt[c[pn[i]]]] = pn[i];
    }
    cn[p[0]] = 0;
    int newClasses = 1;
    for(int i = 1; i < N; i++) {
        pair<int, int> cur = {c[p[i]], c[(p[i] + step) % N]};
        pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + step) % N]};
        if(cur != prev) {
            newClasses++;
        }
        cn[p[i]] = newClasses - 1;
    }
    c.swap(cn);
    classes = newClasses;
}
int k = 0;
for(int i = 0; i < N; i++) {
    if(p[i] == n) {
        continue;
    }
    sa[++k] = p[i] + 1;
}
for(int i = 1; i <= n; i++) {
    rk[sa[i]] = i;
}
k = 0;
for(int i = 0; i < n; i++) {
    int pos = rk[i + 1];
    if(pos == 1) {
        k = 0;
        continue;
    }
    int j = sa[pos - 1] - 1;
    while(i + k < n && j + k < n && str[i + k] == str[j + k]) {
        k++;
    }
    height[pos] = k;
    if(k) {
        k--;
    }
}
}
};

```

SA + height: 两后缀 LCP

8.8 · SA + height: 两后缀 LCP

```
int main() {
    SA s("banana");
    int a = s.rk[2], b = s.rk[4];
    if(a > b) {
        swap(a, b);
    }
    int ans = INT_MAX;
    for(int i = a + 1; i <= b; i++) {
        ans = min(ans, s.height[i]);
    }
    assert(ans == 3); // anana 与 ana; 多询问用ST预处理height。
}
```

8.9 最小循环表示

minRotation(s) 返回0-based起点, 答案=s.substr(k)+s.substr(0,k), $O(n)$ 。空串返回0; 多个最优起点任取。不同字符串的循环同构比较先规范化, 不能仅比较最优起点编号。

8.9 最小循环表示 · 主体

```
using namespace std;
int minRotation(const string& s) {
    int n = int(s.size());
    if(n == 0) {
        return 0;
    }
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while(i < n && j < n && k < n) {
        unsigned char a = t[i + k];
        unsigned char b = t[j + k];
        if(a == b) {
            k++;
            continue;
        }
        if(a > b) {
            i += k + 1;
            if(i == j) {
                j++;
            }
        } else {
            j += k + 1;
            if(i == j) {
                j++;
            }
        }
        k = 0;
    }
    return min(i, j);
}
```

8.10 Manacher

longestPal 返回最长回文长度, $O(n)$; manacherRadius 的扩展字符数组用独立整数哨兵, 原串含 \$/ #/ & 也安全。半径包括中心, 原串回文长度是半径-1; 空串为0。

8.10 Manacher · 主体

```
using namespace std;
vector<int> manacherRadius(const string& s) {
    vector<int> t = {257, 256};
    for(char c : s) {
        t.push_back((unsigned char)c);
        t.push_back(256);
    }
    t.push_back(258);
    int n = int(t.size());
    vector<int> P(n);
    int R = 0, C = 0;
    for(int i = 1; i < n - 1; i++) {
        P[i] = i < R ? min(P[2 * C - i], P[C] + C - i) : 1;
        while(t[i + P[i]] == t[i - P[i]]) {
            P[i]++;
        }
        if(P[i] + i > R) {
            R = P[i] + i;
            C = i;
        }
    }
    return P;
}
int longestPal(const string& s) {
    vector<int> P = manacherRadius(s);
    int ans = 1;
    for(int x : P) {
        ans = max(ans, x);
    }
    return ans - 1;
}
```

8.11 回文自动机 PAM

PAM(s) 一次构建小写串, 0为长度-1根, 1为长度0根, 实回文节点 ≥ 2 。distinct 返回不同回文个数; count 后 cnt 为出现次数, 重复count不再累加。len*cnt 需转 i64; count 后不再 extend。

8.11 回文自动机 PAM · 主体

```
using namespace std;
struct PAM {
    struct Node {
        int len, link;
        array<int, 26> next{};
        int cnt = 0;
        Node() : len(0), link(0), cnt(0) {}
    };
    vector<Node> tr;
    string s;
    int last;
    bool propagated = false;
    PAM(const string& str) : s(str), last(1) {
        tr.resize(2);
        tr[0].len = -1;
        tr[0].link = 0;
        tr[1].len = 0;
        tr[1].link = 0;
        for(int i = 0; i < int(s.size()); i++) {
            extend(i);
        }
    }
    int extend(int pos) {
        assert(!propagated);
        int c = s[pos] - 'a';
        int cur = last;
```

8.11 回文自动机 PAM · 主体 · 续

```

    while(true) {
        int curlen = tr[cur].len;
        if(pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
            break;
        }
        cur = tr[cur].link;
    }
    if(tr[cur].next[c]) {
        last = tr[cur].next[c];
        tr[last].cnt++;
        return last;
    }
    int now = int(tr.size());
    tr.emplace_back();
    tr[now].len = tr[cur].len + 2;
    tr[cur].next[c] = now;
    tr[now].cnt = 1;
    if(tr[now].len == 1) {
        tr[now].link = 1;
    } else {
        int link = tr[cur].link;
        while(true) {
            int linklen = tr[link].len;
            if(pos - 1 - linklen >= 0 && s[pos - 1 - linklen] == s[pos]) {
                break;
            }
            link = tr[link].link;
        }
        tr[now].link = tr[link].next[c];
    }
    last = now;
    return now;
}

void count() {
    if(propagated) {
        return;
    }
    propagated = true;
    vector<int> order(tr.size());
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(),
        [&](int a, int b) { return tr[a].len > tr[b].len; });
    for(int v : order) {
        if(v >= 2) {
            tr[tr[v].link].cnt += tr[v].cnt;
        }
    }
}

int distinct() const {
    return int(tr.size()) - 2;
}

int size() const {
    return int(tr.size());
}
};

```

回文出现次数的应用

8.11 · 回文出现次数的应用

```
int main() {
    PAM s("ababa");
    s.count();
    long long ans = 0;
    for(int u = 2; u < s.size(); u++) {
        ans = max(ans, 1LL * s.tr[u].len * s.tr[u].cnt);
    }
    assert(s.distinct() == 5 && ans == 6);
}
```

9. 计算几何

9.1 点与基础几何运算

整型坐标优先 `Point<i64>`, `dot/cross/square` 的乘法先提升 `i128`; 加减坐标仍是原类型。建议 $|\text{坐标}| \leq 1e12$, 保证差与后续128乘积范围。排序比较必须严格, 不能在 `operator<` 里随便加 `EPS`; 零向量极角归属按 `polarOrder` 定义。

9.1 点与基础几何运算 · 主体

```
using namespace std;
using db = long double;
template <class T> struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Point& a, const Point& b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};
template <class T> auto dot(const Point<T>& a, const Point<T>& b) {
    using W = conditional_t<is_integral_v<T>, __int128_t, long double>;
    return W(a.x) * b.x + W(a.y) * b.y;
}
template <class T> auto cross(const Point<T>& a, const Point<T>& b) {
```

9.1 点与基础几何运算 · 主体 · 续

```

    using W = conditional_t<is_integral_v<T>, __int128_t, long double>;
    return W(a.x) * b.y - W(a.y) * b.x;
}
template <class T> auto square(const Point<T>& p) {
    return dot(p, p);
}
template <class T> db length(const Point<T>& p) {
    return sqrt(db(square(p)));
}
template <class T> db dis(const Point<T>& a, const Point<T>& b) {
    return length(a - b);
}
template <class T> Point<db> normalize(const Point<T>& p) {
    return Point<db>(db(p.x), db(p.y)) / length(p);
}
template <class T> Point<T> rotate(const Point<T>& a) {
    return Point<T>(-a.y, a.x);
}
template <class T> int sgn(const Point<T>& a) {
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}
// 以 o 为极心的极角排序 (同角按距离近优先), 叉积用平移后的向量。
template <class T> void polarSort(vector<Point<T>>& p, const Point<T>& o = Point<T>()) {
    sort(p.begin(), p.end(), [&](const Point<T>& a, const Point<T>& b) {
        Point<T> ad = a - o;
        Point<T> bd = b - o;
        int sa = sgn(ad);
        int sb = sgn(bd);
        if(sa != sb) {
            return sa > sb;
        }
        auto c = cross(ad, bd);
        if(c != 0) {
            return c > 0;
        }
        return square(ad) < square(bd);
    });
}
template <class T>
vector<int> polarOrder(const vector<Point<T>>& pts, const Point<T>& o = Point<T>()) {
    int n = int(pts.size());
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    sort(p.begin(), p.end(), [&](int i, int j) {
        Point<T> ad = pts[i] - o;
        Point<T> bd = pts[j] - o;
        int sa = sgn(ad);
        int sb = sgn(bd);
        if(sa != sb) {
            return sa > sb;
        }
        auto c = cross(ad, bd);
        if(c != 0) {
            return c > 0;
        }
        return square(ad) < square(bd);
    });
    return p;
}

```

9.2 直线

Line<T>(a,b), $a \neq b$ 才是真直线; onLeft 是严格左侧, onLine 整型精确判定。lineCross 调用前确认不平行; 整型交点转 long double。distancePL 也处理退化点; 浮点共线判定应按题目尺度设置容差。

主体依赖: 9.1 点与基础几何运算。先复制这些基础定义, 再复制下列代码。

9.2 直线 · 主体

```

template <class T> struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T>& a_ = Point<T>(), const Point<T>& b_ = Point<T>()) : a(a_), b(b_) {}
};

template <class T> db length(const Line<T>& l) {
    return length(l.a - l.b);
}

template <class T> bool parallel(const Line<T>& l1, const Line<T>& l2) {
    return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}

template <class T> bool onLeft(const Point<T>& p, const Line<T>& l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

template <class T> bool onLine(const Point<T>& p, const Line<T>& l) {
    return cross(p - l.a, l.b - l.a) == 0;
}

// 整型坐标返回 Point<db>, 浮点坐标返回原类型。
template <class T> auto lineCross(const Line<T>& l1, const Line<T>& l2) {
    using RT = typename conditional<is_integral_v<T>, db, T>::type;
    RT num = RT(cross(l2.b - l2.a, l1.a - l2.a));
    RT den = RT(cross(l2.b - l2.a, l1.a - l1.b));
    RT t = num / den;
    return Point<RT>(RT(l1.a.x) + RT(l1.b.x - l1.a.x) * t,
                    RT(l1.a.y) + RT(l1.b.y - l1.a.y) * t);
}

template <class T> db distancePL(const Point<T>& p, const Line<T>& l) {
    if(l.a == l.b) {
        return length(p - l.a);
    }
    return db(fabs1((long double)cross(l.a - l.b, l.a - p))) / length(l);
}

```

9.3 线段交与距离

segCross 返回 (类型,p,q): 0不交, 1内点相交, 2共线重叠段, 3接触于单点 (含退化线段)。onSeg 含端点; distancePS、distanceSS。整型判交精确, 输出坐标为浮点; 包含竖直/负斜率重叠与点段退化。

主体依赖: 9.1 点与基础几何运算; 9.2 直线。先复制这些基础定义, 再复制下列代码。

9.3 线段交与距离 · 主体

```

template <class T> bool onSeg(const Point<T>& p, const Line<T>& l) {
    return cross(p - l.a, l.b - l.a) == 0 && min(l.a.x, l.b.x) <= p.x &&
           p.x <= max(l.a.x, l.b.x) && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

template <class T> db distancePS(const Point<T>& p, const Line<T>& l) {
    if(dot(p - l.a, l.b - l.a) < 0) {
        return dis(p, l.a);
    }
    if(dot(p - l.b, l.a - l.b) < 0) {
        return dis(p, l.b);
    }
    return distancePL(p, l);
}

template <class T>
tuple<int, Point<db>, Point<db>> segCross(const Line<T>& l1, const Line<T>& l2) {
    auto castPoint = [](const Point<T>& p) { return Point<db>(db(p.x), db(p.y)); };
    auto emptyPoint = []() { return Point<db>(); };
    if(l1.a == l1.b) {
        if(l2.a == l2.b) {
            return l1.a == l2.a
                ? tuple<int, Point<db>, Point<db>>{3, castPoint(l1.a), castPoint(l1.a)}
                : tuple<int, Point<db>, Point<db>>{0, emptyPoint(), emptyPoint()};
        }
    }
}

```

9.3 线段交与距离 · 主体 · 续

```

    }
    return onSeg(l1.a, l2)
        ? tuple<int, Point<db>, Point<db>>>{3, castPoint(l1.a), castPoint(l1.a)}
        : tuple<int, Point<db>, Point<db>>>{0, emptyPoint(), emptyPoint()};
}
if(l2.a == l2.b) {
    return onSeg(l2.a, l1)
        ? tuple<int, Point<db>, Point<db>>>{3, castPoint(l2.a), castPoint(l2.a)}
        : tuple<int, Point<db>, Point<db>>>{0, emptyPoint(), emptyPoint()};
}
if(max(l1.a.x, l1.b.x) < min(l2.a.x, l2.b.x) ||
    min(l1.a.x, l1.b.x) > max(l2.a.x, l2.b.x) ||
    max(l1.a.y, l1.b.y) < min(l2.a.y, l2.b.y) ||
    min(l1.a.y, l1.b.y) > max(l2.a.y, l2.b.y)) {
    return {0, emptyPoint(), emptyPoint()};
}
if(cross(l1.b - l1.a, l2.b - l2.a) == 0) {
    if(cross(l1.b - l1.a, l2.a - l1.a) != 0) {
        return {0, emptyPoint(), emptyPoint()};
    }
    Point<T> p1(max(min(l1.a.x, l1.b.x), min(l2.a.x, l2.b.x)),
                max(min(l1.a.y, l1.b.y), min(l2.a.y, l2.b.y)));
    Point<T> p2(min(max(l1.a.x, l1.b.x), max(l2.a.x, l2.b.x)),
                min(max(l1.a.y, l1.b.y), max(l2.a.y, l2.b.y)));
    if(!onSeg(p1, l1)) {
        swap(p1.y, p2.y);
    }
    if(p1 == p2) {
        return {3, castPoint(p1), castPoint(p2)};
    }
    return {2, castPoint(p1), castPoint(p2)};
}
auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);
if((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) ||
    (cp3 < 0 && cp4 < 0)) {
    return {0, emptyPoint(), emptyPoint()};
}
vector<Point<T>> shared;
for(Point<T> q : {l1.a, l1.b, l2.a, l2.b}) {
    if(onSeg(q, l1) && onSeg(q, l2)) {
        shared.push_back(q);
    }
}
sort(shared.begin(), shared.end());
shared.erase(unique(shared.begin(), shared.end()), shared.end());
if(!shared.empty()) {
    Point<db> p = castPoint(shared[0]);
    return {3, p, p};
}
db num = db(cross(l2.b - l2.a, l1.a - l2.a));
db den = db(cross(l2.b - l2.a, l1.a - l1.b));
db t = num / den;
Point<db> p(db(l1.a.x) + db(l1.b.x - l1.a.x) * t, db(l1.a.y) + db(l1.b.y - l1.a.y) * t);
return {1, p, p};
}
template <class T> db distanceSS(const Line<T>& l1, const Line<T>& l2) {
    if(get<0>(segCross(l1, l2)) != 0) {
        return 0.0;
    }
    return min({distancePS(l1.a, l2), distancePS(l1.b, l2), distancePS(l2.a, l1),
                distancePS(l2.b, l1)});
}

```

9.4 凸包 Andrew

convexHull 去重，返回不重复首点的逆时针严格凸包，共线点只留端点；0/1/2点都处理。O(n log n)。
 polygonArea 返回有向面积，逆时针为正（浮点）；需精确半整数面积时自己累计 i128 又积得到两倍面积。

主体依赖：9.1 点与基础几何运算。先复制这些基础定义，再复制下列代码。

9.4 凸包 Andrew · 主体

```
template <class T> vector<Point<T>> convexHull(vector<Point<T>> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = int(p.size());
    if(n <= 2) {
        return p;
    }
    sort(p.begin(), p.end());
    vector<Point<T>> h;
    for(int i = 0; i < n; i++) {
        while(int(h.size()) >= 2 &&
            cross(h.back() - h[int(h.size()) - 2], p[i] - h.back()) <= 0) {
            h.pop_back();
        }
        h.push_back(p[i]);
    }
    int k = int(h.size());
    for(int i = n - 2; i >= 0; i--) {
        while(int(h.size()) > k &&
            cross(h.back() - h[int(h.size()) - 2], p[i] - h.back()) <= 0) {
            h.pop_back();
        }
        h.push_back(p[i]);
    }
    h.pop_back();
    return h;
}
// 多边形面积（鞋带公式，逆时针为正）。
template <class T> db polygonArea(const vector<Point<T>>& p) {
    db area = 0;
    int n = int(p.size());
    for(int i = 0; i < n; i++) {
        area += db(cross(p[i], p[(i + 1) % n]));
    }
    return area / 2.0;
}
```

9.5 点在简单多边形内

pointInPolygon 包含边界，空多边形 false，O(n)；顶点不必凸，顺/逆时针均可。采用射线奇偶规则，不适用自交多边形面积意义；若要三态，先 onSeg 检测边界再做内部判定。

主体依赖：9.1 点与基础几何运算；9.2 直线；9.3 线段交与距离。先复制这些基础定义，再复制下列代码。

9.5 点在简单多边形内 · 主体

```
template <class T> bool pointInPolygon(const Point<T>& a, const vector<Point<T>>& p) {
    int n = int(p.size());
    for(int i = 0; i < n; i++) {
        if(onSeg(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }
    int t = 0;
    for(int i = 0; i < n; i++) {
        Point<T> u = p[i];
        Point<T> v = p[(i + 1) % n];
        if(u.x < a.x && v.x >= a.x && onLeft(a, Line(v, u))) {
            t ^= 1;
        }
    }
    return t;
}
```

9.5 点在简单多边形内 · 主体 · 续

```

    }
    if(u.x >= a.x && v.x < a.x && onLeft(a, Line(u, v))) {
        t ^= 1;
    }
}
return t == 1;
}

```

9.6 最近点对

`closest2(vector<Point>)` 返回最小平方距离 `i128`, `<2` 点为 `CP_INF`; 整型坐标, 无浮点误差。分治后按 `y` 归并, 条带逐点检查, $O(n \log n)$; 重复点返回 0。此 `Point` 为简洁的非模板独立版本, 别与基础 `Point<T>` 重复定义。

9.6 最近点对 · 主体

```

using namespace std;
struct Point {
    long long x, y;
};
typedef __int128 i128;
const i128 CP_INF = i128(1) << 120;
i128 dist2(Point a, Point b) {
    i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
    return x * x + y * y;
}
i128 closest2(vector<Point> a) {
    sort(a.begin(), a.end(),
        [](Point x, Point y) { return x.x < y.x || (x.x == y.x && x.y < y.y); });
    vector<Point> tmp(a.size());
    function<i128(int, int)> dfs = [&](int l, int r) -> i128 {
        if(r - l <= 3) {
            i128 d = CP_INF;
            for(int i = l; i < r; i++) {
                for(int j = i + 1; j < r; j++) {
                    d = min(d, dist2(a[i], a[j]));
                }
            }
            sort(a.begin() + l, a.begin() + r, [](Point x, Point y) { return x.y < y.y; });
            return d;
        }
        int m = (l + r) / 2;
        long long x = a[m].x;
        i128 d = min(dfs(l, m), dfs(m, r));
        merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, tmp.begin() + l,
            [](Point p, Point q) { return p.y < q.y; });
        copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        vector<Point> s;
        for(int i = l; i < r; i++) {
            i128 dx = i128(a[i].x) - x;
            if(dx * dx >= d) {
                continue;
            }
            for(int j = int(s.size()) - 1; j >= 0; j--) {
                i128 dy = i128(a[i].y) - s[j].y;
                if(dy * dy >= d) {
                    break;
                }
                d = min(d, dist2(a[i], s[j]));
            }
            s.push_back(a[i]);
        }
        return d;
    };
    return dfs(0, a.size());
}

```

9.7 旋转卡壳：凸包直径

diameter2(h) 输入逆时针凸包（不重复首点、去除中间共线点），返回最大平方距离，整型为i128；<2点为0，2点直接计算。先凸包 $O(n \log n)$ ，卡壳 $O(n)$ 。不能直接输入乱序点或凹多边形。

主体依赖：9.1 点与基础几何运算。先复制这些基础定义，再复制下列代码。

9.7 旋转卡壳：凸包直径 · 主体

```
template <class T> auto dist2(const Point<T>& a, const Point<T>& b) {
    using W = conditional_t<is_integral_v<T>, __int128_t, long double>;
    W dx = W(a.x) - b.x, dy = W(a.y) - b.y;
    return dx * dx + dy * dy;
}
template <class T> auto diameter2(const vector<Point<T>>& hull) {
    int n = int(hull.size());
    using W = decltype(dist2(Point<T>(), Point<T>()));
    if(n < 2) {
        return W(0);
    }
    W best = 0;
    int j = 1;
    for(int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
        while(cross(hull[ni] - hull[i], hull[(j + 1) % n] - hull[j]) > 0) {
            j = (j + 1) % n;
        }
        best = max(best, dist2(hull[i], hull[j]));
        best = max(best, dist2(hull[ni], hull[j]));
    }
    return best;
}
```

9.8 外接圆与圆交

独立 long double Point、Circle。circumcircle(a,b,c,out) 共线 false；circleCross 返回0/1/2交点，正半径重合为-1，两个相同零圆为一个点。半径 ≥ 0 ；EPS=1e-12需按坐标尺度调整，接近退化时没有浮点绝对精确保证。

9.8 外接圆与圆交 · 主体

```
using namespace std;
using ld = long double;
const ld EPS = 1e-12;
struct Point {
    ld x, y;
    Point(ld x = 0, ld y = 0) : x(x), y(y) {}
}
Point operator+(Point p) const {
    return {x + p.x, y + p.y};
}
Point operator-(Point p) const {
    return {x - p.x, y - p.y};
}
Point operator*(ld k) const {
    return {x * k, y * k};
}
};
ld dot(Point a, Point b) {
    return a.x * b.x + a.y * b.y;
}
ld cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}
ld dist(Point a, Point b) {
    return hypotl(a.x - b.x, a.y - b.y);
}
struct Circle {
    Point c;
```

9.8 外接圆与圆交 · 主体 · 续

```

    ld r;
};
bool circumcircle(Point a, Point b, Point c, Circle& ans) {
    Point u = b - a, v = c - a;
    ld d = 2 * cross(u, v);
    if(fabs(d) < EPS) {
        return false;
    }
    Point o =
        a + Point(dot(u, u) * v.y - dot(v, v) * u.y, u.x * dot(v, v) - v.x * dot(u, u)) *
            (1 / d);
    ans = {o, dist(o, a)};
    return true;
}
// Return 0/1/2 intersections; -1 means coincident circles with positive radius.
int circleCross(Circle a, Circle b, vector<Point>& ans) {
    assert(a.r >= 0 && b.r >= 0);
    ans.clear();
    Point v = b.c - a.c;
    ld d = hypotl(v.x, v.y);
    if(d < EPS) {
        if(fabs(a.r - b.r) >= EPS) {
            return 0;
        }
        if(a.r < EPS) {
            ans.push_back(a.c);
            return 1;
        }
        return -1;
    }
    if(d > a.r + b.r + EPS || d < fabs(a.r - b.r) - EPS) {
        return 0;
    }
    ld x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
    ld h = sqrtl(max(ld(0), a.r * a.r - x * x));
    Point m = a.c + v * (x / d), u = Point(-v.y, v.x) * (h / d);
    ans.push_back(m + u);
    if(h > EPS) {
        ans.push_back(m - u);
    }
    return ans.size();
}

```

9.9 半平面交

每个有向 Line(a,b) 保留左侧含边界, $a \neq b$ 。halfPlanes 返回逆时针顶点; 适用于有界且正面积的交集, 空返回也包含点/线退化情形。若题目本来有边界, 把真实边界加入; 不要用随意大框把无界问题伪装成有界。角度严格排序, 同向平行留最紧, $O(n \log n)$ 。

9.9 半平面交 · 主体

```

using namespace std;
using ld = long double;
const ld EPS = 1e-12;
struct Point {
    ld x, y;
    Point(ld x = 0, ld y = 0) : x(x), y(y) {}
}
Point operator+(Point p) const {
    return {x + p.x, y + p.y};
}
Point operator-(Point p) const {
    return {x - p.x, y - p.y};
}
Point operator*(ld k) const {

```

9.9 半平面交 · 主体 · 续

```

        return {x * k, y * k};
    }
};
ld cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}
ld dot(Point a, Point b) {
    return a.x * b.x + a.y * b.y;
}
struct Line {
    Point p, v;
    ld ang;
    Line(Point a, Point b) : p(a), v(b - a) {
        ld len = hypotl(v.x, v.y);
        assert(len > 0);
        v = v * (1 / len);
        ang = atan2l(v.y, v.x);
        if(ang < 0) {
            ang += 2 * acosl(-1);
        }
    }
    bool out(Point a) const {
        return cross(v, a - p) < -EPS;
    }
};
Point intersect(Line a, Line b) {
    return a.p + a.v * (cross(b.v, a.p - b.p) / cross(a.v, b.v));
}
// Intersection of closed LEFT half-planes; bounded positive-area case.
vector<Point> halfPlanes(vector<Line> a) {
    sort(a.begin(), a.end(), [](Line x, Line y) { return x.ang < y.ang; });
    vector<Line> b;
    for(Line l : a) {
        if(!b.empty() && fabs1(cross(b.back().v, l.v)) < EPS && dot(b.back().v, l.v) > 0) {
            if(cross(b.back().v, l.p - b.back().p) > 0) {
                b.back() = l;
            }
        } else {
            b.push_back(l);
        }
    }
    deque<Line> q;
    for(Line l : b) {
        while(q.size() > 1 && l.out(intersect(q[q.size() - 2], q.back()))) {
            q.pop_back();
        }
        while(q.size() > 1 && l.out(intersect(q[0], q[1]))) {
            q.pop_front();
        }
        if(!q.empty() && fabs1(cross(q.back().v, l.v)) < EPS) {
            return {};
        }
        q.push_back(l);
    }
    while(q.size() > 2 && q[0].out(intersect(q[q.size() - 2], q.back()))) {
        q.pop_back();
    }
    while(q.size() > 2 && q.back().out(intersect(q[0], q[1]))) {
        q.pop_front();
    }
    if(q.size() < 3 || fabs1(cross(q.front().v, q.back().v)) < EPS) {
        return {};
    }
    vector<Point> p;
    for(int i = 0; i < int(q.size()); i++) {

```

9.9 半平面交 · 主体 · 续

```
p.push_back(intersect(q[i], q[(i + 1) % q.size()]));
}
return p;
}
```

四条真实边界构成矩形

9.9 · 四条真实边界构成矩形

```
int main() {
    vector<Point> p = {{0, 0}, {3, 0}, {3, 2}, {0, 2}};
    vector<Line> a;
    for(int i = 0; i < 4; i++) {
        a.emplace_back(p[i], p[(i + 1) % 4]);
    }
    auto h = halfPlanes(a);
    long double area = 0;
    for(int i = 0; i < int(h.size()); i++) {
        area += cross(h[i], h[(i + 1) % h.size()]);
    }
    assert(fabssl(area / 2 - 6) < 1e-10);
}
```

9.10 Minkowski 和

输入两个逆时针凸包，不重复首点、去除中间共线点；minkowskiSum 返回它们和的凸包。正常多边形 $O(n+m)$ ，点/线段退化情况合并候选点后重做凸包。非凸集合先取凸包将改变原集合，只有求凸包意义时可这样做。

主体依赖：9.1 点与基础几何运算。先复制这些基础定义，再复制下列代码。

9.10 Minkowski 和 · 主体

```
template <class T>
vector<Point<T>> minkowskiSum(const vector<Point<T>>& a, const vector<Point<T>>& b) {
    int n = int(a.size());
    int m = int(b.size());
    if(n == 0 || m == 0) {
        return {};
    }
    if(n <= 2 || m <= 2) {
        vector<Point<T>> p;
        for(auto x : a) {
            for(auto y : b) {
                p.push_back(x + y);
            }
        }
        sort(p.begin(), p.end(),
            [](Point<T> x, Point<T> y) { return x.x < y.x || (x.x == y.x && x.y < y.y); });
        p.erase(unique(p.begin(), p.end()), p.end());
        if(p.size() <= 2) {
            return p;
        }
        vector<Point<T>> h;
        for(auto x : p) {
            while(h.size() >= 2 && cross(h.back() - h[h.size() - 2], x - h.back()) <= 0) {
                h.pop_back();
            }
            h.push_back(x);
        }
        int k = h.size();
        for(int i = int(p.size()) - 2; i >= 0; i--) {
            while(int(h.size()) > k &&
                cross(h.back() - h[h.size() - 2], p[i] - h.back()) <= 0) {
                h.pop_back();
            }
            h.push_back(p[i]);
        }
    }
}
```

9.10 Minkowski 和 · 主体 · 续

```

    }
    h.pop_back();
    return h;
}
auto lowest = [](const vector<Point<T>>& p) {
    int k = 0;
    for(int i = 1; i < int(p.size()); i++) {
        if(p[i].y < p[k].y || (p[i].y == p[k].y && p[i].x < p[k].x)) {
            k = i;
        }
    }
    return k;
};
auto edge = [](const vector<Point<T>>& p, int k) {
    return p[(k + 1) % int(p.size())] - p[k];
};
auto angleLess = [](const Point<T>& u, const Point<T>& v) {
    int su = sgn(u);
    int sv = sgn(v);
    if(su != sv) {
        return su > sv;
    }
    return cross(u, v) > 0;
};
int ia = lowest(a);
int ib = lowest(b);
vector<Point<T>> result;
result.push_back(a[ia] + b[ib]);
int i = 0, j = 0;
while(i < n || j < m) {
    Point<T> ea = i < n ? edge(a, (ia + i) % n) : Point<T>(0, 0);
    Point<T> eb = j < m ? edge(b, (ib + j) % m) : Point<T>(0, 0);
    Point<T> take;
    if(i == n) {
        take = eb;
        j++;
    } else if(j == m) {
        take = ea;
        i++;
    } else if(angleLess(ea, eb)) {
        take = ea;
        i++;
    } else if(angleLess(eb, ea)) {
        take = eb;
        j++;
    } else {
        take = ea + eb;
        i++;
        j++;
    }
    Point<T> nxt = result.back() + take;
    while(int(result.size()) >= 2 &&
        cross(result.back() - result[int(result.size()) - 2], nxt - result.back()) <=
            0) {
        result.pop_back();
    }
    result.push_back(nxt);
}
while(int(result.size()) >= 2 && cross(result.back() - result[int(result.size()) - 2],
    result[0] - result.back()) <= 0) {
    result.pop_back();
}
if(int(result.size()) >= 2 && result.front() == result.back()) {
    result.pop_back();
}
return result;
}

```

9.11 矩形面积并

rectArea 接受 {x1,y1,x2,y2}, 轴平行闭矩形, 零面积忽略, 反向端点归一化。扫描x与压缩y区间长度, $O(n \log n)$, 结果i128。压缩的是端点, 叶子代表 [ys[i],ys[i+1]) 的实际长度; 同x事件间宽度0。

9.11 矩形面积并 · 主体

```
using namespace std;
using i64 = long long;
struct CoverSeg {
    int n;
    const vector<i64>& ys;
    vector<int> cover;
    vector<__int128> len;
    CoverSeg(const vector<i64>& ys_)
        : n(int(ys_.size()) - 1), ys(ys_), cover(4 * max(1, n)), len(4 * max(1, n)) {}
    void pull(int p, int l, int r) {
        if(cover[p]) {
            len[p] = (__int128)ys[r + 1] - ys[l];
        } else if(l == r) {
            len[p] = 0;
        } else {
            len[p] = len[p << 1] + len[p << 1 | 1];
        }
    }
    void add(int p, int l, int r, int x, int y, int v) {
        if(x <= l && r <= y) {
            cover[p] += v;
            pull(p, l, r);
            return;
        }
        int m = (l + r) >> 1;
        if(x <= m) {
            add(p << 1, l, m, x, y, v);
        }
        if(y > m) {
            add(p << 1 | 1, m + 1, r, x, y, v);
        }
        pull(p, l, r);
    }
    void add(int l, int r, int v) {
        if(l <= r) {
            add(1, 0, n - 1, l, r, v);
        }
    }
    __int128 coveredLength() const {
        return n ? len[1] : 0;
    }
};

__int128 rectArea(const vector<array<i64, 4>>& rects) {
    struct Event {
        i64 x, y1, y2;
        int v;
    };
    vector<Event> events;
    vector<i64> ys;
    for(auto [x1, y1, x2, y2] : rects) {
        if(x1 > x2) {
            swap(x1, x2);
        }
        if(y1 > y2) {
            swap(y1, y2);
        }
        if(x1 == x2 || y1 == y2) {
            continue;
        }
    }
}
```

9.11 矩形面积并 · 主体 · 续

```
        events.push_back({x1, y1, y2, 1});
        events.push_back({x2, y1, y2, -1});
        ys.push_back(y1);
        ys.push_back(y2);
    }
    if(events.empty()) {
        return 0;
    }
    sort(events.begin(), events.end(),
        [](const Event& a, const Event& b) { return a.x < b.x; });
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    CoverSeg seg(ys);
    __int128 ans = 0;
    i64 last = events[0].x;
    for(int i = 0, j; i < int(events.size()); i = j) {
        ans += (__int128)seg.coveredLength() * ((__int128)events[i].x - last);
        for(j = i; j < int(events.size()) && events[j].x == events[i].x; j++) {
            int l = int(lower_bound(ys.begin(), ys.end(), events[j].y1) - ys.begin());
            int r = int(lower_bound(ys.begin(), ys.end(), events[j].y2) - ys.begin()) - 1;
            seg.add(l, r, events[j].v);
        }
        last = events[i].x;
    }
    return ans;
}
```

10. 离线算法与技巧

10.1 二分答案边界 [速查]

先证明判定单调并找到有解端点。首个true: $lo < hi$ 时 $mid = lo + (hi - lo) / 2$, true令 $hi = mid$, 否则 $lo = mid + 1$ 。最后true用上中位数 $mid = lo + (hi - lo + 1) / 2$, true令 $lo = mid$, 否则 $hi = mid - 1$ 。整个差值若可超i64, 先转i128; 浮点用固定迭代次数。

10.2 普通莫队: 不同数计数

`moDistinct(a, queries)`, a 是1-based, 可有负值, 内部压缩; 询问为闭区间, 返回按输入0-based序的不同值数。块长约 $\sqrt{cnt(n)}$, 蛇形排序, $O((n+q)\sqrt{n})$ 。带修改/树上路径需额外时间维或欧拉序转换, 本版不包含。

10.2 普通莫队: 不同数计数 · 主体

```
using namespace std;
// a[1..n]. qs are closed intervals; values are compressed internally.
vector<int> moDistinct(vector<int> a, const vector<pair<int, int>>& qs) {
    struct Q {
        int l, r, id;
    };
    int n = a.size() - 1, b = max(1, int(sqrt(max(1, n))));
    vector<Q> v;
    for(size_t i = 0; i < qs.size(); i++) {
        v.push_back(Q{qs[i].first, qs[i].second, int(i)});
    }
    sort(v.begin(), v.end(), [&](Q x, Q y) {
        int p = (x.l - 1) / b, q = (y.l - 1) / b;
        if(p != q) {
            return p < q;
        }
        return p & 1 ? x.r > y.r : x.r < y.r;
    });
    vector<int> values(a.begin() + 1, a.end());
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    int m = values.size();
    for(int i = 1; i <= n; i++) {
        a[i] = lower_bound(values.begin(), values.end(), a[i]) - values.begin();
    }
    vector<int> cnt(m), ans(qs.size());
    int l = 1, r = 0, sum = 0;
    auto add = [&](int i) {
        if(cnt[a[i]]++ == 0) {
            ++sum;
        }
    };
    auto del = [&](int i) {
        if(--cnt[a[i]] == 0) {
            --sum;
        }
    };
    for(size_t i = 0; i < v.size(); i++) {
        Q x = v[i];
        while(l > x.l) {
            add(--l);
        }
        while(r < x.r) {
            add(++r);
        }
        while(l < x.l) {
            del(l++);
        }
        while(r > x.r) {
            del(r--);
        }
        ans[i] = sum;
    }
}
```

10.2 普通莫队：不同数计数 · 主体 · 续

```

    }
    ans[x.id] = sum;
}
return ans;
}

```

10.3 CDQ 三维偏序

dominance(p) 输入0-based三元组，返回每点满足 $x \leq X, y \leq Y, z \leq Z$ 的“其他点数”。重复三元组统一计数，自身扣1；z内部压缩，CDQ 按y归并，BIT回滚， $O(n \log^2 n)$ 。改为严格不等式要同步调整去重与扫序。

10.3 CDQ 三维偏序 · 主体

```

using namespace std;
// For every point, count OTHER points with  $x \leq X, y \leq Y, z \leq Z$ .
vector<long long> dominance(vector<array<int, 3>> p) {
    struct P {
        int x, y, z, c;
        long long ans = 0;
    };
    int n = p.size();
    vector<int> id(n), zs;
    iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int x, int y) { return p[x] < p[y]; });
    for(auto v : p) {
        zs.push_back(v[2]);
    }
    sort(zs.begin(), zs.end());
    zs.erase(unique(zs.begin(), zs.end()), zs.end());
    vector<P> a;
    vector<int> bel(n);
    for(int k : id) {
        int z = lower_bound(zs.begin(), zs.end(), p[k][2]) - zs.begin() + 1;
        if(a.empty() || a.back().x != p[k][0] || a.back().y != p[k][1] || a.back().z != z) {
            a.push_back({p[k][0], p[k][1], z, 0});
        }
        bel[k] = a.size() - 1;
        ++a.back().c;
    }
    // Store the original group index as an additional array, then carry it through merges.
    struct Q {
        P p;
        int id;
    };
    vector<Q> v, buf;
    for(int i = 0; i < int(a.size()); i++) {
        v.push_back({a[i], i});
    }
    buf.resize(v.size());
    vector<int> bit(zs.size() + 1);
    auto add = [&](int x, int k) {
        for(; x < int(bit.size()); x += x & -x) {
            bit[x] += k;
        }
    };
    auto sum = [&](int x) {
        int s = 0;
        for(; x; x -= x & -x) {
            s += bit[x];
        }
        return s;
    };
    function<void(int, int)> dfs = [&](int l, int r) {
        if(r - l <= 1) {
            return;
        }

```

10.3 CDQ 三维偏序 · 主体 · 续

```

int m = (l + r) / 2;
dfs(l, m);
dfs(m, r);
int i = l, j = m, k = 1;
while(j < r) {
    while(i < m && v[i].p.y <= v[j].p.y) {
        add(v[i].p.z, v[i].p.c), buf[k++] = v[i++];
    }
    v[j].p.ans += sum(v[j].p.z);
    buf[k++] = v[j++];
}
for(int t = l; t < i; t++) {
    add(v[t].p.z, -v[t].p.c);
}
while(i < m) {
    buf[k++] = v[i++];
}
copy(buf.begin() + l, buf.begin() + r, v.begin() + l);
};
dfs(0, v.size());
vector<long long> g(a.size()), ans(n);
for(auto q : v) {
    g[q.id] = q.p.ans + q.p.c - 1;
}
for(int i = 0; i < n; i++) {
    ans[i] = g[bel[i]];
}
return ans;
}

```

三维偏序的重复点

10.3 · 三维偏序的重复点

```

int main() {
    auto a = dominance({{1, 1, 1}, {1, 1, 1}, {2, 2, 2}});
    assert(a == vector<long long>({1, 1, 2}));
}

```

10.4 整体二分：动态区间第k小

KthOffline::Event 类型0是位置pos增减d份值x，类型1是[l,r]第k小，id为0-based答案编号；所有事件必须保持时间顺序。初始数组用n次+1插入，修改先-1旧值再+1新值；每个询问k合法。solve(n,events,q) 返回值， $O((n+q)\log V \log n)$ 。

递归按值域中点划分，按时间序扫小值修改并用BIT数询问区间内小值；不足k就送右边且k减去小值数。扫描结束必须回滚BIT；事件分区要稳定，不能排序打乱历史顺序。

10.4 整体二分：动态区间第k小 · 主体

```

using namespace std;
struct KthOffline {
    // type=0: add d copies of value x at pos. type=1: kth query on [l,r], answer id.
    struct Event {
        int type, pos, x, d, l, r, k, id;
    };
    vector<Event> a, buf;
    vector<int> bit, v, ans, side;
    int n;
    void add(int x, int d) {
        for(; x <= n; x += x & -x) {
            bit[x] += d;
        }
    }
    int sum(int x) {

```

10.4 整体二分：动态区间第k小 · 主体 · 续

```

    int s = 0;
    for(; x; x -= x & -x) {
        s += bit[x];
    }
    return s;
}

void dfs(int l, int r, int L, int R) {
    if(l == r) {
        return;
    }
    if(L == R) {
        for(int i = l; i < r; i++) {
            if(a[i].type == 1) {
                ans[a[i].id] = v[L];
            }
        }
        return;
    }
    int m = (L + R) / 2, cnt = 0;
    for(int i = l; i < r; i++) {
        Event& e = a[i];
        side[i] = 0;
        if(e.type == 0) {
            if(e.x <= m) {
                add(e.pos, e.d), side[i] = 1;
            }
        } else {
            int s = sum(e.r) - sum(e.l - 1);
            if(e.k <= s) {
                side[i] = 1;
            } else {
                e.k -= s;
            }
        }
        cnt += side[i];
    }
    for(int i = l; i < r; i++) {
        if(a[i].type == 0 && side[i]) {
            add(a[i].pos, -a[i].d);
        }
    }
    int x = l, y = l + cnt;
    for(int i = l; i < r; i++) {
        buf[side[i] ? x++ : y++] = a[i];
    }
    copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
    dfs(l, l + cnt, L, m);
    dfs(l + cnt, r, m + 1, R);
}

vector<int> solve(int n_, vector<Event> e, int q) {
    n = n_;
    a = move(e);
    v.clear();
    for(Event e : a) {
        if(!e.type) {
            v.push_back(e.x);
        }
    }
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    for(Event& e : a) {
        if(!e.type) {
            e.x = lower_bound(v.begin(), v.end(), e.x) - v.begin();
        }
    }
    buf.resize(a.size());
    side.resize(a.size());
}

```

10.4 整体二分：动态区间第k小 · 主体 · 续

```

        bit.assign(n + 1, 0);
        ans.assign(q, 0);
        if(!a.empty()) {
            assert(!v.empty());
            dfs(0, a.size(), 0, v.size() - 1);
        }
        return ans;
    }
};

```

修改数组后的区间第 k 小

10.4 · 修改数组后的区间第 k 小

```

int main() {
    using E = KthOffline::Event;
    vector<E> e;
    e.push_back({0, 1, 5, 1, 0, 0, 0, 0});
    e.push_back({0, 2, 1, 1, 0, 0, 0, 0});
    e.push_back({1, 0, 0, 0, 1, 2, 1, 0});
    e.push_back({0, 2, 1, -1, 0, 0, 0, 0});
    e.push_back({0, 2, 7, 1, 0, 0, 0, 0});
    e.push_back({1, 0, 0, 0, 1, 2, 1, 1});
    auto a = KthOffline().solve(2, e, 2);
    assert(a == vector<int>({1, 5}));
}

```

10.5 时间线段树 + 回滚 DSU

OfflineConn(n,q), 时间1..q。add(l,r,u,v) 表示边在闭区间有效；solve(ask) 每个时刻一对点，返回1-based连通性。插入时刻t、删除时刻d的边有效段[t,d-1]；未删除延伸到q，重边维护独立次数/生命周期。O(m logq logn+q logn)。

DFS进入时间节点记snapshot，合并覆盖该时间段的边，左右子树结束后恢复snapshot；不能路径压缩，否则撤销记录不够。

10.5 时间线段树 + 回滚 DSU · 主体

```

using namespace std;
struct OfflineConn {
    int n, q;
    vector<int> fa, sz;
    vector<array<int, 3>> stk;
    vector<vector<pair<int, int>>> seg;
    OfflineConn(int n_, int q_)
        : n(n_), q(q_), fa(n_ + 1, 1), sz(n_ + 1, 1), seg(4 * max(1, q_)) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {
        while(x != fa[x]) {
            x = fa[x];
        }
        return x;
    }
    void merge(int u, int v) {
        u = find(u);
        v = find(v);
        if(u == v) {
            return;
        }
        if(sz[u] < sz[v]) {
            swap(u, v);
        }
        stk.push_back({{u, v, sz[u]}});
        fa[v] = u;
        sz[u] += sz[v];
    }
}

```

10.5 时间线段树 + 回滚 DSU · 主体 · 续

```

void undo(int s) {
    while((int)stk.size() > s) {
        array<int, 3> a = stk.back();
        stk.pop_back();
        fa[a[1]] = a[1];
        sz[a[0]] = a[2];
    }
}

void put(int p, int l, int r, int x, int y, pair<int, int> e) {
    if(x <= l && r <= y) {
        seg[p].push_back(e);
        return;
    }
    int m = (l + r) / 2;
    if(x <= m) {
        put(p * 2, l, m, x, y, e);
    }
    if(y > m) {
        put(p * 2 + 1, m + 1, r, x, y, e);
    }
}

void add(int l, int r, int u, int v) {
    if(l <= r) {
        put(1, 1, q, l, r, make_pair(u, v));
    }
}

void dfs(int p, int l, int r, const vector<pair<int, int>>& ask, vector<int>& ans) {
    int s = stk.size();
    for(size_t i = 0; i < seg[p].size(); i++) {
        merge(seg[p][i].first, seg[p][i].second);
    }
    if(l == r) {
        ans[l] = (find(ask[l].first) == find(ask[l].second));
    } else {
        int m = (l + r) / 2;
        dfs(p * 2, l, m, ask, ans);
        dfs(p * 2 + 1, m + 1, r, ask, ans);
    }
    undo(s);
}

vector<int> solve(const vector<pair<int, int>>& ask) {
    vector<int> ans(q + 1);
    if(q) {
        dfs(1, 1, q, ask, ans);
    }
    return ans;
}
};

```

边的生存时间

10.5 · 边的生存时间

```

int main() {
    OfflineConn s(3, 4);
    s.add(1, 2, 1, 2); // 时刻3删去, 所以截止2。
    s.add(2, 4, 2, 3);
    vector<pair<int, int>> ask = {{0, 0}, {1, 3}, {1, 3}, {1, 3}, {2, 3}};
    auto a = s.solve(ask);
    assert(a == vector<int>({0, 0, 1, 0, 1}));
}

```

10.6 根号分块 [速查]

块长 B , 整块信息处理 $O(n/B)$, 两侧散块 $O(B)$ 。lazy与块聚合必须兼容; 赋值/加法的组合顺序明确。 $B \approx \sqrt{n}$ 只在两侧代价相近时合理, 按更新查询比例与常数调节。

10.7 图上重轻点〔速查〕

按度数阈值B：轻点遍历邻接 $O(B)$ ，重点个数 $O(m/B)$ 。预存重点相关贡献，修改时只更新关联重点；是否能做到这些取决于询问形式，不能把树的HLD用于一般图。

10.8 时间分块〔速查〕

一批B次修改先记录，静态部分预处理，询问补偿这批改变；批末重建。总成本常为 $q \cdot B + (q/B) \cdot \text{重建成本}$ ，取平衡点；补偿必须基于“最后一次修改”去重。

10.9 倒序离线〔速查〕

只有删边的连通性：先建最终图，倒序删除变加边，用DSU。混合增删或询问历史版本时不能直接倒序解决，改用时间线段树+回滚。

10.10 双指针〔速查〕

维护区间满足性质，左右指针各至多前进n次，需要扩展/缩小对可行性有单调影响。含负数的区间和 $\leq K$ 一般不满足；删除元素时频次和答案要精确回退。

10.11 折半搜索〔速查〕

把 $n \approx 40$ 的子集分两半枚举，约 $O(2^{(n/2)})$ 空间时间，再排序二分/双指针配对。不要枚举两边笛卡尔积；重复和的保留方式取决于求最优还是计数。

10.12 坐标压缩〔速查〕

排序去重后 $\text{rank} = \text{lower_bound} + 1$ ，保留原值数组。只保大小关系不保距离；长度、面积、差值必须用原坐标。若离线结构随后还出现新值，提前把它们一起收集。

10.13 扫描线〔速查〕

把几何问题变成一维事件；相同坐标的进入、离开、询问顺序对应闭/开区间约定。面积用相邻坐标差乘当前覆盖长度；只查点数时可用BIT，不必上覆盖长度树。

10.14 Log Trick〔速查〕

固定右端点的所有子段 gcd/按位AND/OR可把相同值合并，携带最早下标/数量。gcd正值严格变化至少减半，AND/OR每次改变至少一位，状态数为 $O(\log A)$ 或位数；sum不满足这个性质。

10.15 前缀与差分〔速查〕

1-based区间和 $\text{pre}[r] - \text{pre}[l-1]$ ；闭区间加： $d[l] += v, d[r+1] -= v$ ，数组须开 $n+2$ 。二维包含-排除四项；树上路径用前述树差分，不能拿一维下标差分代替。

10.16 贡献法〔速查〕

把“枚举所有对象”改为每个元素/事件对答案的贡献，明确对象是否有序、是否重叠。动态维护时先撤旧贡献，再改状态，再加新贡献；边界与自身项必须单独考虑。

10.17 有限群状态〔速查〕

前缀状态若有逆元，区间状态由 $\text{pre}[l-1]$ 的逆与 $\text{pre}[r]$ 组合；非交换群要注意乘法顺序。仅有结合律的么半群没有一般“前缀相减”，需要分治/线段树。

10.18 容斥〔速查〕

并集大小 $= \sum \text{非空 } S \cdot (-1)^{|S|+1} |\cap S|$ ；小集合数直接 2^k 枚举。整除约束交集用 lcm 并在超过上界时停止，乘法先除 gcd 再用 i128 检查；相同条件需去重防重复。

10.19 反悔贪心〔速查〕

维护已选集合，加入新项后用堆删除最差项恢复约束。必须证明可交换性与局部替换不损失全局最优，常用于截止时间/容量排序后的选择；不是“遇到冲突删最大”就自动成立。

10.20 上次出现位置〔速查〕

维护 $\text{last}[\text{value}]$ 可把区间不同值数离线转为 BIT：扫 r ，在旧 last 减 1、新 r 加 1，询问 $[l, r]$ 的和。需按 r 排序，压缩值后存 last ；这是完整公式，泛化到其他约束需重新推导。

10.21 二进制对齐区间〔速查〕

将 $[l, r]$ 分成互不交的 2 次幂对齐块，块数 $O(\log \text{范围})$ 。取块长要同时满足左端对齐和不越过 r ； $l=0$ 时 $\text{lowbit}=0$ 须特别处理，64 位全域端点常需 u128。

10.22 相邻交换排序〔速查〕

比较 AB 与 BA 代价证明局部交换顺序；比较器还须满足严格弱序，不能出现循环偏好。乘积比较用 i128，平值返回 false/稳定下标；把证明中的 $\leq$ 直接写比较器会出错。

10.23 区间与点匹配〔速查〕

最大数量匹配：按点坐标升序，加入左端 $\leq$ 点的区间，把右端 $<$ 点的过期项弹掉，匹配最小右端。 $O((n+m)\log n)$ ，点/区间各只能使用一次；加权目标不保证此贪心。

10.24 可行性判定构造〔速查〕

若可行性判定能处理已固定决策，逐位尝试最小候选并保留可完成的一项，可得字典序最小方案。需要 q 次判定，复杂度 $= q \cdot \text{判定代价}$ ；不能只检查局部约束而忽略未来。

10.25 加权中位数〔速查〕

非负权重，最小化 $\sum w_i |x_i - x|$ ；累积权首次达到总权一半附近即最优区间。总权 0 时任意点最优；偶数/平分可有整个区间，权值前缀用 i128 避免溢出。不同于最小平方误差的均值。

10.26 固定半径空间哈希〔速查〕

把平面分格，格长与搜索半径匹配，只查附近常数格；负坐标整除需向下取整。格内点可能很密，最坏仍退化到二次；没有密度/分离条件不能承诺 $O(n)$ 。

10.27 L1 坐标变换〔速查〕

$u=x+y, v=x-y$, 则曼哈顿距离 $=\max(|\Delta u|, |\Delta v|)$, 二维直径可由 u/v 极值得到。逆变换 $x=(u+v)/2, y=(u-v)/2$, 整数点需奇偶一致; 加减先用足够宽类型。

10.28 预留坐标〔速查〕

离线收集所有会使用的位置、端点和必要的相邻哨兵, 排序压缩后再执行。是否加 ± 1 取决于离散点或连续长度意义, 不能无条件加; 极值处防整数溢出。

10.29 区间取模剪枝〔速查〕

非负 a , 取模 $x>0$ 时若区间 $\max<x$ 无需递归; 有效变化后值至少减半 ($a>=x$ 时 $a\%x<a/2$), 因此每点有效变化 $O(\log A)$ 。配合单点赋值时按每次新值重新计摊还; 不支持负数同样的证明。

11. 公式与博弈

11.1 常用恒等式〔速查〕

$\Sigma i = n(n+1)/2$; $\Sigma i^2 = n(n+1)(2n+1)/6$; $\Sigma (i=0..n) C(n,i) = 2^n$; $\Sigma_k C(n,k) k = n 2^{n-1}$ 。乘除在整数里先约分或用 $i128$ ；模数非素数时 2、6 未必可逆。

11.2 SG / Nim〔速查〕

有限、无环的正常规则 impartial game: $SG(x) = \text{mex}\{SG(\text{next})\}$ ，独立游戏和取 xor；Nim 堆大小即 SG。终态 $SG=0$ 。反常规则（最后操作者输）或有环不直接适用；递归记忆化先保证状态图无环，规则决定转移。

复核与参考

本版逐项重写了说明，使下标、区间、返回值与实际代码一致；通用模板保留完整实现，12个较窄的题意相关条目移出打印正文，但修订后的内容仍在完整源稿和逐项记录中。

修正包括：GNU++17不兼容接口、欧拉路仅框架、朴素匹配、SAM次数与clone约定、深链递归、流的自环反向编号、上下界流的人工边处理、FPS截断混叠、空串Z、Manacher哨兵冲突、凸包重复点、线性基“无答案”值冲突、几何整数乘法宽度。

运行日志与源码：tests/revised/compile.json、verification.json、examples.json、printed.json。固定随机种子712367；暴力参照包括枚举匹配/流/生成树/子集，数组直接维护，朴素字符串匹配与几何枚举。

主要参考的是原资料与下列维护者文档，用于核对算法条件；此册代码与教程已按本地接口重新整理。查阅日期2026-10-02。

OI Wiki · 最小生成树 / Boruvka

<https://oi-wiki.org/graph/mst/>

OI Wiki · 差分约束

<https://oi-wiki.org/graph/diff-constraints/>

OI Wiki · 二分图匹配

<https://oi-wiki.org/graph/graph-matching/bigraph-match/>

OI Wiki · 后缀自动机

<https://oi-wiki.org/string/sam/>

OI Wiki · 杜教筛

<https://oi-wiki.org/math/number-theory/hyperbola/>

OI Wiki · Min_25 筛

<https://oi-wiki.org/math/number-theory/min-25/>